

- LỜI CAM ĐOAN
- TÓM TẮT ĐỒ ÁN TỐT NGHIỆP - ABSTRACT
- LỜI CẢM ƠN - ACKNOWLEDGEMENTS
- MỤC LỤC - TABLE OF CONTENT
- DANH MỤC BẢNG - LIST OF TABLES
- DANH MỤC HÌNH ẢNH VÀ ĐỒ THỊ - LIST OF PICTURES AND GRAPHS
- DANH MỤC TỪ VIẾT TẮT - LIST OF ABBREVIATION
- CHƯƠNG 1. GIỚI THIỆU DỰ ÁN – PROJECT OVERVIEW
 - 1.1. Đặt vấn đề / Bối cảnh của dự án
 - 1.1.1. Bối cảnh thị trường cho thuê xe tự lái tại Việt Nam
 - 1.1.2. Thách thức kỹ thuật
 - 1.1.3. Động lực thực hiện dự án
 - 1.2. Mục tiêu và phạm vi của dự án
 - 1.2.1. Mục tiêu tổng quát
 - 1.2.2. Phạm vi dự án
 - 1.3. Các tiêu chí cần đạt được của dự án
 - 1.3.1. Tiêu chí phần cứng
 - 1.3.2. Tiêu chí phần mềm thiết bị nhúng (firmware)
 - 1.3.3. Tiêu chí máy chủ và dịch vụ xử lý dữ liệu
 - 1.3.4. Tiêu chí giao diện web
 - 1.4. Phương pháp tiếp cận thiết kế kỹ thuật
 - 1.4.1. Phương pháp luận tổng thể
 - 1.4.2. Kiến trúc hệ thống
 - 1.4.3. Chiến lược quản lý năng lượng
 - 1.4.4. Công nghệ sử dụng
 - 1.5. Kết quả và khuyến nghị
 - 1.5.1. Kết quả đạt được
 - 1.5.2. Khuyến nghị và hướng phát triển
- CHƯƠNG 2. PHÂN TÍCH VẤN ĐỀ KỸ THUẬT
 - 2.1. Mô tả vấn đề - Problem statement
 - 2.1.1. Bối cảnh vận hành
 - 2.1.2. Các vấn đề kỹ thuật chính
 - 2.1.3. Mục tiêu kỹ thuật của chương
 - 2.2. Bối cảnh và cơ sở kỹ thuật - Background and Technical reviews
 - 2.2.1. Các hướng quản lý phương tiện hiện có
 - 2.2.2. Đặc điểm kỹ thuật cần xét
 - 2.2.3. Yêu cầu rút ra từ cơ sở kỹ thuật

- 2.3. Yêu cầu kỹ thuật và các tiêu chuẩn thiết kế - Design criteria and Constraints
 - 2.3.1. Yêu cầu chức năng
 - 2.3.2. Yêu cầu phi chức năng
 - 2.3.3. Ràng buộc thiết kế
 - 2.3.4. Tiêu chuẩn và tài liệu kỹ thuật áp dụng
 - 2.3.5. Chỉ tiêu nghiệm thu định lượng
- 2.4. Yêu cầu từ các bên liên quan - Stakeholder requirements
 - 2.4.1. Các bên liên quan
 - 2.4.2. Ma trận liên hệ yêu cầu - bên liên quan
- CHƯƠNG 3. CÁC GIẢI PHÁP THIẾT KẾ - DESIGN SOLUTIONS
 - 3.1. Phân tích tổng hợp và nguyên tắc lựa chọn - General analysis
 - 3.1.1. Chuỗi chức năng của hệ thống
 - 3.1.2. Các mâu thuẫn thiết kế chính
 - 3.1.3. Tiêu chí đánh giá theo khối
 - 3.2. Đề xuất và lựa chọn giải pháp phần cứng trên xe - Hardware solution
 - 3.2.1. Lựa chọn vi điều khiển trung tâm
 - 3.2.2. Lựa chọn mô-đun truyền dữ liệu và định vị
 - 3.2.3. Lựa chọn phương án đọc dữ liệu OBD2
 - 3.2.4. Lựa chọn cảm biến phát hiện chuyển động
 - 3.2.5. Lựa chọn kiến trúc nguồn và bảo vệ
 - 3.2.6. Tổng hợp phần cứng được chọn
 - 3.3. Đề xuất và lựa chọn giải pháp firmware - Firmware solution
 - 3.3.1. Lựa chọn nền tảng phát triển firmware
 - 3.3.2. Mô hình điều phối trạng thái thiết bị
 - 3.3.3. Các nhóm chức năng chính trong firmware
 - 3.3.4. Kết luận lựa chọn firmware
 - 3.4. Đề xuất và lựa chọn giải pháp kết nối giữa thiết bị và máy chủ
 - 3.4.1. So sánh giao thức truyền dữ liệu thiết bị - máy chủ
 - 3.4.2. Tổ chức luồng bản tin được chọn
 - 3.5. Đề xuất và lựa chọn giải pháp phía máy chủ
 - 3.5.1. Lựa chọn mô hình tổ chức hạ tầng máy chủ
 - 3.5.2. Lựa chọn MQTT broker
 - 3.5.3. Lựa chọn lớp lưu trữ và quan sát vận hành
 - 3.5.4. Lựa chọn MQTT Bridge
 - 3.5.5. Lựa chọn backend
 - 3.5.6. Lựa chọn frontend
 - 3.5.7. Lựa chọn cách đóng gói và triển khai các dịch vụ

- 3.5.8. Đánh giá tổng hợp và lựa chọn phương án khả thi
 - 3.5.9. Phương án thiết kế tối ưu
- CHƯƠNG 4. TRIỂN KHAI GIẢI PHÁP VÀ KẾT QUẢ - IMPLEMENTATION AND RESULTS
 - 4.1. Thiết kế chi tiết giải pháp - Detailed design solution
 - 4.1.1. Thiết kế thiết bị trên xe
 - 4.1.2. Thiết kế firmware
 - 4.1.3. Thiết kế máy chủ và giao diện web quản lý
 - 4.2. Chế tạo, tích hợp và lắp ráp hệ thống - Manufacture and Assembly
 - 4.3. Đo lường và kết quả - Measurements and Results
 - 4.3.1. Thiết lập môi trường thử nghiệm
 - 4.3.2. Kết quả kiểm thử nguồn và phần cứng
 - 4.3.3. Kết quả kiểm thử firmware và dữ liệu thiết bị
 - 4.3.4. Kết quả kiểm thử máy chủ và giao diện web quản lý
 - 4.3.5. Đối chiếu với chỉ tiêu nghiệm thu
 - 4.3.6. Kết luận chương 4
- CHƯƠNG 5. ĐÁNH GIÁ VÀ KHUYẾN NGHỊ - EVALUATION AND RECOMMENDATIONS
 - 5.1. Đánh giá hiệu năng
 - 5.1.1. Đánh giá chung
 - 5.1.2. Nhận xét theo yêu cầu thiết kế
 - 5.2. Đánh giá kinh tế và môi trường
 - 5.2.1. Đánh giá kinh tế
 - 5.2.2. Đánh giá môi trường
 - 5.3. Đánh giá rủi ro và biện pháp giảm thiểu
 - 5.4. Khuyến nghị cho tương lai
- CHƯƠNG 6. PHẢN HỒI VÀ BÀI HỌC KINH NGHIỆM - REFLECTION AND LESSONS LEARNED
 - 6.1. Ứng dụng kiến thức kỹ thuật - Application of prior coursework
 - 6.2. Giải quyết các vấn đề kỹ thuật phức tạp - Complex engineering problems
 - 6.3. Tác động đạo đức và xã hội - Ethical and social impacts
 - 6.4. Tổng kết và bài học kinh nghiệm - Reflection and Lessons Learned
- TÀI LIỆU TRÍCH DẪN - REFERENCES
- TÀI LIỆU TRÍCH DẪN - REFERENCES
- TÀI LIỆU TRÍCH DẪN - REFERENCES
- TÀI LIỆU TRÍCH DẪN - REFERENCES
 - [A] Tài liệu tiếng Việt

- [B] Tài liệu tiếng Anh (Books & Papers)
- [C] Tài liệu kỹ thuật (Datasheets & Technical Documents)
- [D] Tài liệu trực tuyến (Online Resources)
- PHỤ LỤC - APPENDICES
- PHỤ LỤC 1: BÁO CÁO TÀI CHÍNH - FINANCE REPORT
 - 1.1. Bảng kê chi phí linh kiện (Bill of Materials - BOM)
 - 1.2. Chi phí hạ tầng đám mây (ước tính hàng tháng)
 - 1.3. Tổng hợp chi phí dự án
- PHỤ LỤC 2: CÁC TIÊU CHUẨN THIẾT KẾ - STANDARDS
 - 2.1. Tiêu chuẩn OBD2 (SAE J1979 / ISO 15031–5)
 - 2.2. MQTT 3.1.1 trong triển khai thiết bị
 - 2.3. Bảo mật thông tin (ISO 27001 - tham khảo)
 - 2.4. Thiết kế REST API (RFC 7231 và thực hành tốt)
 - 2.5. Ma trận kiểm soát tuân thủ chuẩn theo thành phần
- PHỤ LỤC 3: KẾ HOẠCH THỰC HIỆN - ASSIGNMENT AND TIMELINES
 - 3.1. Phân chia giai đoạn dự án
 - 3.2. Kế hoạch thực hiện dự án theo giai đoạn
 - 3.3. Chi tiết công việc theo giai đoạn
 - 3.4. Kế hoạch thực hiện theo tuần (24 tuần)
- PHỤ LỤC 4: TÀI LIỆU BỔ TRỢ - SUPPORTING MATERIALS
 - 4.1. Hồ sơ thiết kế và mã nguồn
 - 4.2. Tóm tắt nhóm API sử dụng trong nguyên mẫu
 - 4.3. Checklist kiểm tra nguyên mẫu
 - 4.4. Ghi chú vận hành thử nghiệm

\

BỘ GIÁO DỤC VÀ ĐÀO TẠO ĐẠI HỌC PHENIKAA

ĐỒ ÁN TỐT NGHIỆP THIẾT KẾ HỆ THỐNG IoT CHO ỨNG DỤNG QUẢN LÝ PHƯƠNG TIỆN GIAO THÔNG TRONG LĨNH VỰC CHO THUÊ XE TỰ LÁI

Sinh viên: Lê Trọng An **Mã số sinh viên:** 21010389 **Khóa:** K15 **Ngành:** Kỹ thuật Cơ Điện tử **Hệ:** Đại học chính quy **Giảng viên hướng dẫn:** TS. Nguyễn Đức Nam

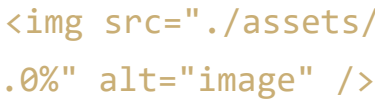
Hà Nội - 2026



ĐỒ ÁN TỐT NGHIỆP THIẾT KẾ HỆ THỐNG IoT CHO ỨNG DỤNG QUẢN LÝ PHƯƠNG TIỆN GIAO THÔNG TRONG LĨNH VỰC CHO THUÊ XE TỰ LÁI

Sinh viên: Lê Trọng An **Mã số sinh viên:** 21010389 **Khóa:** K15 **Ngành:** Kỹ thuật Cơ Điện tử **Hệ:** Đại học chính quy **Giảng viên hướng dẫn:** TS. Nguyễn Đức Nam

Hà Nội – 2026

PHENIKAA UNIVERSITY FACULTY OF MECHANICAL ENGINEERING AND MECHATRONICS 

MEM710071 – CAPSTONE DESIGN 2 DESIGN OF AN IoT SYSTEM FOR VEHICLE MANAGEMENT IN SELF-DRIVE CAR RENTAL SERVICES Semester: I Academic year: 2025–2026

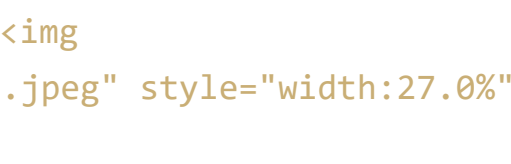
Project Team:

Student name	Student ID	Cohort
Le Trong An	21010389	K15

Course Instructor: Dr. Nguyen Duc Nam

This report is submitted to the Faculty of Mechanical Engineering and Mechatronics, School of Engineering, Phenikaa University in partial fulfillment of the requirements of the Graduation Thesis course MEM710071.

HANOI – 2026

ĐẠI HỌC PHENIKAA KHOA CƠ KHÍ – CƠ ĐIỆN TỬ 

MEM710071 ĐỒ ÁN TỐT NGHIỆP THIẾT KẾ HỆ THỐNG IoT CHO ỨNG DỤNG QUẢN LÝ PHƯƠNG TIỆN GIAO THÔNG TRONG LĨNH VỰC CHO THUÊ XE TỰ LÁI
Học kỳ I Năm học 2025–2026

Họ và tên sinh viên Mã số sinh viên Lớp

Lê Trọng An 21010389 K15-KTCĐT2

Giảng viên hướng dẫn: TS. Nguyễn Đức Nam

HÀ NỘI, 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO ĐẠI HỌC PHENIKAA\

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM Độc lập - Tự do - Hạnh phúc\

NHẬN XÉT ĐỒ ÁN TỐT NGHIỆP CỦA GIẢNG VIÊN HƯỚNG DẪN

Giảng viên hướng dẫn: TS. Nguyễn Đức Nam Khoa: Cơ khí – Cơ Điện tử. Tên đề tài: Thiết kế hệ thống IoT cho ứng dụng quản lý phương tiện giao thông trong lĩnh vực cho thuê xe tự lái. Sinh viên thực hiện: Lê Trọng An. Lớp: K15-KTCĐT2.

NỘI DUNG NHẬN XÉT

I. Nhận xét Đồ án tốt nghiệp:\

- Nhận xét về hình thức:\
- Tính cấp thiết của đề tài:\
- Mục tiêu của đề tài:\
- Nội dung của đề tài:\
- Tài liệu tham khảo:\
- Phương pháp nghiên cứu:\

Tính sáng tạo và ứng dụng: II. Nhận xét về tinh thần và thái độ làm việc của sinh viên: III. Kết quả đạt được: IV. Kết luận: Đồng ý cho bảo vệ: ☐ Không đồng ý cho bảo vệ: ☐

Hà Nội, ngày ..., tháng ... năm 20... GIẢNG VIÊN HƯỚNG DẪN (Ký, ghi rõ họ tên)
TS. Nguyễn Đức Nam

BỘ GIÁO DỤC VÀ ĐÀO TẠO ĐẠI HỌC PHENIKAA\

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM Độc lập - Tự do - Hạnh phúc\

NHẬN XÉT ĐỒ ÁN TỐT NGHIỆP CỦA GIẢNG VIÊN PHẢN BIỆN

Giảng viên phản biện: Theo quyết định phân công phản biện của Khoa **Tên đề tài:** Thiết kế hệ thống IoT cho ứng dụng quản lý phương tiện giao thông trong lĩnh vực cho thuê xe tự lái. **Sinh viên thực hiện:** Lê Trọng An **Lớp:** K15-KTCĐT2 **Giảng viên hướng dẫn:** TS. Nguyễn Đức Nam

NỘI DUNG NHẬN XÉT

I. Nhận xét Đồ án tốt nghiệp:

- Bố cục, hình thức trình bày:
- Đảm bảo tính cấp thiết, hiện đại, không trùng lặp:
- Nội dung:
- Mức độ thực hiện: **II. Kết quả đạt được:** **III. Ưu nhược điểm:** **IV. Kết luận:** Đồng ý cho bảo vệ: '☐' Không đồng ý cho bảo vệ: '☐'

Hà Nội, ngày ..., tháng ... năm 20... **GIẢNG VIÊN PHẢN BIỆN** (Ký, ghi rõ họ tên)
Theo quyết định phân công phản biện của Khoa

BỘ GIÁO DỤC VÀ ĐÀO TẠO ĐẠI HỌC PHENIKAA

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM Độc lập - Tự do - Hạnh phúc

BIÊN BẢN ĐÁNH GIÁ ĐỒ ÁN TỐT NGHIỆP

I. Thông tin chung

- Thời gian: từ ...giờ ...đến ...giờ ...ngày .../.../20...
- Địa điểm:
- Họ và tên sinh viên: Lê Trọng An Mã số SV: 21010389
- Lớp: K15-KTCĐT2 Ngành: Kỹ thuật Cơ Điện tử Khóa: K15
- Tên đề tài: Thiết kế hệ thống IoT cho ứng dụng quản lý phương tiện giao thông trong lĩnh vực cho thuê xe tự lái.
- Giảng viên hướng dẫn: TS. Nguyễn Đức Nam

II. Thành phần hội đồng

- Chủ tịch
- Thư ký
- Giảng viên phản biện
- Ủy viên 1

III. Tổng hợp câu hỏi của thành viên hội đồng:

IV. Tổng hợp nội dung trả lời của sinh viên:

V. Nội dung đánh giá của Hội đồng

- Ý nghĩa của đề án:
- Về nội dung, kết cấu của đề án:
- Phương pháp nghiên cứu:
- Các kết quả nghiên cứu đạt được:

VI. Điểm kết luận: (Bằng chữ:)

CHỦ TỊCH HỘI ĐỒNG (Ký, ghi rõ họ tên)\

THƯ KÝ (Ký, ghi rõ họ tên)\

GIẢI TRÌNH CÁC CHỈNH SỬA (Nếu có)

Chưa phát sinh nội dung cần giải trình.

LỜI CAM ĐOAN

Tên tôi là: Lê Trọng An. Mã sinh viên: 21010389 Lớp: K15-KTCĐT2 Ngành: Kỹ thuật Cơ Điện tử.

Tôi đã thực hiện đề án tốt nghiệp với đề tài: Thiết kế hệ thống IoT cho ứng dụng quản lý phương tiện giao thông trong lĩnh vực cho thuê xe tự lái.

Tôi xin cam đoan đây là đề tài nghiên cứu của riêng tôi và được sự hướng dẫn của TS. Nguyễn Đức Nam.

Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa được công bố dưới bất kỳ hình thức nào. Nếu phát hiện có bất kỳ hình thức gian lận nào, tôi xin hoàn toàn chịu trách nhiệm trước pháp luật.

Hà Nội, ngày ..., tháng ... năm 20...

GIẢNG VIÊN HƯỚNG DẪN (Ký, ghi rõ họ tên) TS. Nguyễn Đức Nam

SINH VIÊN (Ký, ghi rõ họ tên) Lê Trọng An

TÓM TẮT ĐỒ ÁN TỐT NGHIỆP - ABSTRACT

Sự phát triển nhanh của dịch vụ cho thuê xe tự lái tại Việt Nam làm gia tăng nhu cầu giám sát phương tiện từ xa. Nhu cầu này không chỉ dừng ở việc xác định vị trí xe, mà còn bao gồm theo dõi trạng thái vận hành, nhận biết sớm các dấu hiệu bất thường và hỗ trợ người quản lý đưa ra quyết định kịp thời trong quá trình vận hành đội xe. Từ yêu cầu đó, đồ án tập trung thiết kế và triển khai một hệ thống giám sát phương tiện hoàn chỉnh theo kiến trúc tích hợp, gồm thiết bị gắn trên xe, khối tiếp nhận và xử lý dữ liệu trên máy chủ, cùng giao diện theo dõi dành cho người quản lý.

Ở tầng thiết bị, hệ thống được xây dựng trên bo mạch PCB tự thiết kế. Thiết bị sử dụng ESP32-S3 làm bộ điều khiển trung tâm, kết hợp modem SIM7600CE-T để truyền dữ liệu qua mạng di động và lấy vị trí từ vệ tinh, cảm biến gia tốc LIS3DH để nhận biết chuyển động khi xe đỗ, cùng bộ chuyển đổi OBD2 qua BLE để thu thập các thông tin chẩn đoán từ xe. Kiến trúc nguồn được tổ chức theo nhiều nhánh để thiết bị vẫn duy trì hoạt động khi xe tắt máy, đồng thời giảm nguy cơ ảnh hưởng tới ắc quy chính.

Ở tầng máy chủ, dữ liệu gửi từ thiết bị được tiếp nhận, phân loại và chuẩn hóa trước khi đưa vào các lớp lưu trữ và xử lý phù hợp. Cách tổ chức này giúp hệ thống vừa theo dõi được dữ liệu mới theo thời gian thực, vừa lưu lại lịch sử vận hành, trạng thái thiết bị, cảnh báo và tiến trình cập nhật phần mềm từ xa để phục vụ quản trị về sau.

Ở tầng ứng dụng, hệ thống cung cấp một bảng điều khiển web để người quản lý theo dõi vị trí xe, trạng thái thiết bị, lịch sử hành trình và các cảnh báo trên cùng một giao diện thống nhất. Khi dữ liệu trên máy chủ có thay đổi, giao diện được cập nhật gần như theo thời gian thực, nhờ đó người vận hành không phải thao tác tải lại thủ công mà vẫn theo dõi được diễn biến của đội xe.

Kết quả đạt được là một hệ thống IoT giám sát phương tiện hoàn chỉnh từ phần cứng đến phần mềm, có khả năng theo dõi vị trí thời gian thực, đọc dữ liệu chẩn đoán OBD2, thiết lập vùng cho phép, phát sinh cảnh báo tự động, cập nhật firmware từ xa và hỗ trợ quản lý đội xe. Trong bối cảnh đó, hệ thống cho thấy một hướng triển khai phù hợp với doanh nghiệp cho thuê xe tự lái trong nước: chi phí hợp lý, có thể tùy biến và còn có khả năng mở rộng trong các giai đoạn tiếp theo.

Từ khóa: IoT, giám sát phương tiện, GPS, OBD2, MQTT, ESP32-S3, thời gian thực, vùng cho phép

Abstract (English)

As the self-drive car rental market in Vietnam grows, the need for remote vehicle monitoring and real-time fleet management becomes increasingly practical for operators. This thesis presents the design and implementation of an integrated vehicle-monitoring system with three main layers: an onboard device that collects and transmits data, a server-side processing layer that receives and organizes that data, and a web-based dashboard that helps operators follow the fleet in daily operation.

At the device layer, the system is built around a custom PCB rather than a simple combination of development boards. The onboard unit uses an ESP32-S3 as the main controller, a SIM7600CE-T module for mobile-data communication and GNSS positioning, a LIS3DH accelerometer to detect movement when the vehicle is parked, and an external BLE-based OBD2 adapter to read diagnostic data from the vehicle. Its power architecture is split into multiple power paths so that the device can continue operating when the engine is off while reducing the risk of draining the main battery.

At the server layer, incoming device messages are first received, classified, and normalized before being sent to the appropriate storage and processing paths. This organization allows the system to support both real-time monitoring and longer-term history review, while also keeping track of device state, alerts, and remote firmware-update progress.

At the application layer, the system provides a web dashboard for monitoring vehicle position, device status, trip history, and alerts in a unified interface. When new data arrives, the dashboard is updated almost in real time, allowing operators to follow fleet activity without manual refresh actions.

The result is a complete end-to-end IoT vehicle tracking system, from hardware to software, capable of real-time location tracking, OBD2 diagnostic data retrieval, allowed-zone monitoring, automated alerts, remote firmware updates, and fleet-management support for self-drive car rental services. In this context, the system demonstrates a practical implementation approach for domestic self-drive rental operators: cost conscious, adaptable, and open to further expansion in later stages.

Keywords: IoT, vehicle tracking, GPS, OBD2, MQTT, ESP32-S3, real-time, allowed-zone monitoring

LỜI CẢM ƠN - ACKNOWLEDGEMENTS

Trước tiên, tôi xin gửi lời cảm ơn chân thành và sâu sắc nhất đến TS. Nguyễn Đức Nam — người đã trực tiếp hướng dẫn, định hướng và tận tình hỗ trợ tôi trong suốt quá trình thực hiện đồ án tốt nghiệp này. Những góp ý chuyên môn và sự động viên của thầy là nguồn động lực quan trọng giúp tôi hoàn thành đề tài.

Tôi xin trân trọng cảm ơn quý thầy cô trong Khoa Cơ khí - Cơ Điện tử, Trường Kỹ thuật - Đại học Phenikaa đã truyền đạt những kiến thức nền tảng vững chắc trong suốt quá trình học tập, tạo điều kiện thuận lợi để tôi có thể áp dụng vào thực tiễn thông qua đồ án này.

Tôi cũng xin gửi lời cảm ơn đến gia đình, bạn bè và các đồng nghiệp đã luôn ủng hộ, chia sẻ và đồng hành cùng tôi trong suốt thời gian thực hiện đề tài.

Mặc dù đã nỗ lực hoàn thiện đồ án trong phạm vi thời gian và kiến thức cho phép, nội dung chắc chắn vẫn còn những thiếu sót. Tôi rất mong nhận được những ý kiến góp ý từ quý thầy cô và hội đồng để tiếp tục hoàn thiện đề tài.

Xin chân thành cảm ơn!

Hà Nội, ngày ... tháng ... năm 20... **SINH VIÊN THỰC HIỆN** Lê Trọng An

MỤC LỤC - TABLE OF CONTENT

DANH MỤC BẢNG - LIST OF TABLES

DANH MỤC HÌNH ẢNH VÀ ĐỒ THỊ - LIST OF PICTURES AND GRAPHS

DANH MỤC TỪ VIẾT TẮT - LIST OF ABBREVIATION

Từ viết tắt	Tiếng Anh	Tiếng Việt
ADC	Analog-to-Digital Converter	Bộ chuyển đổi tương tự sang số

Từ viết tắt	Tiếng Anh	Tiếng Việt
API	Application Programming Interface	Giao diện lập trình ứng dụng
BLE	Bluetooth Low Energy	Bluetooth năng lượng thấp
BOM	Bill of Materials	Danh sách linh kiện
CI/CD	Continuous Integration / Continuous Deployment	Tích hợp liên tục / Triển khai liên tục
CORS	Cross-Origin Resource Sharing	Cơ chế chia sẻ tài nguyên khác nguồn
DDD	Domain-Driven Design	Thiết kế hướng miền
Docker	Docker Container Platform	Nền tảng đóng gói và chạy ứng dụng bằng container
EMQX	Erlang MQTT Broker	Broker MQTT xây dựng trên Erlang/OTP
ESP32	Espressif System-on-Chip 32-bit	Hệ thống trên chip 32-bit của Espressif
FreeRTOS	Free Real-Time Operating System	Hệ điều hành thời gian thực mã nguồn mở
GPIO	General Purpose Input/Output	Cổng vào/ra đa năng
GNSS	Global Navigation Satellite System	Hệ thống vệ tinh dẫn đường toàn cầu
GPS	Global Positioning System	Hệ thống định vị toàn cầu
HTTP	Hypertext Transfer Protocol	Giao thức truyền siêu văn bản
HTTPS	Hypertext Transfer Protocol Secure	Giao thức truyền siêu văn bản bảo mật
IMU	Inertial Measurement Unit	Đơn vị đo lường quán tính
IoT	Internet of Things	Internet vạn vật
JWT	JSON Web Token	Mã thông báo web định dạng JSON
MCU	Microcontroller Unit	Vi điều khiển

Từ viết tắt	Tiếng Anh	Tiếng Việt
MQTT	Message Queuing Telemetry Transport	Giao thức truyền bản tin nhẹ theo mô hình publish/subscribe
OBD2	On-Board Diagnostics II	Chẩn đoán trên xe thế hệ 2
ORM	Object-Relational Mapping	Ánh xạ đối tượng - quan hệ
PCB	Printed Circuit Board	Bảng mạch in
REST	Representational State Transfer	Kiểu kiến trúc REST cho dịch vụ web
RTOS	Real-Time Operating System	Hệ điều hành thời gian thực
SoC	System on Chip	Hệ thống trên chip
SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc
TLS	Transport Layer Security	Bảo mật tầng vận chuyển
UART	Universal Asynchronous Receiver-Transmitter	Bộ thu phát bất đồng bộ đa năng
WebSocket	WebSocket Protocol	Giao thức giao tiếp hai chiều thời gian thực

CHƯƠNG 1. GIỚI THIỆU DỰ ÁN – PROJECT OVERVIEW

1.1. Đặt vấn đề / Bối cảnh của dự án

1.1.1. Bối cảnh thị trường cho thuê xe tự lái tại Việt Nam

Những năm gần đây, thị trường cho thuê xe tại Việt Nam tăng trưởng rõ rệt, đặc biệt tại TP. Hồ Chí Minh, Hà Nội và Đà Nẵng. Theo một báo cáo thị trường, quy mô thị trường cho thuê xe Việt Nam được ước tính đạt 376,4 triệu USD năm 2025 và có thể đạt 1.006,7 triệu USD vào năm 2034, tương ứng tốc độ tăng trưởng kép 11,55% trong

giai đoạn 2026–2034 [1]. Xu hướng này làm tăng nhu cầu đối với các hệ thống giám sát và quản lý phương tiện từ xa.

Với doanh nghiệp vận hành đội xe, tốc độ tăng trưởng này mang lại cơ hội kinh doanh nhưng cũng kéo theo một yêu cầu rất thực tế: phải biết xe đang ở đâu, đang được sử dụng như thế nào, có an toàn hay không, và tình trạng vận hành của từng xe đang diễn biến ra sao. Trên thực tế, nhiều đơn vị hiện nay không hoàn toàn quản lý xe theo cách thủ công, mà đã sử dụng các thiết bị định vị GPS hoặc các giải pháp giám sát thương mại có sẵn trên thị trường. Tuy nhiên, khi quy mô đội xe mở rộng, những công cụ này vẫn bộc lộ các giới hạn rõ rệt về mức độ tập trung dữ liệu, khả năng quan sát trạng thái kỹ thuật và mức độ phù hợp với nhu cầu vận hành đặc thù của doanh nghiệp cho thuê xe tự lái. Trong thực tế, các doanh nghiệp cho thuê xe tự lái hiện nay thường gặp bốn nhóm khó khăn sau:

- **Dữ liệu giám sát còn rời rạc và thiếu tập trung:** Nhiều doanh nghiệp đã gắn thiết bị GPS hoặc dùng nền tảng giám sát sẵn có, nhưng thông tin thu được thường chỉ dừng ở vị trí, tốc độ hoặc lịch sử di chuyển cơ bản. Dữ liệu vận hành vì thế nằm rải rác giữa nhiều công cụ khác nhau và chưa hình thành được một góc nhìn tập trung để người quản lý theo dõi toàn bộ đội xe trên cùng một hệ thống.
- **Khó kiểm soát việc sử dụng xe theo ngữ cảnh vận hành thực tế:** Khi hệ thống hiện có chủ yếu phục vụ định vị, doanh nghiệp vẫn khó phát hiện sớm các tình huống như xe đi vượt phạm vi cho phép, sử dụng sai mục đích, hoạt động bất thường ngoài thời gian dự kiến hoặc có dấu hiệu bị chiếm đoạt. Việc nhận biết các tình huống này thường chậm hơn nhu cầu can thiệp thực tế, làm tăng rủi ro về tài sản và an toàn vận hành [3], [8], [9].
- **Thiếu khả năng theo dõi trạng thái kỹ thuật từ xa:** Phần lớn các giải pháp định vị thương mại không đi sâu vào dữ liệu vận hành của xe, nên doanh nghiệp khó theo dõi tình trạng kỹ thuật từ xa và thường chỉ xử lý bảo trì khi sự cố đã biểu hiện rõ. Điều này làm tăng chi phí sửa chữa, giảm tính chủ động trong bảo trì và có thể rút ngắn tuổi thọ phương tiện. Để đáp ứng nhu cầu này, hệ thống cần kết nối được với xe qua cổng OBD2 (On-Board Diagnostics II) — giao diện chẩn đoán tiêu chuẩn trên hầu hết xe ô tô sản xuất từ năm 1996 trở về sau, cho phép đọc các thông số vận hành như tốc độ động cơ, mức tiêu thụ nhiên liệu, nhiệt độ làm mát, điện áp acquy và mã lỗi chẩn đoán.
- **Khó tùy biến và chi phí tích lũy cao khi mở rộng:** Các hệ thống giám sát thương mại hiện có trên thị trường thường đi kèm chi phí thiết bị, phí dịch vụ định kỳ và những ràng buộc nhất định về nền tảng. Khi doanh nghiệp muốn tích hợp

sâu hơn với quy trình quản lý riêng, bổ sung cảnh báo theo nghiệp vụ, hoặc mở rộng theo quy mô đội xe, tổng chi phí và độ phụ thuộc vào nhà cung cấp có thể tăng lên đáng kể.

Những khó khăn trên cho thấy nhu cầu giám sát là rất rõ, nhưng chưa tự động dẫn tới một lời giải khả thi. Muốn biến nhu cầu vận hành thành một hệ thống thật sự dùng được, bài toán phải được chuyển thành các thách thức kỹ thuật cụ thể.

1.1.2. Thách thức kỹ thuật

Để giải quyết những khó khăn trên, hệ thống không thể chỉ dừng ở một thiết bị định vị gắn thêm lên xe. Khi chuyển từ nhu cầu biết vị trí sang nhu cầu quản lý vận hành, bài toán lập tức mở rộng thành bài toán hệ thống nhiều tầng: thiết bị phải bám được nguồn xe trong thời gian dài, vẫn nhận biết được bất thường khi xe đỗ, chịu được điều kiện kết nối di động biến động, và bảo đảm dữ liệu khi đi tới máy chủ vẫn đủ nhất quán để người quản lý sử dụng trong thực tế.

Vì vậy, thách thức kỹ thuật cốt lõi của đề tài không nằm ở một chức năng đơn lẻ, mà ở việc tổ chức được toàn bộ chuỗi từ thu thập, truyền thông, xử lý đến khai thác thông tin sao cho phù hợp với môi trường vận hành thực tế của đội xe. Các nhóm vấn đề này sẽ được phân tích chi tiết ở Chương 2.

1.1.3. Động lực thực hiện dự án

Động lực thực hiện đề tài xuất phát từ nhu cầu thực tế của doanh nghiệp cho thuê xe tự lái: không chỉ cần biết vị trí phương tiện, mà còn cần quan sát được trạng thái vận hành, phát hiện bất thường đủ sớm và có cơ sở dữ liệu để hỗ trợ quyết định trong quá trình khai thác đội xe. Trong bối cảnh đó, các công cụ theo dõi hiện có thường mới giải quyết tốt bài toán định vị, trong khi nhu cầu quản lý lại đòi hỏi một chuỗi xử lý đầy đủ hơn, từ thu thập dữ liệu trên xe, truyền dữ liệu ổn định, lưu trữ và xử lý tập trung, đến hiển thị thông tin theo cách dễ theo dõi đối với người vận hành.

Từ yêu cầu đó, đề tài **Thiết kế hệ thống IoT cho ứng dụng quản lý phương tiện giao thông trong lĩnh vực cho thuê xe tự lái** được thực hiện với định hướng xây dựng một hệ thống đồng bộ gồm thiết bị phần cứng trên xe, phần mềm thiết bị nhúng, hạ tầng máy chủ và giao diện khai thác thông tin. Cách tiếp cận này phù hợp hơn với đặc thù của bài toán quản lý đội xe, đồng thời tạo cơ sở để phát triển một giải pháp có khả năng mở rộng, tùy biến và thích nghi với điều kiện triển khai thực tế tại Việt Nam.

1.2. Mục tiêu và phạm vi của dự án

1.2.1. Mục tiêu tổng quát

Mục tiêu chính của dự án là thiết kế và triển khai một hệ thống IoT theo dõi phương tiện hoàn chỉnh cho bài toán quản lý đội xe cho thuê tự lái. Mục tiêu này bao gồm cả thiết bị phần cứng trên xe, phần mềm thiết bị nhúng, hạ tầng tiếp nhận và xử lý dữ liệu, cùng các lớp ứng dụng phục vụ người vận hành. Từ yêu cầu của thực tế khai thác đội xe, hệ thống cần vừa thu thập được dữ liệu hữu ích, vừa duy trì được độ ổn định khi lắp đặt lâu dài trên xe, đồng thời bảo đảm dữ liệu được xử lý và đưa tới người dùng đủ nhanh để hỗ trợ ra quyết định. Trên cơ sở đó, các mục tiêu cụ thể của dự án được xác định như sau:

1. **Theo dõi vị trí và hành trình của phương tiện:** Hình thành khả năng quan sát đội xe theo thời gian thực và truy xuất lại lịch sử vận hành khi cần.
2. **Bổ sung lớp dữ liệu trạng thái vận hành cơ bản của xe:** Mở rộng từ bài toán định vị sang bài toán theo dõi trạng thái kỹ thuật ở mức đủ dùng cho quản lý đội xe.
3. **Duy trì giám sát ổn định trên xe trong nhiều trạng thái vận hành:** Hệ thống phải hoạt động được khi xe chạy, khi xe đỗ và khi điều kiện kết nối thay đổi.
4. **Phát hiện và chuyển tải được các tình huống bất thường:** Cảnh báo phải được hình thành đủ kịp thời để hỗ trợ can thiệp ở cấp vận hành.
5. **Xây dựng nền tảng xử lý và khai thác dữ liệu tập trung:** Dữ liệu từ xe cần được tiếp nhận, lưu trữ, xử lý và hiển thị theo một chuỗi đủ nhất quán cho người quản lý sử dụng.

Các mục tiêu trên xác lập đích đến ở cấp đề tài; các yêu cầu kỹ thuật và ngưỡng đánh giá tương ứng được chuẩn hóa ở Chương 2.

1.2.2. Phạm vi dự án

Để bảo đảm phạm vi nghiên cứu tập trung, nội dung triển khai được xác định theo từng tầng của hệ thống: đối tượng ứng dụng, thiết bị phần cứng, phần mềm thiết bị nhúng, hạ tầng máy chủ và lớp ứng dụng khai thác. Cách phân chia này làm rõ ranh

giới giữa phần được thiết kế, chế tạo và kiểm chứng trực tiếp với các nội dung chỉ dừng ở mức chức năng cơ bản hoặc định hướng phát triển tiếp theo.

Đối tượng ứng dụng:

Đề tài tập trung vào nhóm ô tô con sử dụng trong dịch vụ cho thuê xe tự lái, như sedan hoặc SUV cỡ nhỏ, với hệ thống điện 12V hoặc 24V và dải tần phổ thông thường gặp trên thị trường. Nhóm người dùng hướng tới là các doanh nghiệp cho thuê xe tự lái quy mô vừa và nhỏ tại Việt Nam, nơi nhu cầu theo dõi tập trung, chi phí đầu tư và khả năng tùy biến là những yếu tố được đặt lên hàng đầu.

Phạm vi phần cứng thiết bị:

Ở tầng thiết bị, đồ án tập trung xây dựng một bộ theo dõi gắn trên xe dưới dạng bo mạch riêng, gồm khối điều khiển trung tâm, khối truyền dữ liệu qua mạng di động và định vị vệ tinh, khối đọc dữ liệu chẩn đoán từ xe, khối cảm biến chuyển động và khối nguồn dự phòng. Phạm vi này đủ để thiết bị thực hiện được chuỗi chức năng cốt lõi: thu thập dữ liệu, nhận biết trạng thái, truyền thông tin đi và duy trì hoạt động khi nguồn chính bị gián đoạn trong thời gian ngắn.

Phạm vi phần mềm thiết bị nhúng:

Phần mềm thiết bị nhúng là lớp làm cho phần cứng có thể vận hành thực sự trên xe. Trong phạm vi đồ án, lớp này bao gồm điều phối các khối thu thập dữ liệu, giao tiếp với thiết bị đọc OBD2, quản lý kết nối truyền dữ liệu, đóng gói bản tin gửi lên máy chủ và tổ chức các chế độ năng lượng tương ứng với trạng thái chạy, đỗ hoặc cảnh báo. Thuật ngữ *firmware* trong đồ án được hiểu theo nghĩa đó: phần mềm chạy trực tiếp trên thiết bị để điều khiển hành vi của hệ thống ở tầng xe.

Phạm vi máy chủ và dịch vụ xử lý dữ liệu:

Ở phía máy chủ, đồ án tập trung vào chuỗi chức năng cần thiết để dữ liệu sau khi rời xe có thể tiếp tục được tiếp nhận, phân loại, lưu trữ và xử lý phục vụ khai thác. Chuỗi này bao gồm lớp tiếp nhận bản tin, lớp xử lý và phân luồng dữ liệu, lớp lưu trữ cho dữ liệu nghiệp vụ, dữ liệu chuỗi thời gian và nhật ký vận hành, cùng lớp dịch vụ cung cấp giao diện lập trình ứng dụng cho web và di động. Phạm vi này được lựa chọn nhằm bảo đảm hệ thống có thể xử lý dữ liệu với độ trễ thấp, lưu giữ trong thời gian đủ dài và tránh hình thành các điểm nghẽn ngay từ giai đoạn đầu.

Phạm vi ứng dụng khai thác:

Giao diện web là bề mặt khai thác chính của hệ thống trong đồ án, phục vụ theo dõi vị trí, lịch sử hành trình, cảnh báo và trạng thái thiết bị trên một bảng điều khiển tập trung. Bên cạnh đó, ứng dụng di động cũng được đưa vào phạm vi đề tài ở mức chức năng cơ bản, chủ yếu phục vụ theo dõi nhanh vị trí và trạng thái phương tiện, nhận thông tin cảnh báo và truy cập các dữ liệu vận hành thiết yếu.

Giới hạn phạm vi nghiên cứu:

- Đề tài chưa đi sâu vào các chức năng chẩn đoán OBD2 chuyên sâu; phạm vi hiện tại ưu tiên các thông số vận hành cơ bản và một phần mã lỗi chẩn đoán.
- Bài toán phát hiện va chạm chưa phải trọng tâm chính; cảm biến chuyển động hiện được dùng chủ yếu để nhận biết rung động và trạng thái bất thường khi xe đang đỗ.
- Hệ thống chưa triển khai các cơ chế điều khiển tác động trực tiếp tới xe như khóa máy hoặc can thiệp động cơ; chức năng hiện tại dừng ở mức phát hiện, cảnh báo và hỗ trợ ra quyết định.
- Ứng dụng di động mới được triển khai ở phạm vi cơ bản, chưa bao quát toàn bộ nghiệp vụ quản trị như trên giao diện web.

1.3. Các tiêu chí cần đạt được của dự án

Sau khi xác định mục tiêu và phạm vi, dự án cần một bộ tiêu chí ở cấp định hướng để đối chiếu các quyết định thiết kế ở các chương sau. Trong Chương 1, các tiêu chí này chỉ đóng vai trò khung tham chiếu tổng quát cho toàn bộ đề tài; các yêu cầu định lượng và ràng buộc kỹ thuật chi tiết sẽ được chuyển sang Chương 2 để tránh chồng lặp giữa phần giới thiệu và phần phân tích vấn đề.

1.3.1. Tiêu chí phần cứng

Ở tầng phần cứng, thiết bị cần đáp ứng bốn yêu cầu định hướng:

- Làm việc ổn định khi lắp đặt lâu dài trên xe và không gây ảnh hưởng bất lợi tới hệ thống điện của phương tiện.
- Chịu được các điều kiện vận hành thường gặp như rung động, dao động điện áp và môi trường nhiệt độ thực tế.
- Có cơ chế duy trì hoạt động hoặc chuyển trạng thái phù hợp khi nguồn chính suy giảm hoặc bị gián đoạn ngắn.

- Đủ gọn và đủ thực dụng để có thể lắp đặt trên xe thật, thay vì chỉ phù hợp trong mô hình thử nghiệm.

1.3.2. Tiêu chí phần mềm thiết bị nhúng (firmware)

Ở tầng firmware, tiêu chí cốt lõi là khả năng điều phối thiết bị theo đúng bối cảnh vận hành:

- Thu thập, chuẩn hóa và đóng gói được dữ liệu cần thiết từ các khối trên xe.
- Quản lý được nhiều trạng thái làm việc thay vì vận hành thiết bị theo một chế độ cố định.
- Phản ứng được với tín hiệu bất thường ở mức phù hợp với phạm vi đề tài.
- Có khả năng phục hồi sau gián đoạn kết nối và hạn chế làm mất dữ liệu quan trọng.

1.3.3. Tiêu chí máy chủ và dịch vụ xử lý dữ liệu

Ở phía máy chủ và xử lý dữ liệu, hệ thống cần:

- Tiếp nhận và xử lý dữ liệu liên tục với độ trễ đủ thấp cho giám sát vận hành.
- Tổ chức dữ liệu theo bản chất sử dụng để thuận lợi cho lưu trữ, truy vấn và theo dõi trạng thái hệ thống.
- Cung cấp lớp dịch vụ đủ ổn định cho web, di động và các chức năng quản trị cốt lõi.
- Duy trì cơ chế xác thực, phân quyền và quan sát hệ thống ở mức phù hợp với một nguyên mẫu có khả năng triển khai thử nghiệm.

1.3.4. Tiêu chí giao diện web

Giao diện web là nơi chuyển kết quả xử lý kỹ thuật thành công cụ quan sát và ra quyết định. Vì vậy, tiêu chí định hướng ở đây là:

- Hiển thị đủ rõ vị trí, trạng thái thiết bị, hành trình và cảnh báo trên cùng một bề mặt khai thác.

- Cập nhật thông tin đủ nhanh để người vận hành không bị lệch khỏi trạng thái thực tế của đội xe.
- Ưu tiên tính dễ theo dõi, dễ tra cứu và phù hợp với bối cảnh điều hành nhiều phương tiện cùng lúc.

Các tiêu chí trên tạo thành bộ tham chiếu chung cho toàn bộ đồ án. Các giá trị kỹ thuật cụ thể dùng để so sánh, ràng buộc và đánh giá phương án sẽ được chuẩn hóa ở Chương 2.

1.4. Phương pháp tiếp cận thiết kế kỹ thuật

1.4.1. Phương pháp luận tổng thể

Đề tài bao gồm nhiều lớp kỹ thuật gắn chặt với nhau: thiết bị trên xe, phần mềm thiết bị nhúng, truyền thông dữ liệu, hạ tầng máy chủ và lớp ứng dụng. Vì vậy, cách tiếp cận của đồ án không phải tối ưu từng phần riêng lẻ rồi mới ghép lại ở cuối, mà kiểm chứng từng lớp trong mối liên hệ với toàn hệ thống.

Trên cơ sở đó, dự án được triển khai theo hai nguyên tắc. Thứ nhất là **thiết kế từ dưới lên (bottom-up design)**: kiểm chứng trước những giả định rủi ro nhất ở tầng thiết bị rồi mới mở rộng dần lên các tầng phía trên. Thứ hai là **phát triển lặp (iterative development)**: thực hiện theo từng vòng nhỏ, có đo đạc và điều chỉnh liên tục để phát hiện sớm các xung đột về nguồn, kết nối, lưu trữ và hiệu năng. Tiến trình được tổ chức như sau:

- Giai đoạn 1 — Nghiên cứu và thiết kế phần cứng**: Khảo sát linh kiện, đối chiếu datasheet, thiết kế sơ đồ nguyên lý và tính toán nguồn, nhằm tạo được một nền tảng phần cứng đủ ổn định để lắp trên xe.
- Giai đoạn 2 — Phát triển firmware**: Xây dựng phần mềm cho thiết bị để thu thập dữ liệu vận hành cần thiết, điều phối các khối ngoại vi, quản lý năng lượng và gửi dữ liệu lên máy chủ theo đúng trạng thái làm việc của xe.
- Giai đoạn 3 — Xây dựng hạ tầng máy chủ**: Tổ chức lớp tiếp nhận bản tin, lớp lưu trữ và lớp nhật ký vận hành để dữ liệu sau khi rời xe có thể được tiếp nhận và xử lý liên tục.

4. **Giai đoạn 4 — Phát triển API máy chủ:** Chuyển dữ liệu kỹ thuật thành các chức năng quản trị như đăng nhập, quản lý xe, xem lịch sử, xử lý cảnh báo và cung cấp dữ liệu thời gian thực cho lớp ứng dụng.
5. **Giai đoạn 5 — Phát triển giao diện web:** Trình bày dữ liệu dưới dạng bản đồ, bảng thông tin và biểu đồ để người vận hành quan sát và thao tác thuận tiện.
6. **Giai đoạn 6 — Tích hợp và kiểm thử:** Ghép toàn bộ các lớp lại thành một chuỗi hoàn chỉnh từ xe đến giao diện web quản lý, sau đó kiểm thử, đo đạc và tối ưu các điểm còn nghẽn.

Quy trình phát triển dự án theo các giai đoạn

1.4.2. Kiến trúc hệ thống

Kiến trúc hệ thống được tổ chức theo ba lớp chức năng nối tiếp nhau từ xe đến người quản lý.

- **Lớp trên xe:** thu thập dữ liệu vị trí, trạng thái vận hành và các tín hiệu bất thường; thực hiện tiền xử lý cần thiết rồi gửi dữ liệu ra ngoài.
- **Lớp máy chủ:** tiếp nhận bản tin, phân loại, lưu trữ và xử lý dữ liệu; đồng thời cung cấp các dịch vụ phục vụ tra cứu, cảnh báo và quản trị.
- **Lớp ứng dụng:** chuyển dữ liệu đã xử lý thành bản đồ, lịch sử hành trình, trạng thái thiết bị và các công cụ theo dõi để người quản lý sử dụng.

Việc phân lớp xác định rõ trách nhiệm của từng phần và giữ luồng dữ liệu theo một chuỗi nhất quán từ thu thập, xử lý đến khai thác thông tin. Trong phạm vi đề án, ứng dụng di động chỉ được triển khai ở mức chức năng cơ bản; trọng tâm vẫn là chuỗi chính từ thiết bị đến giao diện web quản lý.

Kiến trúc tổng thể hệ thống IoT Vehicle Tracking

Ngay từ giai đoạn kiến trúc, dữ liệu cũng được phân biệt theo ba nhóm: nghiệp vụ, chuỗi thời gian và nhật ký. Việc phân nhóm này nhằm làm rõ nhu cầu lưu trữ và xử lý cho từng loại dữ liệu; phần phân tích chi tiết và cơ sở lựa chọn công nghệ sẽ được trình bày ở Chương 2 và Chương 3.

1.4.3. Chiến lược quản lý năng lượng

Quản lý năng lượng của thiết bị là bài toán cân bằng giữa khả năng giám sát và việc bảo vệ ắc quy xe. Nếu ưu tiên theo dõi liên tục, hệ thống dễ tiêu thụ điện lớn; nếu tiết

kiệm quá mức, thiết bị có thể bỏ lỡ sự kiện quan trọng. Vì vậy, đồ án định hướng tiếp cận theo nhiều trạng thái làm việc, trong đó mỗi trạng thái chỉ duy trì những khối thật sự cần thiết cho bối cảnh chạy xe, đỗ xe hoặc phát hiện bất thường.

Song song với đó, hệ thống cần có cơ chế bảo vệ điện áp thấp và nguồn dự phòng để tránh rút cạn ắc quy xe trong quá trình bám nguồn dài hạn. Cách tiếp cận này được xác lập ở mức nguyên tắc trong Chương 1; các tham số và cơ chế triển khai cụ thể sẽ được làm rõ ở các chương sau.

Sơ đồ chuyển đổi giữa các chế độ năng lượng

1.4.4. Công nghệ sử dụng

Ở mức tổng quan, hệ thống sử dụng một tập công nghệ trải từ thiết bị trên xe, phần mềm thiết bị nhúng, hạ tầng máy chủ, lưu trữ dữ liệu đến giao diện khai thác. Việc lựa chọn công nghệ được định hướng bởi bốn yêu cầu: phù hợp với môi trường xe, hỗ trợ truyền thông thời gian thực, tách được các loại dữ liệu khác nhau và thuận lợi cho triển khai thử nghiệm.

- **Tầng thiết bị:** vi điều khiển trung tâm, khối định vị và truyền dữ liệu di động, kết nối đọc dữ liệu xe, cảm biến chuyển động và nguồn dự phòng.
- **Tầng firmware:** framework nhúng và cơ chế điều phối tác vụ thời gian thực để điều khiển hành vi của thiết bị.
- **Tầng máy chủ và dữ liệu:** broker bản tin, dịch vụ API, lưu trữ dữ liệu nghiệp vụ, chuỗi thời gian và nhật ký.
- **Tầng ứng dụng và vận hành:** giao diện web quản trị, cơ chế cập nhật thời gian thực, công cụ bản đồ và biểu đồ, cùng lớp container hóa và giám sát hạ tầng.

Các công nghệ cụ thể được phân tích theo vai trò kỹ thuật ở Chương 2 và được lựa chọn trong Chương 3.

1.5. Kết quả và khuyến nghị

1.5.1. Kết quả đạt được

Kết quả của đồ án được tóm lược theo bốn lớp triển khai chính: phần cứng, firmware, máy chủ và giao diện khai thác. Bốn lớp này hình thành chuỗi vận hành từ thiết bị trên xe đến lớp sử dụng của người quản lý.

Về phần cứng:

- Hoàn thiện một thiết bị theo dõi dạng bo mạch riêng, tích hợp khối xử lý trung tâm, khối truyền dữ liệu và định vị, cảm biến chuyển động, nguồn dự phòng và khả năng kết nối để đọc dữ liệu OBD2.
- Kiến trúc nguồn và cơ chế bảo vệ điện áp thấp cho phép thiết bị duy trì hoạt động theo nhiều trạng thái vận hành mà không làm cạn ắc quy xe trong điều kiện sử dụng bình thường.
- Thiết bị đạt yêu cầu tiêu thụ thấp ở chế độ ngủ sâu ($< 500 \mu A$) và có thể tự chuyển sang nguồn dự phòng khi điện áp nguồn chính xuống dưới ngưỡng an toàn.

Về firmware:

- Hoàn thiện phần mềm thiết bị có khả năng thu thập dữ liệu vận hành cơ bản, quản lý trạng thái năng lượng, phát hiện chuyển động bất thường và gửi dữ liệu lên máy chủ.
- Hệ thống có thể được đánh thức từ chế độ ngủ sâu trong thời gian ngắn (< 3 giây) khi xuất hiện sự kiện cần theo dõi.

Về hạ tầng máy chủ và xử lý dữ liệu:

- Xây dựng được chuỗi tiếp nhận, phân loại, lưu trữ và xử lý dữ liệu từ thiết bị, đồng thời tách dữ liệu nghiệp vụ, dữ liệu thời gian và nhật ký vận hành theo đúng mục đích sử dụng.
- Hoàn thiện lớp dịch vụ phục vụ các chức năng quản trị cốt lõi như theo dõi phương tiện, quản lý thiết bị, xem dữ liệu vận hành, xử lý cảnh báo và quản lý vùng giám sát.
- Hệ thống đã được kiểm thử ở mức hơn 50 thiết bị đồng thời với độ trễ xử lý chính dưới 500 ms, cho thấy còn dư địa mở rộng trong giai đoạn tiếp theo.

Về giao diện web:

- Hoàn thiện giao diện quản lý tập trung với bản đồ thời gian thực, lịch sử hành trình, biểu đồ phân tích và hệ thống cảnh báo trực tuyến.
- Dữ liệu vị trí và trạng thái có thể được cập nhật lên giao diện với độ trễ dưới 2 giây.

1.5.2. Khuyến nghị và hướng phát triển

Từ kết quả hiện tại, các hướng phát triển tiếp theo được sắp theo thứ tự ưu tiên từ việc củng cố hệ lõi đến mở rộng giá trị sử dụng của hệ thống:

1. **Ứng dụng di động (giai đoạn 2):** Phát triển ứng dụng di động riêng để người quản lý giám sát đội xe trên điện thoại và nhận thông báo đầy khi có cảnh báo [8].
2. **Tăng cường bảo mật:** Triển khai TLS/SSL cho kết nối MQTT trong môi trường vận hành thật, áp dụng danh sách quyền MQTT chi tiết hơn để mỗi thiết bị chỉ đi đúng kênh dữ liệu của mình, và bổ sung cơ chế mã hóa dữ liệu từ thiết bị tới máy chủ.
3. **Tối ưu hóa hiệu năng:** Nghiên cứu và áp dụng các kỹ thuật nén dữ liệu vận hành gửi liên tục, chẳng hạn chỉ gửi phần sai khác hoặc dùng định dạng gói gọn hơn, để giảm băng thông 4G, đặc biệt khi vận hành đội xe lớn (> 500 xe).
4. **Tích hợp trí tuệ nhân tạo:** Áp dụng các mô hình học máy để phân tích hành vi lái xe, dự đoán thời điểm cần bảo trì và phát hiện các dấu hiệu bất thường từ dữ liệu vận hành đã thu thập.
5. **Mở rộng phạm vi OBD2:** Hỗ trợ đọc thêm nhiều thông số chẩn đoán xe, bao gồm mã lỗi DTC (Diagnostic Trouble Codes), dữ liệu động cơ nâng cao, và tích hợp với các loại xe điện (EV).
6. **Cải thiện khả năng mở rộng:** Nghiên cứu kiến trúc tách nhiều dịch vụ nhỏ và bổ sung cơ chế đệm hoặc hàng đợi bản tin chuyên dụng để xử lý luồng dữ liệu lớn hơn khi số lượng thiết bị tăng lên hàng ngàn.
7. **Tối ưu PCB cho sản xuất:** Nâng thiết kế PCB hiện tại theo hướng dễ sản xuất hơn và ít nhiễu điện từ hơn, giảm kích thước, tăng độ bền cơ khí và chuẩn bị cho sản xuất hàng loạt.

Lộ trình phát triển dự án theo các giai đoạn

CHƯƠNG 2. PHÂN TÍCH VẤN ĐỀ KỸ THUẬT

Nhu cầu quản lý xe cho thuê tự lái cần được chuyển thành các yêu cầu kỹ thuật có thể kiểm tra được. Phần phân tích dưới đây xác định các vấn đề kỹ thuật, ràng buộc, chỉ tiêu nghiệm thu và tiêu chuẩn tham chiếu làm cơ sở cho quá trình thiết kế hệ thống.

2.1. Mô tả vấn đề - Problem statement

2.1.1. Bối cảnh vận hành

Đối tượng của đề tài là phương tiện cho thuê tự lái. Trong quá trình khai thác, xe có thể ở ba trạng thái chính: đang chạy, đang đỗ hoặc nằm ngoài vùng quản lý dự kiến. Ở mỗi trạng thái, thiết bị giám sát phải thu được dữ liệu phù hợp, truyền được thông tin về máy chủ và không gây ảnh hưởng xấu đến hệ điện nguyên bản của xe.

Bài toán kỹ thuật không dừng ở việc xác định tọa độ phương tiện. Một hệ thống dùng được trong vận hành cần kết hợp được vị trí, trạng thái xe, trạng thái thiết bị, nguồn điện và cảnh báo bất thường thành một chuỗi dữ liệu đủ tin cậy để người quản lý theo dõi.

2.1.2. Các vấn đề kỹ thuật chính

Từ bối cảnh trên, đề tài tập trung vào bốn nhóm vấn đề kỹ thuật sau.

Vấn đề về nguồn điện trên xe. Thiết bị phải bám nguồn trong thời gian dài nhưng không được làm cạn ắc quy, đặc biệt khi xe đỗ. Ngoài ra, điện áp trên ô tô có thể dao động theo trạng thái đề máy, sạc, tắt máy hoặc tải phụ khác. Vì vậy, thiết kế cần có khả năng ổn áp, chuyển chế độ tiêu thụ thấp và ngắt bảo vệ khi điện áp xuống thấp.

Vấn đề về thu thập dữ liệu. Hệ thống cần lấy dữ liệu vị trí, trạng thái kết nối, chuyển động khi xe đỗ và một số thông tin vận hành cơ bản từ xe. Các nguồn dữ liệu này khác nhau về chu kỳ cập nhật, độ tin cậy và cách xử lý, nên cần xác định rõ dữ liệu nào bắt buộc, dữ liệu nào là bổ trợ.

Vấn đề về truyền dữ liệu. Xe di chuyển qua nhiều khu vực có chất lượng sóng khác nhau. Thiết bị cần gửi được dữ liệu định kỳ, ưu tiên cảnh báo quan trọng và có khả năng phục hồi sau mất kết nối ngắn hạn.

Vấn đề về khai thác dữ liệu. Người quản lý cần nhìn thấy vị trí, lịch sử hành trình, cảnh báo và trạng thái thiết bị trên giao diện tập trung. Do đó, dữ liệu sau khi gửi về máy chủ phải được lưu trữ và trình bày theo cách dễ tra cứu, không chỉ tồn tại dưới dạng bản tin rời rạc.

2.1.3. Mục tiêu kỹ thuật của chương

Các vấn đề trên được chuyển thành ba nhóm đầu ra kỹ thuật: yêu cầu chức năng, ràng buộc thiết kế và chỉ tiêu nghiệm thu. Những nội dung này là cơ sở để đánh giá phương án ở Chương 3 và đối chiếu kết quả ở Chương 4.

2.2. Bối cảnh và cơ sở kỹ thuật - Background and Technical reviews

2.2.1. Các hướng quản lý phương tiện hiện có

Trong thực tế có ba hướng tiếp cận phổ biến: quản lý thủ công, sử dụng thiết bị định vị cơ bản và sử dụng nền tảng thương mại hoàn chỉnh. Bảng 2.1 so sánh các hướng này theo các tiêu chí liên quan trực tiếp đến đề tài.

Bảng 2.1: So sánh các hướng quản lý phương tiện

Tiêu chí	Quản lý thủ công	Thiết bị định vị cơ bản	Nền tảng thương mại
Theo dõi vị trí	Phụ thuộc liên lạc với khách thuê	Có vị trí và lịch sử cơ bản	Có vị trí, lịch sử và cảnh báo
Theo dõi trạng thái kỹ thuật	Không có	Hạn chế	Có ở một số hệ thống
Phát hiện bất thường khi xe đỗ	Rất hạn chế	Phụ thuộc loại thiết bị	Tốt hơn nhưng phụ thuộc gói dịch vụ
Chi phí ban đầu	Thấp	Thấp đến trung bình	Trung bình đến cao
Khả năng tùy biến theo nghiệp vụ riêng	Thấp	Hạn chế	Phụ thuộc nhà cung cấp

Tiêu chí	Quản lý thủ công	Thiết bị định vị cơ bản	Nền tảng thương mại
Phù hợp với đồ án	Không đủ dữ liệu	Có thể tham khảo	Có giá trị tham chiếu, nhưng chi phí và phụ thuộc cao

Từ bảng trên, khoảng trống của đề tài nằm ở một hệ thống đủ dữ liệu hơn thiết bị định vị cơ bản, nhưng vẫn có thể tùy biến và kiểm soát chi phí tốt hơn so với việc phụ thuộc hoàn toàn vào nền tảng thương mại.

2.2.2. Đặc điểm kỹ thuật cần xét

Để hệ thống hoạt động ổn định trên xe, các nhóm đặc điểm sau cần được xét trước khi chọn giải pháp.

Bảng 2.2: Đặc điểm kỹ thuật ảnh hưởng đến thiết kế

Nhóm kỹ thuật	Đặc điểm cần xét	Ảnh hưởng đến thiết kế
Nguồn trên xe	Điện áp 12 V hoặc 24 V, có dao động khi đề máy và khi tải thay đổi	Cần ổn áp, bảo vệ quá dòng, bảo vệ điện áp thấp và nguồn dự phòng
Định vị	Vị trí cần đủ chính xác để theo dõi hành trình, nhưng không nhất thiết đạt độ chính xác đo đạc địa hình	Cần mô-đun định vị phù hợp và chu kỳ cập nhật hợp lý
Dữ liệu vận hành	Dữ liệu từ xe chỉ cần ở mức cơ bản trong phạm vi đồ án	Ưu tiên phương án đọc ít xâm lấn, dễ lắp và dễ bảo trì
Phát hiện bất thường	Xe đỗ cần tiêu thụ thấp nhưng vẫn nhận biết được rung/chuyển động bất thường	Cần cảm biến tiêu thụ thấp và cơ chế đánh thức thiết bị
Truyền thông	Chất lượng mạng di động thay đổi theo khu vực	Cần cơ chế gửi lại, phục hồi kết nối và ưu tiên bản tin cảnh báo

Nhóm kỹ thuật	Đặc điểm cần xét	Ảnh hưởng đến thiết kế
Giao diện quản lý	Người dùng không phải là kỹ sư phần mềm	Cần giao diện trực quan, tập trung vào bản đồ, trạng thái và cảnh báo

2.2.3. Yêu cầu rút ra từ cơ sở kỹ thuật

Từ các đặc điểm trên, hệ thống cần được đánh giá theo năm nhóm: độ an toàn nguồn, mức độ ít xâm lấn khi lắp đặt, khả năng thu dữ liệu đủ dùng, độ ổn định truyền dữ liệu và tính dễ sử dụng của giao diện. Năm nhóm đánh giá này được chuyển thành yêu cầu chức năng, yêu cầu phi chức năng và chỉ tiêu nghiệm thu định lượng.

2.3. Yêu cầu kỹ thuật và các tiêu chuẩn thiết kế - Design criteria and Constraints

2.3.1. Yêu cầu chức năng

Bảng 2.3: Yêu cầu chức năng của hệ thống

Mã yêu cầu	Nội dung yêu cầu	Kết quả cần đạt
YC-01	Theo dõi vị trí phương tiện	Ghi nhận tọa độ, tốc độ và thời điểm cập nhật của xe
YC-02	Lưu lịch sử hành trình	Cho phép xem lại tuyến đường và các mốc thời gian chính
YC-03	Đọc trạng thái vận hành cơ bản	Thu thập được một số thông tin chẩn đoán/hoạt động phù hợp với phạm vi OBD2 của xe
YC-04	Phát hiện bất thường khi xe đỗ	Nhận biết rung động hoặc chuyển động bất thường và gửi cảnh báo
YC-05	Quản lý năng lượng	Chuyển giữa chế độ hoạt động, chờ và ngủ sâu; có bảo vệ điện áp thấp

Mã yêu cầu	Nội dung yêu cầu	Kết quả cần đạt
YC-06	Truyền dữ liệu về máy chủ	Gửi bản tin định kỳ và bản tin cảnh báo qua mạng di động
YC-07	Hiển thị trên giao diện quản lý	Hiển thị vị trí, trạng thái, lịch sử và cảnh báo theo từng xe
YC-08	Cập nhật cấu hình thiết bị	Cho phép thay đổi một số tham số vận hành như chu kỳ gửi dữ liệu, ngưỡng cảnh báo và chế độ tiết kiệm điện

2.3.2. Yêu cầu phi chức năng

Các yêu cầu phi chức năng xác định mức chất lượng tối thiểu để hệ thống có giá trị sử dụng trong môi trường thật.

Bảng 2.4: Yêu cầu phi chức năng

Nhóm yêu cầu	Chỉ tiêu định hướng	Ý nghĩa
Độ trễ dữ liệu	Dữ liệu từ thiết bị lên giao diện trong điều kiện mạng bình thường không vượt quá vài giây	Người quản lý theo dõi được trạng thái gần thời gian thực
Độ ổn định nguồn	Thiết bị không reset khi mô-đun truyền dữ liệu hoạt động tải cao	Tránh mất dữ liệu và mất kết nối bất thường
Tiêu thụ khi xe đỗ	Dòng tiêu thụ ở chế độ ngủ sâu phải đủ thấp để không ảnh hưởng đáng kể đến ắc quy	Phù hợp lắp đặt dài ngày
Khả năng bảo trì	Có thể xác định lỗi theo nhóm: nguồn, thiết bị, kết nối, máy chủ hoặc giao diện	Thuận lợi khi vận hành thử nghiệm
Khả năng mở rộng	Cấu trúc hệ thống cho phép tăng số xe thử nghiệm mà không phải thiết kế lại từ đầu	Phù hợp triển khai thử nghiệm thí điểm

Nhóm yêu cầu	Chỉ tiêu định hướng	Ý nghĩa
An toàn dữ liệu	Có xác thực người dùng và giới hạn truy cập theo vai trò	Bảo vệ thông tin vị trí và lịch sử xe

2.3.3. Ràng buộc thiết kế

Bảng 2.5: Ràng buộc thiết kế

Loại ràng buộc	Nội dung	Tác động đến đồ án
Nguồn xe	Thiết bị làm việc với hệ điện 12 V/24 V	Cần dải nguồn vào rộng và mạch bảo vệ
Lắp đặt	Không can thiệp sâu vào hệ thống điều khiển nguyên bản của xe	Ưu tiên kết nối ít xâm lấn, dễ tháo lắp
Chi phí	Phù hợp với nguyên mẫu sinh viên và khả năng nhân rộng thử nghiệm	Ưu tiên linh kiện phổ biến, tài liệu rõ, dễ mua
Thời gian	Triển khai trong thời lượng đồ án	Cần chọn phương án có rủi ro tích hợp vừa phải
Môi trường	Thiết bị làm việc trong xe, chịu rung động và nhiệt độ thực tế	Cần bố trí cơ khí, nguồn và dây nối chắc chắn
Dữ liệu cá nhân	Vị trí và hành trình xe là dữ liệu nhạy cảm	Cần có nguyên tắc phân quyền và chỉ thu thập dữ liệu phục vụ mục tiêu quản lý

2.3.4. Tiêu chuẩn và tài liệu kỹ thuật áp dụng

Các tiêu chuẩn và tài liệu kỹ thuật dưới đây được dùng để xác định ranh giới thiết kế và kiểm chứng của nguyên mẫu.

Bảng 2.6: Tiêu chuẩn và tài liệu kỹ thuật tham chiếu

Nhóm	Tài liệu/chuẩn tham chiếu	Phạm vi áp dụng
Chẩn đoán trên xe	SAE J1979, ISO 15031-5, ISO 15765-2	Cơ sở đọc dữ liệu OBD2 ở mức thông số vận hành và mã chẩn đoán liên quan [26], [27], [28]
Mạch nguồn và pin	Datasheet IC nguồn, IC sạc, pin 18650	Xác định miền điện áp, dòng tải và giới hạn sạc/xả [30]
Truyền dữ liệu IoT	Tài liệu chuẩn của giao thức truyền bản tin nhẹ	Tham chiếu mô hình gửi dữ liệu theo bản tin, mức tin cậy và kênh truyền [25]
Bảo vệ thông tin	Nguyên tắc xác thực, phân quyền và ghi nhật ký truy cập	Bảo vệ dữ liệu vị trí, lịch sử và cảnh báo
Thiết kế PCB	Quy tắc bố trí nguồn, mass, anten và đường tín hiệu	Giảm nhiễu, tăng độ ổn định khi modem hoạt động

2.3.5. Chỉ tiêu nghiệm thu định lượng

Bảng 2.7: Chỉ tiêu nghiệm thu của nguyên mẫu

Nhóm kiểm chứng	Chỉ tiêu cần đạt	Phương pháp kiểm tra
Nguồn chính	Thiết bị hoạt động ổn định trong dải nguồn 12 V/24 V	Cấp nguồn thử, quan sát điện áp nhánh nguồn và trạng thái reset
Chế độ ngủ sâu	Dòng tiêu thụ ở mức thấp, mục tiêu dưới 500 μ A	Đo dòng bằng đồng hồ/máy đo công suất khi thiết bị vào chế độ ngủ
Nguồn dự phòng	Thiết bị vẫn duy trì hoạt động trong thời gian ngắn khi nguồn chính gián đoạn	Ngắt nguồn chính có kiểm soát và quan sát thời gian duy trì
Định vị	Thiết bị thu được tọa độ và gửi được lên máy chủ	Đối chiếu bản tin thiết bị và vị trí trên giao diện

Nhóm kiểm chứng	Chỉ tiêu cần đạt	Phương pháp kiểm tra
Dữ liệu xe	Đọc được dữ liệu OBD2 cơ bản qua thiết bị trung gian	Kiểm tra bản tin có trường dữ liệu xe và trạng thái kết nối
Cảnh báo	Phát sinh cảnh báo khi có chuyển động bất thường hoặc vượt vùng giám sát	Tạo tình huống thử và kiểm tra cảnh báo trên giao diện
Toàn tuyến	Dữ liệu đi từ thiết bị đến giao diện trong điều kiện mạng bình thường	Đo thời gian từ khi thiết bị gửi bản tin đến khi giao diện cập nhật
Cập nhật cấu hình	Thiết bị nhận và áp dụng cấu hình mới an toàn	Thay đổi cấu hình thử và kiểm tra giá trị đang dùng trên thiết bị

2.4. Yêu cầu từ các bên liên quan - Stakeholder requirements

2.4.1. Các bên liên quan

Bảng 2.8: Các bên liên quan và nhu cầu chính

Bên liên quan	Nhu cầu	Ảnh hưởng đến thiết kế
Chủ doanh nghiệp cho thuê xe	Theo dõi vị trí, lịch sử, trạng thái xe và cảnh báo	Cần giao diện tập trung, dữ liệu dễ tra cứu và cảnh báo đủ kịp thời
Nhân viên vận hành	Lắp đặt nhanh, kiểm tra tình trạng thiết bị và xử lý cảnh báo	Cần thiết bị gọn, báo trạng thái rõ và thao tác vận hành đơn giản
Khách thuê xe	Thiết bị không ảnh hưởng tới quá trình sử dụng xe	Cần lắp đặt ít xâm lấn, an toàn điện và bảo vệ dữ liệu cá nhân
Đội kỹ thuật/bảo trì	Dễ kiểm tra lỗi, thay linh kiện và cập nhật phần	Cần tách rõ các khối chức năng, có trạng thái chẩn đoán và đầu nối

Bên liên quan	Nhu cầu	Ảnh hưởng đến thiết kế
	mềm	kiểm tra phù hợp
Đơn vị quản trị dữ liệu	Kiểm soát người dùng, quyền truy cập và lịch sử thao tác	Cần cơ chế xác thực, phân quyền và nhật ký truy cập ở mức phù hợp với nguyên mẫu

2.4.2. Ma trận liên hệ yêu cầu - bên liên quan

Bảng 2.9: Ma trận liên hệ yêu cầu với bên liên quan

Yêu cầu	Chủ doanh nghiệp	Nhân viên vận hành	Khách thuê xe	Đội kỹ thuật/bảo trì	Quản trị dữ liệu
YC-01, YC-02: vị trí và hành trình	X	X			X
YC-03: trạng thái xe	X	X		X	
YC-04: cảnh báo	X	X		X	X
YC-05: quản lý năng lượng	X	X	X	X	
YC-06: truyền dữ liệu	X	X		X	X
YC-07: giao diện	X	X			X
YC-08: cấu hình thiết bị		X		X	X

Các yêu cầu trên là căn cứ để đánh giá phương án thiết kế theo các ràng buộc về nguồn, lắp đặt, dữ liệu và khả năng sử dụng.

CHƯƠNG 3. CÁC GIẢI PHÁP THIẾT KẾ - DESIGN SOLUTIONS

Chương 3 trình bày quá trình chuyển các yêu cầu kỹ thuật ở Chương 2 thành các quyết định thiết kế cụ thể. Trọng tâm của chương không chỉ là liệt kê linh kiện hoặc công nghệ được dùng, mà là giải thích vì sao từng phương án được chọn trong điều kiện của đồ án: thiết bị phải lắp được trên xe thật, đọc được dữ liệu đủ dùng, truyền dữ liệu ổn định, tiết kiệm năng lượng và có thể kiểm chứng bằng nguyên mẫu.

Các giải pháp được phân tích theo từng khối chức năng: phần cứng trên xe, firmware, truyền thông dữ liệu, máy chủ và giao diện khai thác. Với mỗi khối, báo cáo đối chiếu các phương án khả thi theo yêu cầu vận hành, mức độ can thiệp vào xe, chi phí, độ phức tạp tích hợp và khả năng mở rộng. Cách trình bày này giúp liên kết trực tiếp giữa yêu cầu thiết kế, lựa chọn kỹ thuật và kết quả triển khai ở các chương sau.

3.1. Phân tích tổng hợp và nguyên tắc lựa chọn - General analysis

3.1.1. Chuỗi chức năng của hệ thống

Trước khi đi vào từng phương án, cần xác định chuỗi chức năng mà hệ thống phải bảo đảm từ xe đến người quản lý. Chuỗi này bắt đầu ở thiết bị gắn trên xe, nơi dữ liệu vị trí, dữ liệu OBD2, chuyển động khi xe đỗ và trạng thái nguồn được thu thập. Dữ liệu sau đó được đóng gói và truyền qua mạng di động về máy chủ để lưu trữ, xử lý cảnh báo và cung cấp cho giao diện web. Người quản lý tiếp nhận kết quả cuối cùng dưới dạng bản đồ, lịch sử hành trình, trạng thái thiết bị và thông tin cảnh báo.

 Hình 3.1 - Sơ đồ khối tổng thể của hệ thống thiết bị theo dõi

Hình 3.1: Sơ đồ khối tổng thể của hệ thống theo dõi phương tiện

Nguồn: Hình vẽ của tác giả

Hình 3.1 cho thấy hệ thống được tổ chức theo một tuyến dữ liệu liên tục thay vì các khối rời rạc. Vì vậy, một lựa chọn ở tầng thiết bị có thể tạo ràng buộc cho các tầng sau: đọc OBD2 qua BLE yêu cầu vi điều khiển có BLE ổn định; modem LTE/GNSS có dòng

tải xung cao nên cần nhánh nguồn riêng; dữ liệu vị trí và trạng thái gửi theo chu kỳ đòi hỏi máy chủ có cơ chế lưu trữ phù hợp với dữ liệu thời gian. Các ràng buộc liên tầng này là cơ sở để xây dựng tiêu chí lựa chọn ở các mục tiếp theo.

3.1.2. Các mâu thuẫn thiết kế chính

Bài toán của đề án không chỉ là ghép các linh kiện có sẵn. Khi lắp thiết bị lên xe thật, các yêu cầu thường kéo nhau theo hướng trái ngược. Bốn mâu thuẫn dưới đây chi phối trực tiếp việc chọn vi điều khiển, modem, cảm biến, khối nguồn, giao thức truyền dữ liệu và cách tổ chức máy chủ.

Bảng 3.1: Mâu thuẫn thiết kế và hệ quả lựa chọn

Mâu thuẫn thiết kế	Diễn giải	Hệ quả đối với phương án thiết kế
Dữ liệu đầy đủ và lắp đặt ít xâm lấn	Muốn đọc nhiều thông tin xe thì dễ phải can thiệp sâu vào hệ thống điện; nhưng xe cho thuê cần thiết bị tháo lắp nhanh và hạn chế thay đổi dây nguyên bản	Ưu tiên đọc OBD2 qua bộ chuyển đổi rời, không đấu trực tiếp vào bus xe trong nguyên mẫu
Sẵn sàng cảnh báo và tiết kiệm năng lượng	Thiết bị cần nhận biết bất thường khi xe đỗ, nhưng không thể để modem và BLE luôn hoạt động	Dùng cảm biến gia tốc có ngắt để đánh thức MCU khi cần
Mất nguồn chính và duy trì cảnh báo	Thiết bị có thể bị rút khỏi nguồn xe hoặc mất điện khi ắc quy bị tháo; nếu tắt ngay thì hệ thống không kịp ghi nhận sự kiện bất thường	Bổ sung pin dự phòng, mạch chuyển nguồn tự động và giám sát điện áp nguồn chính
Dữ liệu thời gian thực và hệ thống dễ bảo trì	Dữ liệu phải cập nhật nhanh nhưng không nên dồn mọi xử lý vào một chương trình duy nhất	Tách kênh nhận bản tin, lớp lưu trữ và lớp giao diện theo trách nhiệm rõ ràng

Từ các mâu thuẫn trên, tiêu chí chọn phương án không phải là "linh kiện mạnh nhất" hay "phần mềm nhiều chức năng nhất", mà là cấu hình cân bằng giữa lắp đặt, khả

năng duy trì hoạt động khi mất nguồn chính, khả năng lấy dữ liệu và mức độ dễ kiểm chứng trong phạm vi đồ án.

3.1.3. Tiêu chí đánh giá theo khối

Các tiêu chí ở Bảng 3.2 được xác định theo vai trò của từng khối trong chuỗi vận hành. Nhờ đó, việc so sánh không chỉ dựa trên thông số riêng lẻ của linh kiện mà còn xét đến khả năng tích hợp, đo kiểm và vận hành trên xe.

Bảng 3.2: Tiêu chí lựa chọn theo khối thiết kế

Khối thiết kế	Tiêu chí ưu tiên	Lý do kỹ thuật
Vi điều khiển trung tâm	Đủ giao tiếp ngoại vi, có BLE, hỗ trợ ngủ sâu, cộng đồng và tài liệu tốt	Phải điều phối modem, OBD2 BLE, cảm biến gia tốc, đo nguồn và các trạng thái vận hành
Truyền dữ liệu và định vị	Có mạng di động LTE, định vị vệ tinh GNSS, tài liệu rõ, bố trí phần cứng gọn	Thiết bị cần vừa gửi dữ liệu qua mạng di động vừa lấy vị trí
Đọc dữ liệu OBD2	Ít xâm lấn, dễ lắp, tương thích nhiều xe, dễ thay thế	Phù hợp đặc thù xe cho thuê tự lái
Cảm biến chuyển động	Tiêu thụ thấp, có ngắt phần cứng, cấu hình được ngưỡng	Phát hiện rung/chuyển động khi xe đỗ mà không cần bật toàn hệ thống
Nguồn và bảo vệ	Chịu được nguồn xe 12 V/24 V, có dự phòng, theo dõi điện áp bằng ADC và chủ động tắt bằng phần mềm	Tránh reset, bảo vệ ắc quy và duy trì giám sát khi mất nguồn ngắn
Truyền thông thiết bị - máy chủ	Gửi bản tin nhỏ, có cơ chế tin cậy, hỗ trợ lệnh điều khiển từ máy chủ	Phù hợp bản tin vị trí, cảnh báo và cấu hình từ xa
Máy chủ và giao diện	Dễ triển khai, dễ lưu dữ liệu theo thời gian, dễ theo dõi	Phù hợp nguyên mẫu và giai đoạn thử nghiệm đội xe nhỏ

3.2. Đề xuất và lựa chọn giải pháp phần cứng trên xe - Hardware solution

Thiết bị trên xe là phần chịu ràng buộc vật lý rõ nhất của đồ án. Thiết bị phải lấy được dữ liệu có ích nhưng không làm thay đổi sâu hệ điện xe; phải đủ nhỏ để lắp kín; phải chịu được nguồn xe dao động; đồng thời phải duy trì được một mức giám sát nhất định khi xe tắt máy. Vì vậy, phần cứng được chọn theo từng khối chức năng nhưng luôn kiểm tra sự ăn khớp giữa các khối.

3.2.1. Lựa chọn vi điều khiển trung tâm

Vi điều khiển là phần điều phối toàn bộ thiết bị. Nó cần giao tiếp với modem LTE/GNSS, đọc cảm biến gia tốc, đo điện áp nguồn, kết nối BLE với bộ chuyển đổi OBD2 và quản lý chế độ ngủ. Từ yêu cầu đó, ba hướng được cân nhắc:

- **MCU tích hợp không dây**, đại diện là ESP32-S3: có Wi-Fi/BLE sẵn, tài nguyên xử lý tốt và hệ sinh thái phát triển lớn.
- **MCU tiết kiệm điện kết hợp BLE rời**, đại diện là STM32L4: tiêu thụ thấp ở riêng MCU nhưng phải bổ sung phần vô tuyến ngoài.
- **MCU không dây thiên về BLE**, đại diện là nRF52840: có BLE tích hợp và mức ngủ sâu thấp, nhưng dư địa UART hẹp hơn.

Bảng 3.3: So sánh phương án vi điều khiển trung tâm

Tiêu chí	ESP32-S3	STM32L4 + BLE rời	nRF52840
BLE tích hợp để đọc OBD2	BLE 5.0 tích hợp, tương thích ngược BLE 4.0 [19], [20]	Không tích hợp BLE, phải thêm mô-đun ngoài	BLE 5.x tích hợp
UART phần cứng khả dụng cho modem + gỡ lỗi	3 UART, đủ cho modem và kênh gỡ lỗi riêng [19], [20]	Có nhiều USART/UART tùy biến thể, nhưng phải chia thêm tài nguyên cho BLE ngoài	Thường dùng 2 UARTE/UART, dư địa hẹp hơn nếu giữ kênh gỡ lỗi riêng

Tiêu chí	ESP32-S3	STM32L4 + BLE rời	nRF52840
Tài nguyên xử lý và RAM	Dual-core 240 MHz, 512 KB SRAM [19]	Cortex-M4 tới 80 MHz, RAM thấp hơn theo biến thể dùng phổ biến [54]	Cortex-M4F 64 MHz, 256 KB RAM [55], [56]
Dòng ngủ sâu ở mức MCU	Mức microamp tùy cấu hình RTC/IO [19], [20]	Thấp hơn rõ, là lợi thế chính của dòng low-power [53], [54]	Mức dưới microamp ở chế độ System OFF tùy cấu hình [55]
Phần cứng bổ sung để đạt đúng kiến trúc hiện tại	Không cần radio BLE rời	Cần thêm BLE ngoài và miền nguồn tương ứng	Không cần BLE rời nhưng phải cân nhắc lại bài toán UART
Tác động đến sơ đồ và firmware hiện tại	Giữ nguyên sơ đồ modem UART + OBD2 BLE + cảm biến	Tăng BOM, tăng đi dây và tăng công tích hợp	Giảm linh kiện vô tuyến rời nhưng làm kiến trúc cổng giao tiếp chặt hơn
Đánh giá	Phương án chốt	Không ưu tiên trong nguyên mẫu	Có giá trị tham khảo nhưng không thuận bằng ESP32-S3

ESP32-S3 được chọn vì cân bằng tốt giữa tài nguyên xử lý, ngoại vi và độ đơn giản tích hợp. STM32L4 có lợi thế tiết kiệm điện ở cấp MCU, nhưng lợi thế này giảm đi khi phải bổ sung BLE rời để đọc OBD2. nRF52840 là phương án kỹ thuật hợp lý hơn một máy tính nhúng nhỏ, song vẫn kém thuận hơn ESP32-S3 do dư địa UART hẹp hơn trong kiến trúc đang dùng. Với cấu hình hiện tại, tải năng lượng lớn nhất nằm ở modem LTE/GNSS, nên việc giảm số linh kiện phụ và giảm rủi ro tích hợp quan trọng hơn việc tối ưu riêng dòng ngủ của MCU.

 Hình 3.2 - Minh họa ESP32-S3 và các chân kết nối chính

Hình 3.2: ESP32-S3 và các nhóm chân kết nối chính trong thiết bị

Nguồn: Tác giả tổng hợp từ tài liệu kỹ thuật và sơ đồ thiết kế

Bảng 3.4: Thông số ESP32-S3 được dùng trong thiết kế

Thông số / đặc tính	Giá trị hoặc mô tả	Ý nghĩa trong thiết kế
Kiến trúc xử lý	Dual-core Xtensa LX7, tối đa 240 MHz [19]	Có đủ dư địa xử lý cho modem, BLE, cảm biến và logic trạng thái
Kết nối không dây	Wi-Fi và BLE tích hợp [19], [20]	BLE dùng để giao tiếp với bộ chuyển đổi OBD2; Wi-Fi hỗ trợ cấu hình hoặc thử nghiệm
Giao tiếp ngoại vi	UART, I2C, SPI, ADC, GPIO [19], [20]	UART cho modem, I2C cho LIS3DH, ADC cho giám sát điện áp
Chế độ năng lượng thấp	Có chế độ ngủ sâu mức microamp tùy cấu hình [19]	Cho phép giảm tiêu thụ khi xe đỗ dài

3.2.2. Lựa chọn mô-đun truyền dữ liệu và định vị

Khối truyền dữ liệu phải gửi bản tin qua mạng di động, đồng thời cung cấp vị trí GNSS cho hệ thống. Ba phương án được cân nhắc là tổ hợp A7670C + NEO-M8N, tổ hợp EC200U-CN + NEO-M8N, và mô-đun tích hợp SIM7600CE-T. Hai phương án đầu tách LTE và GNSS thành hai khối rời; phương án cuối tích hợp cả hai trong một mô-đun.

Bảng 3.5: So sánh phương án LTE/GNSS

Tiêu chí	A7670C + NEO-M8N	EC200U-CN + NEO-M8N	SIM7600CE-T tích hợp
Số mô-đun cần tích hợp	2 mô-đun: LTE + GNSS rời	2 mô-đun: LTE + GNSS rời	1 mô-đun tích hợp LTE + GNSS [21], [22]
LTE category	Cat-1, đủ cho bản tin IoT nhưng thấp hơn Cat-4	Cat-1 bis, thiên về bài toán dữ liệu vừa	Cat-4, dư địa truyền dữ liệu lớn hơn
GNSS tích hợp trong modem	Không, cần GNSS ngoài	Không rõ ràng bằng phương án tích hợp,	Có GNSS tích hợp ngay trong mô-đun

Tiêu chí	A7670C + NEO-M8N	EC200U-CN + NEO-M8N	SIM7600CE-T tích hợp
		vẫn phải ghép GNSS ngoài	
Kênh điều khiển MCU cần quản lý	LTE và GNSS đi trên các kênh riêng, firmware phải ghép dữ liệu	Tương tự A7670C + NEO-M8N	Điều khiển tập trung qua AT command trên UART [21], [22]
Số đường RF/anten cần bố trí	Nhiều hơn do tách 2 mô-đun	Nhiều hơn do tách 2 mô-đun	Gọn hơn về bố trí phần cứng
Tác động khi reset modem	Có thể giữ khối GNSS rời nếu thiết kế tách nguồn	Tương tự phương án tách rời	Reset modem thường kéo theo khởi tạo lại GNSS
Đánh giá	Dùng được nhưng tăng công tích hợp	Dùng được nhưng chưa thuận bằng phương án chót	Phương án chót

SIM7600CE-T được chọn vì đáp ứng đồng thời hai nhiệm vụ: truyền dữ liệu qua LTE và lấy vị trí GNSS, trong khi vẫn giữ sơ đồ phần cứng gọn nhất. Theo tài liệu phần cứng, mô-đun làm việc trong miền nguồn khoảng 3,4-4,2 V và điều khiển bằng tập lệnh AT [21], [22]. Việc dùng một mô-đun tích hợp giúp giảm số đường điều khiển, giảm số anten cần bố trí và làm firmware dễ kiểm soát hơn trong các bước bật modem, kiểm tra mạng, lấy vị trí, gửi bản tin và reset khi cần.

 Hình 3.3 - Minh họa mô-đun SIM7600CE-T

Hình 3.3: Mô-đun SIM7600CE-T và các kết nối chính với MCU

Bảng 3.6: Thông số SIM7600CE-T được dùng trong thiết kế

Thông số / đặc tính	Giá trị hoặc mô tả	Ý nghĩa trong thiết kế
Chức năng chính	LTE Cat-4 và GNSS tích hợp [21], [22]	Gộp truyền dữ liệu và định vị vào cùng một khối

Thông số / đặc tính	Giá trị hoặc mô tả	Ý nghĩa trong thiết kế
Điều khiển	Tập lệnh AT qua UART [21]	ESP32-S3 có thể điều khiển modem bằng giao tiếp UART
Nguồn cấp	Khoảng 3,4-4,2 V theo tài liệu phần cứng [22]	Cần nhánh nguồn riêng cho modem, không cấp chung trực tiếp với logic 3,3 V
Chân điều khiển	Nhóm chân bật/tắt, reset và UART [21], [22]	Cho phép firmware phục hồi modem khi mạng lỗi hoặc treo kết nối

3.2.3. Lựa chọn phương án đọc dữ liệu OBD2

Mục tiêu của khối OBD2 là lấy được các thông tin vận hành cơ bản của xe, ví dụ tốc độ, vòng tua, trạng thái lỗi hoặc một số dữ liệu chẩn đoán khả dụng. Trong bối cảnh xe cho thuê tự lái, việc lắp đặt phải nhanh, có thể tháo khi cần và hạn chế can thiệp vào dây điện nguyên bản. Vì vậy, ba phương án được cân nhắc là đấu trực tiếp vào đường chẩn đoán, dùng bộ chuyển đổi có dây, hoặc dùng bộ chuyển đổi OBD2 qua BLE (bluetooth tiết kiệm năng lượng).

Bảng 3.7: So sánh phương án đọc dữ liệu OBD2

Tiêu chí	Đấu dây OBD2 trực tiếp	Bộ chuyển đổi OBD2 có dây	Bộ chuyển đổi OBD2 BLE vgate iCar Pro
Mức can thiệp vào xe	Cao, phải đi dây hoặc chạm sâu vào đường chẩn đoán	Vẫn cần dây từ cổng OBD2 đến tracker	Thấp, chỉ cắm adapter vào cổng OBD2
Kết nối giữa tracker và OBD2	Dây tín hiệu trực tiếp, khó tháo lắp	Dây UART/USB từ adapter tới tracker	BLE 4.0, tương thích với BLE tích hợp của ESP32-S3
Tài nguyên phần cứng MCU cần dùng	Có thể chiếm UART/CAN riêng và tăng cách ly phần cứng	Thường chiếm UART hoặc USB bridge	Không chiếm UART của MCU, giữ UART cho modem và kênh gỡ lỗi

Tiêu chí	Đầu dây OBD2 trực tiếp	Bộ chuyển đổi OBD2 có dây	Bộ chuyển đổi OBD2 BLE vgate iCar Pro
Khả năng chuyển giữa nhiều xe	Phải tháo và đi dây lại	Chuyển được nhưng vẫn phải mang theo dây nối	Chỉ cần rút/cắm adapter và ghép nối lại BLE nếu cần
Khả năng giấu thiết bị chính	Tracker buộc ở gần cổng OBD2, dễ lần theo dây	Tracker vẫn bị ràng buộc tương đối gần cổng OBD2	Tracker có thể đặt tách khỏi cổng OBD2 trong khoang lái
Ảnh hưởng khi thiết bị ngủ sâu	Không có bài toán kết nối lại BLE nhưng tăng dây cố định	Tương tự phương án có dây	Phải xử lý kết nối lại BLE sau khi thức dậy
Kết luận chọn	Không ưu tiên	Có thể dùng nhưng kém gọn	Phương án chốt

vgate iCar Pro BLE được chọn vì phù hợp định hướng lắp đặt ít xâm lấn. Thiết bị hỗ trợ BLE, giao tiếp kiểu ELM327 và nhiều giao thức OBD-II thông dụng [29], [64]. Điểm cần lưu ý là bộ chuyển đổi BLE làm firmware phải quản lý trạng thái kết nối lại sau khi thiết bị ngủ sâu. Tuy nhiên, nhược điểm này chấp nhận được vì đổi lại thiết bị gọn hơn, giấu dây trong khoang lái, dễ thay bộ chuyển đổi giữa các xe và giúp vị trí thiết bị chính khó bị phát hiện hơn.

 Hình 3.4 - Minh họa kết nối OBD2 BLE

Hình 3.4: Kết nối BLE giữa ESP32-S3 và bộ chuyển đổi OBD2 vgate iCar Pro

Nguồn: Tác giả tổng hợp từ tài liệu kỹ thuật và sơ đồ thiết kế

Bảng 3.8: Thông số OBD2 BLE được dùng trong thiết kế

Thông số / đặc tính	Giá trị hoặc mô tả	Ý nghĩa trong thiết kế
Kiểu kết nối	BLE [29], [64]	Không cần kéo dây tín hiệu OBD2 về thiết bị định vị
Kiểu giao tiếp	Dạng tập lệnh ELM327 [29]	Firmware có thể gửi lệnh đọc PID theo cách phổ biến

Thông số / đặc tính	Giá trị hoặc mô tả	Ý nghĩa trong thiết kế
Tương thích giao thức	Hỗ trợ nhiều giao thức OBD-II thông dụng [29], [64]	Tăng khả năng dùng với nhiều dòng xe phổ biến
Giới hạn cần kiểm chứng	Không dùng số liệu standby nếu không có nguồn đúng biến thể	Tránh đưa thông số không chắc chắn vào thiết kế

3.2.4. Lựa chọn cảm biến phát hiện chuyển động

Khi xe đang đỗ, thiết bị không nên duy trì modem và kết nối OBD2 ở trạng thái hoạt động liên tục vì sẽ làm tăng tiêu thụ điện. Tuy nhiên, hệ thống vẫn cần phát hiện các tình huống như xe bị rung mạnh, bị kéo hoặc có chuyển động bất thường. Cảm biến chuyển động vì vậy phải tiêu thụ thấp, có thể cấu hình ngưỡng và có chân ngắt để đánh thức vi điều khiển.

Bảng 3.9: So sánh phương án phát hiện chuyển động khi xe đỗ

Tiêu chí	Công tắc rung	Cảm biến gia tốc LIS3DH	IMU 6 trục
Số chiều dữ liệu	Chỉ cho biết có rung hoặc không	Gia tốc 3 trục [23], [24]	Gia tốc 3 trục + con quay 3 trục
Giao tiếp với MCU	Thường chỉ là mức logic đơn giản	I2C hoặc SPI [23]	I2C hoặc SPI
Chân ngắt đánh thức	Có thể có nhưng thông tin nghèo	Có INT1/INT2 để đánh thức MCU [23], [24]	Có ngắt nhưng cấu hình phức tạp hơn
Khả năng đặt ngưỡng và thời gian lọc	Rất hạn chế	Có thể đặt ngưỡng, thời gian và chế độ ngắt	Có nhưng vượt nhu cầu của bài toán
Mức tiêu thụ phù hợp khi xe đỗ	Rất thấp	Mức thấp, phù hợp thiết bị chạy pin [23]	Cao hơn và dư tính năng

Tiêu chí	Công tắc rung	Cảm biến gia tốc LIS3DH	IMU 6 trục
Dữ liệu phục vụ phân tích sau cảnh báo	Gần như không có	Có thể đối chiếu biên độ gia tốc	Nhiều dữ liệu hơn mức cần dùng
Phù hợp đồ án	Quá đơn giản	Phương án chốt	Chưa cần thiết

LIS3DH được chọn vì đủ thông tin cho mục tiêu phát hiện rung/chuyển động bất thường, trong khi vẫn giữ tiêu thụ thấp và mạch đơn giản. So với công tắc rung, LIS3DH cho phép cấu hình ngưỡng bằng phần mềm, giảm khả năng báo giả. So với IMU 6 trục, LIS3DH gọn hơn và không đưa thêm dữ liệu con quay hồi chuyển mà hệ thống chưa dùng đến.

 Hình 3.5 - Minh họa LIS3DH và chân ngắt

Hình 3.5: Cảm biến LIS3DH và kết nối I2C/ngắt với ESP32-S3

Nguồn: Tác giả tổng hợp từ tài liệu kỹ thuật và sơ đồ thiết kế

Bảng 3.10: Thông số LIS3DH được dùng trong thiết kế

Thông số / đặc tính	Giá trị hoặc mô tả	Ý nghĩa trong thiết kế
Số trục đo	3 trục gia tốc [23]	Nhận biết rung/chuyển động theo nhiều hướng
Nguồn cấp	1,71-3,6 V [23]	Phù hợp nhánh nguồn logic 3,3 V
Giao tiếp	I2C hoặc SPI [23]	Dễ ghép với ESP32-S3 bằng bus I2C
Thang đo gia tốc	±2 g, ±4 g, ±8 g, ±16 g [23]	Cho phép chọn độ nhạy theo mức rung thực tế
Chân ngắt phần cứng	INT1/INT2 [23], [24]	Đánh thức vi điều khiển khi xe đỗ có chuyển động bất thường

3.2.5. Lựa chọn kiến trúc nguồn và bảo vệ

Nguồn là khối có ảnh hưởng trực tiếp nhất đến độ tin cậy của thiết bị khi đưa ra xe thật. Nếu khối này thiết kế không đúng, mọi phần khác của hệ thống đều mất ý nghĩa:

modem có thể làm sập áp và reset MCU, thiết bị có thể ngắt đột ngột khi nguồn xe chậm chờn, hoặc nghiêm trọng hơn là chính thiết bị trở thành tải làm ắc quy chính tụt xuống vùng không an toàn.

Điện áp lấy từ xe 12 V hoặc 24 V không ổn định như nguồn trong phòng thí nghiệm. Nó thay đổi theo trạng thái nổ máy, tắt máy, sạc ắc quy và theo các tải điện khác trên xe. Trong khi đó, SIM7600CE-T lại là tải khó nhất để nuôi ổn định, vì khi bám mạng LTE hoặc lấy dữ liệu GNSS, mô-đun này có thể tạo ra các xung dòng lớn hơn đáng kể so với ESP32-S3 và LIS3DH. Bởi vậy, bài toán nguồn không thể giải bằng một bộ hạ áp duy nhất cấp chung cho tất cả khối chức năng.

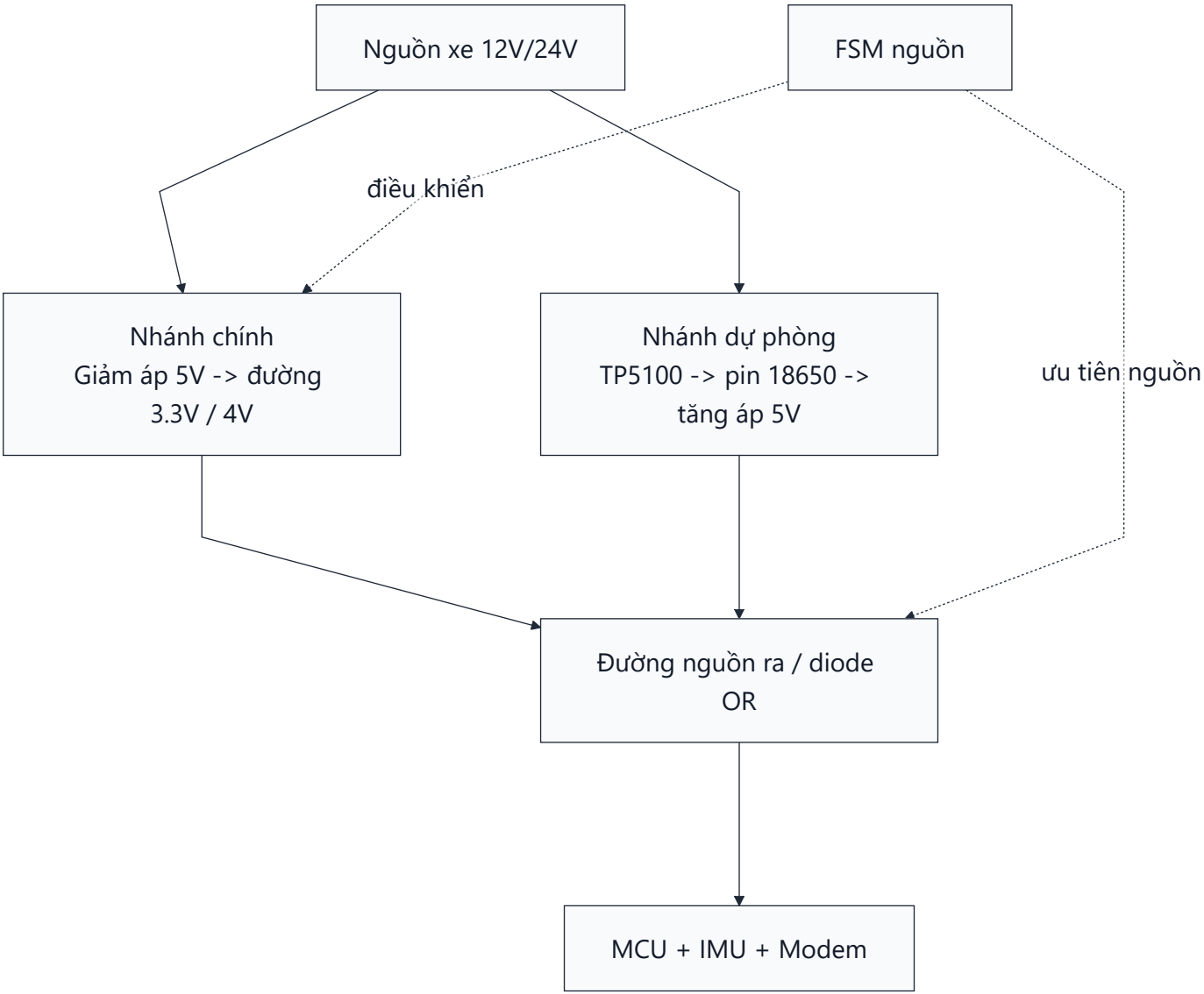
Khối nguồn của thiết bị phải giải cùng lúc bốn yêu cầu. Thứ nhất, tạo ra các mức điện áp phù hợp cho từng nhóm tải. Thứ hai, tách nhánh modem khỏi nhánh logic để xung tải của modem không làm sập vi điều khiển. Thứ ba, duy trì được thiết bị trong các khoảng mất nguồn khoảng vài giờ để còn ghi trạng thái cuối hoặc phát cảnh báo. Thứ tư, có một lớp bảo vệ đủ tin cậy để thiết bị không rút điện ắc quy xe xuống vùng nguy hiểm khi xe đỗ lâu.

Bảng 3.11: So sánh kiến trúc nguồn

Tiêu chí	Một bộ hạ áp cấp chung	Một nhánh chính kèm pin dự phòng	Kiến trúc đa nhánh có dự phòng và bảo vệ
Số rail nguồn chính	1 rail dùng chung cho logic và modem	2 rail chính và dự phòng cơ bản	Bus 5 V, rail 3,3 V logic, rail modem, rail dự phòng
Cách nuôi modem SIM7600CE-T	Dùng chung với logic, dễ sập áp khi phát xung dòng	Có cải thiện nhưng chưa tách hẳn	Có rail riêng khoảng 4 V cho modem
Khả năng duy trì khi mất nguồn xe	Không có	Có, nhưng đường chuyển nguồn còn đơn giản	Có, nhờ pin 18650 1S + boost + mạch chuyển nguồn
Giám sát và bảo vệ ắc quy xe	Phụ thuộc mạch ngoài hoặc logic tối giản	Có thể thêm nhưng chưa thành lớp rõ ràng	Có ADC theo dõi và phản ứng bảo vệ bằng firmware

Tiêu chí	Một bộ hạ áp cấp chung	Một nhánh chính kèm pin dự phòng	Kiến trúc đa nhánh có dự phòng và bảo vệ
Ảnh hưởng của xung dòng modem lên rail logic	Xung dòng đi chung rail, dễ kéo tụt nguồn MCU	Giảm bớt nhưng vẫn còn chia sẻ một phần đường cấp	Được cô lập nhờ rail modem riêng
Khả năng thêm các nhánh 3,3 V, 4 V, sạc và dự phòng	Khó, vì mọi tải bám một rail	Thêm được nhưng ranh giới chưa rõ	Thuận lợi, mỗi miền nguồn có nhiệm vụ riêng
Kết luận chọn	Không đủ ổn định cho tải modem và yêu cầu dự phòng	Chỉ phù hợp nguyên mẫu tối giản	Phương án chốt

Từ so sánh trên, phương án được chọn là kiến trúc đa nhánh: một bus 5 V trung gian dùng chung cho các khối phía sau, một nhánh logic 3,3 V riêng, một nhánh riêng cho modem, một nhánh dự phòng từ pin 18650 1S và một lớp giám sát kết hợp bảo vệ điện áp bằng firmware. Cách chọn này làm mạch phức tạp hơn so với phương án tối giản, nhưng đổi lại mỗi vấn đề quan trọng đều có một giải pháp riêng: tải lớn của modem được tách riêng, nguồn dự phòng có đường đi rõ ràng, còn ắc quy xe và pin dự phòng được bảo vệ nhờ theo dõi ADC và chuỗi tắt chủ động bằng firmware.



Hình 3.6: Sơ đồ khối hệ thống quản lý nguồn của thiết bị

Bảng 3.12: Phân vai các nhánh nguồn trong thiết bị

Miền nguồn / bảo vệ	Linh kiện chính	Tải hoặc tín hiệu liên quan	Vai trò trong kiến trúc
Bus 5 V chính từ nguồn xe	MP2482	Toàn bộ các nhánh phía sau	Tạo một đường nguồn trung gian ổn định để phân phối tiếp
Rail logic 3,3 V	AP2112-3.3	ESP32-S3, LIS3DH, RTC và mức logic	Giữ nguồn sạch cho vi điều khiển và cảm biến
Rail riêng cho modem	TPS54231	SIM7600CE-T	Tách tải động lớn của modem khỏi miền logic

Miền nguồn / bảo vệ	Linh kiện chính	Tải hoặc tín hiệu liên quan	Vai trò trong kiến trúc
Nhánh nguồn dự phòng	SX1308 + mạch chuyển nguồn diode-OR	Bus 5 V dự phòng	Duy trì hoạt động ngắn hạn khi nguồn chính gián đoạn
Nhánh sạc pin dự phòng	TP5100 [30]	Pin Li-ion 18650 1S	Nạp lại pin 1 cell từ bus 5 V chính
Giám sát và bảo vệ	ADC + logic bảo vệ bằng firmware	Điện áp nguồn xe, điện áp pin dự phòng	Theo dõi trạng thái nguồn và chủ động giảm tải, tắt thiết bị khi điện áp không an toàn

a) Nhánh buck 5 V chính từ nguồn xe - MP2482

Nhánh đầu tiên phải làm nhiệm vụ hạ điện áp nguồn xe xuống một mức trung gian để phân phối hơn. Về nguyên tắc, có thể nghĩ tới hai hướng: hoặc hạ thẳng từng mức điện áp riêng ngay từ đầu, hoặc tạo trước một đường nguồn trung gian rồi mới tách tiếp thành các nhánh nhỏ hơn. Đối với đề án này, cách thứ hai hợp lý hơn vì nó làm sơ đồ nguồn rõ ràng: 5 V trở thành bus trung gian, từ đó mới cấp điện cho nguồn hạ áp 3,3 V cho logic, cấp nguồn 3,9 V cho mô-đun SIM7600CE -T và cấp điện cho mạch sạc.

MP2482 được chọn cho vị trí này vì đây là bộ hạ áp chuyển mạch (buck converter) phù hợp với bài toán đầu vào rộng của xe. Linh kiện này làm việc trong dải điện áp vào 4,5-30 V, dòng ra liên tục tới 5 A và chuyển mạch ở 420 kHz. So với phương án dùng bộ ổn áp tuyến tính ở tầng đầu, bộ buck này tránh tổn hao nhiệt quá lớn khi hạ từ 12 V hoặc 24 V xuống mức thấp hơn. So với cách hạ trực tiếp từng nhánh ngay từ đầu, bus 5 V do MP2482 tạo ra giúp cấu trúc nguồn dễ đọc, dễ mở rộng và dễ bố trí hơn trên PCB.

Bảng 3.12A: Thông số nhánh buck 5 V chính

Thông số / đặc tính	Giá trị hoặc mô tả	Ý nghĩa trong thiết kế
Loại linh kiện	Bộ hạ áp chuyển mạch MP2482	Thực hiện bước hạ áp đầu tiên từ nguồn xe
Dải điện áp vào	4,5-30 V	Phù hợp cả hệ điện 12 V và 24 V
Dòng ra liên tục	Tới 5 A	Tạo đủ biên độ cho bus 5 V trung gian
Tần số chuyển mạch	420 kHz cố định	Cân bằng giữa độ ổn định và kích thước linh kiện phụ
Cấu hình trong thiết kế	Đầu ra đặt ở 5 V	Làm bus nguồn trung gian cho các nhánh sau

Vì vậy, MP2482 được chốt cho tầng nguồn đầu vào. Quyết định này xác lập “xương sống” 5 V cho toàn hệ thống, thay vì để từng khối phía sau phải tự xoay xở trực tiếp với nguồn xe.

b) Nhánh logic 3,3 V cho ESP32-S3 và LIS3DH - AP2112-3.3

Sau khi đã có bus 5 V trung gian, miền logic vẫn cần một nhánh riêng ở mức 3,3 V để cấp cho ESP32-S3, LIS3DH, RTC và các mức tín hiệu số. Đây là miền nguồn nhạy với nhiễu nhất của toàn thiết bị. Chỉ cần nhánh này dao động, hệ quả có thể là reset vi điều khiển, sai số đo ADC hoặc mất trạng thái giao tiếp I2C/UART đang chạy.

Ở vị trí này, có thể dùng một bộ hạ áp chuyển mạch khác hoặc dùng bộ ổn áp tuyến tính LDO (Low Dropout Regulator). Với tải logic của đồ án, phương án LDO hợp lý hơn vì mục tiêu chính không phải đẩy hiệu suất lên tối đa, mà là giữ nhánh 3,3 V đủ sạch và mạch đủ đơn giản. AP2112-3.3 phù hợp với yêu cầu đó: dòng tải liên tục tối thiểu 600 mA, điện áp ra 3,3 V với sai số $\pm 1,5\%$, độ sụt áp điển hình 250 mV ở 600 mA và dòng tĩnh chỉ khoảng 55 μ A. So với việc dùng thêm một bộ buck cho miền logic, AP2112-3.3 ít nhiễu hơn, ít linh kiện ngoại vi hơn và dễ ghép vào bus 5 V trung gian.

Bảng 3.12B: Thông số nhánh logic 3,3 V

Thông số / đặc tính	Giá trị hoặc mô tả	Ý nghĩa trong thiết kế
Loại linh kiện	Bộ ổn áp tuyến tính AP2112-3.3	Tạo nhánh 3,3 V riêng cho logic

Thông số / đặc tính	Giá trị hoặc mô tả	Ý nghĩa trong thiết kế
Điện áp ra cố định	3,3 V, sai số $\pm 1,5\%$	Phù hợp với ESP32-S3, LIS3DH và mức logic hệ thống
Dòng tải liên tục	Tối thiểu 600 mA	Đủ cho tải logic của bo mạch
Dropout điện hình	250 mV tại 600 mA	Giúp ổn áp tốt khi đầu vào dao động quanh mức 5 V
Dòng tĩnh điện hình	55 μA	Hạn chế tổn hao nền trên nhánh logic

AP2112-3.3 vì vậy được chót cho nhánh nguồn 3,3 V. Ở nhánh này, tiêu chí quan trọng nhất là độ sạch và tính dễ kiểm soát của nguồn, chứ không phải tối đa hiệu suất bằng mọi giá.

c) Nhánh nguồn riêng cho modem SIM7600CE-T - TPS54231

SIM7600CE-T là tải khó nuôi nhất của thiết bị. Theo tài liệu phần cứng của mô-đun, modem làm việc trong miền khoảng 3,4-4,2 V [22]. Điểm khó không chỉ nằm ở giá trị điện áp, mà ở việc tải của modem thay đổi rất nhanh theo trạng thái mạng. Khi đăng ký mạng, phát LTE hoặc khởi động GNSS, dòng tiêu thụ có thể tăng vọt theo từng xung ngắn. Nếu cho modem dùng chung rail với ESP32-S3 hoặc ghép qua một bộ ổn áp tuyến tính đơn giản, rủi ro sụt áp lan sang MCU và cảm biến sẽ tăng lên rõ rệt.

Vì lý do đó, đồ án chọn một nhánh buck riêng cho modem. TPS54231 phù hợp với vai trò này vì đây là bộ hạ áp chuyển mạch làm việc trong dải 3,5-28 V, dòng ra liên tục tới 2 A, tần số chuyển mạch 570 kHz và có khả năng đặt ngưỡng UVLO (Undervoltage Lockout), tức ngưỡng khóa khi điện áp vào xuống quá thấp. So với cách lấy 5 V rồi hạ tiếp bằng LDO, nhánh buck riêng giảm tổn hao nhiệt và cho phép ưu tiên đúng mức cho tải động lớn nhất của bo mạch.

Bảng 3.12C: Thông số nhánh nguồn modem

Thông số / đặc tính	Giá trị hoặc mô tả	Ý nghĩa trong thiết kế
Loại linh kiện	Bộ hạ áp chuyển mạch TPS54231	Tạo rail riêng cho modem
Dải điện áp vào	3,5-28 V	Phù hợp với nguồn xe 12 V và 24 V

Thông số / đặc tính	Giá trị hoặc mô tả	Ý nghĩa trong thiết kế
Dòng ra liên tục	Tới 2 A	Đáp ứng nhánh tải động lớn của modem khi đi cùng tụ đệm
Tần số chuyển mạch	570 kHz	Giữ kích thước linh kiện phụ ở mức gọn
Cấu hình trong thiết kế	Đầu ra khoảng 4,0 V	Bám miền làm việc của SIM7600CE-T [22]

TPS54231 vì thế được chốt cài đặt điện áp đầu ra khoảng 3,9 V cho nhánh nguồn modem. Đây là quyết định then chốt để modem có thể hoạt động mạnh mà không kéo theo lỗi lan truyền sang nhánh logic.

d) Nhánh nguồn dự phòng 5 V - SX1308 và mạch chuyển nguồn (power path)

Nguồn dự phòng của thiết bị theo dõi được thiết kế như một khoảng đệm năng lượng, không phải để thay thế hoàn toàn ắc quy xe trong thời gian dài. Mục tiêu thực tế hơn là giúp thiết bị không chết ngay khi nguồn chính bị mất, từ đó còn đủ thời gian lưu trạng thái, gửi bản tin cuối hoặc chuyển sang chế độ an toàn hay thậm chí hoạt động tiếp vài giờ. Với mục tiêu đó, pin 18650 1S là lựa chọn hợp lý vì mạch đơn giản và dễ tích hợp.

Sau khi chọn pin 1S, có hai cách tổ chức tiếp theo. Cách thứ nhất là dùng trực tiếp điện áp pin cho một số nhánh tải; cách thứ hai là nâng điện áp pin lên lại 5 V rồi ghép vào bus nguồn hiện có. Đồ án chọn cách thứ hai vì bus 5 V đã là trục phân phối trung tâm của thiết bị. SX1308 là bộ tăng áp (boost converter) phù hợp cho vai trò đó: dải vào 2-24 V, điện áp ra có thể đặt tới 28 V, tần số chuyển mạch 1,2 MHz và giới hạn dòng chuyển mạch 4 A. Trong cấu hình này, SX1308 nâng điện áp pin lên 5 V, sau đó mạch chuyển nguồn diode-OR sẽ tự động chọn giữa 5 V chính và 5 V dự phòng. Cách làm này giữ được sơ đồ nguồn gọn hơn so với việc phải thiết kế một cơ chế chuyển mạch phức tạp hoặc tái kiến trúc toàn bộ các rail phía sau.

Bảng 3.12D: Thông số nhánh nguồn dự phòng

Thông số / đặc tính	Giá trị hoặc mô tả	Ý nghĩa trong thiết kế
Loại linh kiện	Bộ tăng áp SX1308 kết hợp mạch chuyển nguồn diode-OR	Tạo và chuyển sang nhánh 5 V dự phòng

Thông số / đặc tính	Giá trị hoặc mô tả	Ý nghĩa trong thiết kế
Dải điện áp vào của boost	2-24 V	Phù hợp với pin 18650 1S ở nhiều mức xả
Điện áp ra khả dụng	Điều chỉnh được, cấu hình ở 5 V	Đồng bộ với bus 5 V chính của thiết bị
Tần số chuyển mạch	1,2 MHz	Giảm kích thước linh kiện ngoài
Cơ chế chuyển nguồn	Diode-OR giữa 5 V chính và 5 V dự phòng	Chuyển nguồn tự động, giảm nguy cơ reset

SX1308 và mạch chuyển nguồn vì vậy không phải phần “gắn thêm cho có pin dự phòng”. Chúng là cơ chế làm cho thiết bị vẫn còn ý nghĩa vận hành trong các tình huống mất nguồn trong vài giờ.

e) Mạch sạc pin 1S - TP5100

Khi đã chọn pin 18650 1S làm nguồn dự phòng, thiết kế bắt buộc phải có một khối sạc phù hợp với cấu hình một cell hiện tại. Khối sạc này phải nhận điện từ bus 5 V chính, nạp pin ở chế độ đúng với pin Li-ion 1S và hạn chế phát nhiệt quá mức khi dòng sạc tăng lên. Đây là điểm khiến phương án sạc tuyến tính kiểu cũ kém phù hợp hơn cho cấu hình đang dùng.

TP5100 được chọn vì đây là IC sạc chuyển mạch cho pin lithium, hỗ trợ cấu hình 4,2 V ở chế độ 1 cell, dòng sạc lập trình được trong khoảng 0,1-2 A, tần số chuyển mạch 400 kHz và có các pha nạp tiền sạc, dòng không đổi và áp không đổi [30]. So với phương án sạc tuyến tính của cấu hình cũ, TP5100 phù hợp hơn với nhánh dự phòng của đồ án vì ít tổn hao nhiệt hơn khi sạc ở dòng cao và ghép tự nhiên vào bus 5 V do MP2482 tạo ra.

Bảng 3.12E: Thông số nhánh sạc pin dự phòng

Thông số / đặc tính	Giá trị hoặc mô tả	Ý nghĩa trong thiết kế
Loại linh kiện	IC sạc chuyển mạch TP5100	Sạc lại pin 18650 1S của thiết bị

Thông số / đặc tính	Giá trị hoặc mô tả	Ý nghĩa trong thiết kế
Cấu hình pin hỗ trợ	1 cell, điện áp sạc 4,2 V [30]	Phù hợp với cấu hình pin dự phòng hiện tại
Điện áp vào trong thiết kế	5 V từ bus chính [30]	Gắn trực tiếp vào kiến trúc nguồn đã chọn
Dòng sạc lập trình	0,1-2 A [30]	Cho phép đặt mức sạc phù hợp với cell và điều kiện nhiệt
Phương thức sạc	Tiền sạc, dòng không đổi, áp không đổi [30]	Giữ quy trình sạc đúng cho pin Li-ion

TP5100 vì vậy được chót như nhánh sạc của nguồn dự phòng. Vai trò của nó không tách rời nhánh dự phòng, mà là điều kiện để nhánh dự phòng luôn sẵn sàng khi nguồn xe bị gián đoạn.

f) Giám sát điện áp và bảo vệ ắc quy xe - ADC và điều khiển bằng firmware

Nếu chỉ tạo ra các nhánh điện áp mà không biết chúng đang ở trạng thái nào, thiết bị vẫn có thể trở thành rủi ro cho chính chiếc xe đang theo dõi. Vì vậy, khối nguồn cần một lớp quan sát liên tục để firmware biết khi nào nguồn đang khỏe, khi nào chỉ đủ duy trì tối thiểu và khi nào phải giảm tải để tránh rút ắc quy xe xuống quá sâu.

Giải pháp được chọn trong bản thiết kế hiện tại là dùng ADC để đo điện áp, không dùng một mạch cắt cứng riêng. Hai kênh ADC đọc điện áp nguồn xe và điện áp pin dự phòng trên chân **GPIO3** và **GPIO4**, cùng dùng tỷ lệ cầu phân áp 11:1 trong firmware hiện tại. Từ các giá trị đo này, firmware suy luận trạng thái nguồn, dùng ngưỡng ADC cho nhánh kiểm tra ignition dự phòng, chặn OTA khi pin dự phòng quá thấp, và khi điện áp đi vào vùng không an toàn thì thực hiện chuỗi tắt chủ động: ngắt BLE, GNSS, MQTT, LTE, tắt modem rồi đưa thiết bị vào quy trình ngủ hoặc tắt có kiểm soát. Cách làm này không cắt điện bằng phần cứng, nhưng đạt hiệu quả bảo vệ tương đương ở cấp hệ thống vì các tải lớn được gỡ bỏ có kiểm soát trước khi ắc quy xe bị rút xuống quá sâu.

Bảng 3.12F: Thông số giám sát điện áp và bảo vệ bằng firmware

Thông số / đặc tính	Giá trị hoặc mô tả	Ý nghĩa trong thiết kế
Kênh đo nguồn xe	ADC trên GPIO3	Theo dõi điện áp nguồn chính của xe
Kênh đo pin dự phòng	ADC trên GPIO4	Theo dõi trạng thái pin dự phòng
Tỷ lệ cầu phân áp	11:1 trong firmware hiện tại	Quy đổi điện áp ADC về mức điện áp thực tế
Ngưỡng ADC ignition	Cấu hình trong khoảng 11.000-15.000 mV, mặc định 13.000 mV	Dùng như đường suy luận trạng thái máy nổ khi cần
Ngưỡng pin tối thiểu cho OTA	Cấu hình trong khoảng 3.300-4.500 mV, mặc định 3.850 mV	Tránh cập nhật firmware khi pin dự phòng quá yếu
Phản ứng bảo vệ bằng firmware	Tắt modem và các dịch vụ tiêu thụ lớn, sau đó chuyển thiết bị vào quy trình ngủ hoặc tắt có kiểm soát khi điện áp xuống vùng không an toàn	Giảm tải nguồn và tránh xả sâu ắc quy xe

ADC và logic bảo vệ bằng firmware vì vậy là lớp quan sát và bảo vệ bắt buộc của kiến trúc nguồn, không phải phần phụ. Chính nhờ cách tổ chức này, thiết bị mới có thể bám xe lâu mà vẫn giữ an toàn cho ắc quy chính.

Tổng hợp lại, kiến trúc nguồn được chốt theo một chuỗi logic rõ ràng: MP2482 tạo bus 5 V trung gian, AP2112-3.3 làm sạch nhánh 3,3 V cho logic, TPS54231 tách riêng nhánh modem khoảng 4 V, SX1308 cùng mạch chuyển nguồn giữ nhánh dự phòng 5 V, TP5100 sạc lại pin 18650 1S, còn ADC quan sát điện áp để firmware chủ động đưa hệ thống vào chế độ bảo vệ phù hợp. Cấu trúc này nhiều linh kiện hơn phương án tối giản, nhưng đó là cái giá hợp lý để đổi lấy độ ổn định, tính liên tục vận hành và sự an toàn khi thiết bị làm việc trên xe thật.

3.2.6. Tổng hợp phần cứng được chọn

Sau khi so sánh từng khối, cấu hình phần cứng được chốt như Bảng 3.13. Đây là cấu hình có mức tích hợp vừa đủ: không quá tối giản đến mức thiếu ổn định, nhưng cũng không mở rộng quá mức so với phạm vi đồ án.

Bảng 3.13: Trích thông số chính của phần cứng được chọn

Khối chức năng	Linh kiện / phương án chốt	Thông số hoặc đặc tính chính	Ý nghĩa trong thiết kế
MCU	ESP32-S3	Dual-core 240 MHz, BLE, UART, I2C, ADC, ngủ sâu [19], [20]	Điều phối modem, OBD2 BLE, cảm biến và nguồn
LTE/GNSS	SIM7600CE-T	LTE Cat-4, GNSS tích hợp, nguồn khoảng 3,4-4,2 V, AT command [21], [22]	Giảm số mô-đun và giảm trạng thái lỗi
OBD2	vgate iCar Pro BLE	BLE, giao tiếp kiểu ELM327, hỗ trợ nhiều giao thức OBD-II [29], [64]	Lắp nhanh, ít xâm lấn, dễ thay bộ chuyển đổi
Chuyển động	LIS3DH	3 trục, 1,71-3,6 V, I2C/SPI, ±2 g đến ±16 g, INT1/INT2 [23], [24]	Đánh thức thiết bị khi xe đỗ có rung/chuyển động
Nguồn	MP2482, AP2112-3.3, TPS54231, TP5100, SX1308, 18650 1S, mạch chuyển nguồn, ADC	Tách nhánh nguồn logic, rail modem và rail dự phòng; sạc pin bằng TP5100 [30]	Ổn định nhánh nguồn, bảo vệ ắc quy bằng chiến lược phần mềm và duy trì khi mất nguồn ngắn

3.3. Đề xuất và lựa chọn giải pháp firmware - Firmware solution

Sau khi chốt phần cứng ở Mục 3.2, firmware giữ vai trò điều khiển các khối phần cứng đó theo đúng trạng thái của xe. Khác với một thiết bị đo đơn giản chỉ đọc cảm biến rồi

gửi dữ liệu, thiết bị theo dõi phương tiện phải làm việc trong nhiều tình huống khác nhau: xe đang chạy, xe đang đỗ, nguồn chính bị ngắt, mạng 4G yếu, hoặc có rung động bất thường khi xe không hoạt động. Vì vậy, firmware được xem xét theo ba vấn đề chính: nền tảng phát triển, mô hình điều phối trạng thái và các cơ chế bảo đảm vận hành.

3.3.1. Lựa chọn nền tảng phát triển firmware

Nền tảng firmware phải tương thích trực tiếp với ESP32-S3 đã chọn ở phần cứng, đồng thời phải đủ khả năng điều phối các ngoại vi có thời gian phản hồi khác nhau như modem SIM7600CE-T, OBD2 BLE, cảm biến LIS3DH và các kênh đo nguồn. Ba hướng triển khai được cân nhắc là Arduino Core, ESP-IDF kết hợp FreeRTOS và Zephyr RTOS.

Bảng 3.14: So sánh các phương án nền tảng firmware

Tiêu chí	Arduino Core	ESP-IDF + FreeRTOS	Zephyr RTOS
Mức độ bám sát ESP32-S3	Thư viện quen thuộc, phù hợp mẫu nhỏ	Bộ công cụ chính thức của Espressif [47]	Hỗ trợ được nhưng không phải luồng chính của ESP32-S3
BLE, NVS, OTA và quản lý nguồn	Có thư viện nhưng thường qua lớp bọc	Có sẵn trong hệ sinh thái ESP-IDF [47]	Có nhưng cần tự ghép cấu hình nhiều hơn
Tổ chức tác vụ cho modem, BLE, IMU và ADC	Chủ yếu theo vòng lặp chính tuần tự, khó tách tải khi ngoại vi tăng	Có FreeRTOS, hàng đợi, semaphore và nhóm sự kiện [10]	Có RTOS đầy đủ
Điều khiển ngủ sâu và nguồn đánh thức	Làm được nhưng phải tự giữ nhiều nhánh điều kiện trong ứng dụng	Hỗ trợ tốt chế độ ngủ sâu, nguồn đánh thức và miền nguồn	Làm được nhưng công tích hợp cao hơn trên ESP32-S3
Khả năng giữ cấu trúc mã khi ghép	Khởi đầu nhanh nhưng dễ phình	Giữ cấu trúc rõ giữa trình điều khiển, tác	Cần thêm thời gian chuẩn hóa

Tiêu chí	Arduino Core	ESP-IDF + FreeRTOS	Zephyr RTOS
đồng thời modem, BLE và IMU	logic trong một luồng chính	vụ nền và lỗi trạng thái	thư viện và gỡ lỗi
Kết luận chọn	Không ưu tiên cho thiết bị nhiều ngoại vi	Phương án chốt	Có giá trị tham khảo nhưng không thuận bằng ESP-IDF

ESP-IDF kết hợp FreeRTOS được chọn vì cân bằng tốt giữa khả năng phát triển và độ ổn định khi vận hành. ESP-IDF cung cấp các trình điều khiển và dịch vụ nền phù hợp với ESP32-S3 như BLE, NVS, OTA, quản lý nguồn và ghi nhật ký hệ thống [47]. FreeRTOS cho phép chia các công việc có thời gian phản hồi khác nhau thành các tác vụ độc lập, nhờ đó việc đọc OBD2, điều khiển modem, chờ phản hồi mạng và xử lý cảm biến không làm nghẽn toàn bộ chương trình [10].

Bảng 3.15: Các dịch vụ nền được dùng trong firmware

Thành phần	Vai trò trong thiết bị	Ý nghĩa kỹ thuật
ESP-IDF	Cung cấp trình điều khiển, thư viện hệ thống và công cụ biên dịch	Giảm rủi ro khi tích hợp với ESP32-S3
FreeRTOS	Điều phối các tác vụ có thời gian phản hồi khác nhau	Phù hợp với hệ thống có modem, BLE, cảm biến và quản lý nguồn
BLE stack	Kết nối với bộ chuyển đổi OBD2 BLE	Cho phép đọc dữ liệu xe mà không cần đấu trực tiếp vào bus xe
NVS	Lưu cấu hình thiết bị trên flash	Giữ được chu kỳ gửi, ngưỡng cảnh báo và trạng thái ghép nối sau khi khởi động lại
OTA	Cập nhật firmware từ xa	Phù hợp với thiết bị đã lắp trên xe, khó tháo ra để nạp thủ công
Chế độ ngủ sâu	Đưa thiết bị về chế độ tiêu thụ thấp	Gắn trực tiếp với yêu cầu bảo vệ ắc quy khi xe đỗ lâu

3.3.2. Mô hình điều phối trạng thái thiết bị

Thiết bị không nên hoạt động theo một vòng lặp cố định trong mọi hoàn cảnh. Khi xe đang chạy, hệ thống cần ưu tiên vị trí, OBD2 và truyền dữ liệu đều đặn. Khi xe đỗ, yêu cầu quan trọng hơn là giảm tiêu thụ điện nhưng vẫn giữ khả năng phát hiện rung động hoặc mất nguồn. Khi có cảnh báo, thiết bị phải thức dậy nhanh và ưu tiên gửi sự kiện. Từ đặc điểm đó, mô hình điều phối được lựa chọn là máy trạng thái trung tâm kết hợp các tác vụ nền của FreeRTOS.

Bảng 3.16: So sánh các mô hình điều phối firmware

Tiêu chí	Vòng lặp tuần tự	Nhiều tác vụ ngang hàng	Máy trạng thái trung tâm + tác vụ nền
Điểm ra quyết định bật/tắt modem và sleep	Nằm lẫn trong một vòng lặp lớn	Phân tán ở nhiều tác vụ	Tập trung ở lõi trạng thái
Ứng xử khi modem hoặc BLE chờ lâu	Dễ chặn toàn bộ luồng xử lý	Giảm chặn nhưng dễ phát sinh cạnh tranh tài nguyên	Tác vụ nền xử lý chờ, lõi trạng thái vẫn giữ điều phối
Biểu diễn các trạng thái chạy/đỗ/cảnh báo/nhịp duy trì	Khó tách thành nhánh rõ	Làm được nhưng dễ lệch logic giữa các tác vụ	Rõ ràng, thuận tiện viết bảng chuyển trạng thái
Quyền điều khiển modem, BLE và nguồn	Một luồng giữ toàn bộ quyền điều khiển	Nhiều tác vụ có thể cùng chạm vào modem/BLE	Chỉ lõi trạng thái quyết định, tác vụ nền chỉ thực thi
Cách quyết định thời điểm ngủ sâu và thức dậy	Dựa vào điều kiện cục bộ trong vòng lặp	Phải đồng bộ giữa nhiều tác vụ	Gắn trực tiếp với trạng thái PARKED , ALARM , HEARTBEAT
Kết luận chọn	Không phù hợp khi số ngoại vi tăng	Dùng được nhưng khó giữ nhất quán lâu dài	Phương án chốt

Ở phương án được chọn, firmware luôn xác định thiết bị đang ở trạng thái nào trước khi quyết định hành động tiếp theo. Các tác vụ nền vẫn tồn tại để xử lý những phần có

độ trễ riêng như BLE hoặc modem, nhưng các quyết định chính như bật/tắt modem, vào ngủ sâu, gửi cảnh báo hoặc cho phép OTA đều đi qua lõi trạng thái. Cách tổ chức này làm cho hành vi thiết bị nhất quán hơn khi điều kiện xe thay đổi.

 Hình 3.7 - Lưu đồ trạng thái firmware

Hình 3.7: Lưu đồ trạng thái vận hành chính của firmware

Nguồn: Hình vẽ của tác giả

Bảng 3.17: Các trạng thái vận hành chính của firmware

Trạng thái	Ý nghĩa trong vận hành	Hành vi ưu tiên
Khởi tạo	Nạp cấu hình, kiểm tra ngoại vi và chuẩn bị kết nối	Bảo đảm các khối cần thiết sẵn sàng trước khi chạy
Kiểm tra nguồn xe	Xác định xe đang có nguồn vận hành hay đang tắt máy	Chọn nhánh chạy, đỗ hoặc ngủ tiết kiệm điện
Xe đang chạy	Xe có dấu hiệu đang vận hành	Duy trì GNSS, OBD2 và gửi dữ liệu theo chu kỳ
Xe đang đỗ	Xe không hoạt động trong thời gian dài	Giảm hoạt động của modem/BLE, chỉ giữ giám sát cần thiết
Cảnh báo	Có rung động, mất nguồn hoặc sự kiện bất thường	Đánh thức hệ thống và gửi bản tin ưu tiên
Gửi trạng thái định kỳ	Thiết bị thức dậy ngắn khi xe đỗ lâu	Báo tình trạng nguồn, kết nối và vị trí gần nhất
Ngủ sâu	Thiết bị giảm tiêu thụ điện	Tắt các tải lớn và cấu hình nguồn đánh thức

Mô hình trên phù hợp với yêu cầu của thiết bị lắp trên xe vì mỗi trạng thái gắn với một mục tiêu rõ ràng. Ở trạng thái chạy, thiết bị ưu tiên dữ liệu. Ở trạng thái đỗ, thiết bị ưu tiên tiết kiệm điện. Ở trạng thái cảnh báo, thiết bị ưu tiên gửi sự kiện. Nhờ vậy, firmware không chỉ là phần “đọc và gửi dữ liệu” mà trở thành lớp điều phối hành vi của toàn bộ thiết bị.

3.3.3. Các nhóm chức năng chính trong firmware

Dựa trên mô hình trạng thái, firmware được chia thành các nhóm chức năng tương ứng với các nhiệm vụ thực tế của thiết bị. Việc phân chia này không đi sâu vào tên tệp hoặc cấu trúc mã nguồn, mà chỉ làm rõ vai trò kỹ thuật của từng nhóm trong quá trình vận hành.

Bảng 3.18: Các nhóm chức năng chính của firmware

Nhóm chức năng	Nhiệm vụ chính	Lý do cần có
Đọc dữ liệu OBD2 BLE	Kết nối bộ chuyển đổi OBD2, đọc các thông số vận hành cơ bản như tốc độ, vòng tua, điện áp và một số mã lỗi	Bổ sung dữ liệu trạng thái xe nhưng vẫn giữ lắp đặt ít xâm lấn
Quản lý LTE/GNSS	Điều khiển SIM7600CE-T để lấy vị trí và duy trì kết nối dữ liệu di động	Cung cấp vị trí và đường truyền về máy chủ
Đóng gói và gửi bản tin	Chuẩn hóa dữ liệu vị trí, OBD2, nguồn và cảnh báo trước khi gửi	Giữ cấu trúc dữ liệu thống nhất cho máy chủ xử lý
Cấu hình thiết bị	Lưu chu kỳ gửi, ngưỡng cảnh báo và các tham số vận hành	Cho phép điều chỉnh theo từng giai đoạn thử nghiệm mà không phải nạp lại chương trình
Lưu tạm khi mất mạng	Giữ bản tin quan trọng khi mạng 4G hoặc MQTT bị gián đoạn	Giảm mất dữ liệu trong điều kiện mạng di động không ổn định
Quản lý nguồn và ngủ sâu	Theo dõi nguồn xe, pin dự phòng, tắt tải lớn và đánh thức bằng thời gian hoặc cảm biến	Bảo vệ ắc quy xe và duy trì giám sát khi xe đỗ
Cập nhật firmware từ xa	Tải, kiểm tra và chuyển sang phiên bản firmware mới khi điều kiện an toàn	Giảm nhu cầu tháo thiết bị khỏi xe để bảo trì phần mềm

Trong các nhóm trên, quản lý nguồn là chức năng có ảnh hưởng lớn nhất tới độ ổn định của thiết bị. Firmware theo dõi điện áp nguồn xe và pin dự phòng thông qua ADC, từ đó quyết định thiết bị có thể tiếp tục truyền dữ liệu, chuyển sang chế độ ngủ hay ngắt các tải tiêu thụ lớn như modem. Cơ chế này kết hợp với kiến trúc nguồn đã chọn

ở Mục 3.2 để giảm nguy cơ reset khi modem phát dòng xung và hạn chế việc rút sâu ắc quy xe.

Cơ chế lưu tạm khi mất mạng cũng cần thiết cho môi trường xe thực tế. Trong quá trình di chuyển, xe có thể đi qua khu vực sóng yếu hoặc modem mất phiên truyền dữ liệu. Khi đó, các bản tin quan trọng được giữ tạm trong thẻ nhớ microSD và gửi lại sau khi kết nối khôi phục. Thiết kế này không làm mất hoàn toàn dữ liệu trong các khoảng gián đoạn ngắn, đồng thời không buộc thiết bị phải giữ modem ở trạng thái công suất cao liên tục.

3.3.4. Kết luận lựa chọn firmware

Giải pháp firmware được chốt gồm ESP-IDF, FreeRTOS, máy trạng thái trung tâm, các tác vụ nền cho BLE/modem và các cơ chế hỗ trợ gồm lưu cấu hình, OTA, ngủ sâu và lưu tạm bản tin. Cấu hình này đáp ứng các yêu cầu đã đặt ra ở Chương 2: điều phối nhiều ngoại vi, quản lý nguồn theo trạng thái xe, phục hồi sau gián đoạn mạng và có khả năng bảo trì khi thiết bị đã lắp trên xe. Đây cũng là cơ sở để triển khai các luồng chương trình, sơ đồ đấu nối và bài đo kiểm trong giai đoạn hiện thực nguyên mẫu.

3.4. Đề xuất và lựa chọn giải pháp kết nối giữa thiết bị và máy chủ

Sau khi phần cứng và firmware được xác định, yêu cầu tiếp theo là truyền dữ liệu từ thiết bị về máy chủ theo cách ổn định. Dữ liệu của thiết bị theo dõi phương tiện có ba đặc điểm chính: bản tin nhỏ, phát sinh lặp lại theo chu kỳ và có một số sự kiện cần ưu tiên như mất nguồn, rung động bất thường hoặc vượt vùng giám sát. Ngoài chiều gửi dữ liệu lên, thiết bị còn phải nhận cấu hình từ máy chủ như chu kỳ gửi dữ liệu, ngưỡng cảnh báo hoặc lệnh cập nhật firmware.

Do đó, giao thức truyền dữ liệu không thể chỉ chọn theo mức độ phổ biến. Phương án phù hợp phải đáp ứng được kiểu dữ liệu đo từ xa (telemetry), tức các bản tin đo đạc do thiết bị gửi về, làm việc được trên mạng di động và không tạo gánh nặng xử lý quá lớn cho ESP32-S3. Trong phạm vi đề án, ba hướng được xem xét là HTTP/REST, MQTT và TCP socket tự xây dựng.

3.4.1. So sánh giao thức truyền dữ liệu thiết bị - máy chủ

HTTP/REST là mô hình giao tiếp theo kiểu yêu cầu - phản hồi: một phía gửi yêu cầu, phía còn lại trả kết quả. Cách này phù hợp với các thao tác của người dùng như đăng nhập, xem lịch sử hoặc thay đổi cấu hình trên giao diện web. Tuy nhiên, nếu thiết bị dùng HTTP/REST cho từng bản tin định kỳ, mỗi lần gửi đều phải lặp lại phần tiêu đề bản tin và phản hồi, trong khi phần dữ liệu thật chỉ gồm vài trường như tọa độ, tốc độ, điện áp hoặc trạng thái kết nối.

MQTT là giao thức truyền bản tin nhẹ thường dùng cho thiết bị IoT. Thiết bị không gửi dữ liệu trực tiếp tới từng khối xử lý, mà gửi bản tin lên MQTT broker, tức máy chủ trung gian nhận và phân phối bản tin. Các thành phần phía sau chỉ cần đăng ký nhận đúng nhóm bản tin cần xử lý. Cách tổ chức này phù hợp với hệ thống có nhiều xe gửi dữ liệu liên tục, vì thiết bị chỉ phải duy trì một đường kết nối chính tới broker.

TCP socket tự xây dựng là phương án dùng kết nối TCP thuần rồi tự định nghĩa khuôn dạng bản tin. Phương án này linh hoạt nhưng làm tăng khối lượng thiết kế: đồ án phải tự xử lý đóng khung dữ liệu, xác nhận nhận bản tin, gửi lại khi lỗi, bảo mật và khôi phục kết nối. Với phạm vi nguyên mẫu, mức phức tạp đó không đem lại lợi ích tương xứng.

Bảng 3.19: So sánh giao thức truyền dữ liệu thiết bị - máy chủ

Tiêu chí	HTTP/REST	MQTT	TCP socket tự xây dựng
Mô hình truyền	Request/response, mỗi lần gửi phải mở chu kỳ yêu cầu - phản hồi	Publish/subscribe qua broker	Kết nối TCP thuần, tự định nghĩa giao thức
Overhead cho bản tin nhỏ gửi chu kỳ	Cao hơn do lặp header HTTP và phản hồi	Thấp hơn, phù hợp bản tin ngắn	Phụ thuộc hoàn toàn vào khuôn dạng tự thiết kế

Tiêu chí	HTTP/REST	MQTT	TCP socket tự xây dựng
Kênh lệnh ngược từ máy chủ về thiết bị	Cần polling hoặc kênh phụ	Có thể subscribe topic lệnh commands	Phải tự xây dựng cơ chế điều khiển
Mức tin cậy của bản tin	Do ứng dụng tự xử lý	Có QoS 0/1 theo loại bản tin	Do ứng dụng tự xử lý
Khôi phục khi mạng 4G dao động	Làm được nhưng không tối ưu cho luồng gửi liên tục	Phù hợp hơn nhờ phiên liên lạc và cơ chế kết nối lại giữa thiết bị với broker	Phải tự viết kết nối lại, gửi lại và chống trùng
Cách phân loại dữ liệu	Qua endpoint API	Qua topic	Tự định nghĩa
Vai trò phù hợp trong hệ thống	Kênh quản trị và truy vấn của người dùng	Phương án chốt cho thiết bị - máy chủ	Không ưu tiên trong nguyên mẫu

Từ so sánh trên, MQTT được chọn cho kênh thiết bị - máy chủ. HTTP/REST vẫn được sử dụng ở phần frontend (giao diện web) và backend (khối xử lý nghiệp vụ phía máy chủ) để phục vụ các thao tác của người vận hành. Như vậy, mỗi giao thức đảm nhiệm một phần việc rõ ràng: MQTT phục vụ bản tin từ thiết bị, còn HTTP/REST phục vụ truy vấn và thao tác quản lý.

3.4.2. Tổ chức luồng bản tin được chọn

Hệ thống sử dụng MQTT 3.1.1 qua mạng 4G/LTE. Trong mô hình MQTT, broker là thành phần trung gian nhận kết nối từ thiết bị, duy trì phiên liên lạc và phân phối bản tin theo topic. Thiết bị gửi dữ liệu theo các topic, tức các kênh bản tin có tên riêng cho từng nhóm dữ liệu. Nhờ đó, vị trí, trạng thái, cảnh báo và lệnh điều khiển không bị trộn lẫn trong cùng một luồng xử lý.

Sau broker MQTT, một dịch vụ trung gian đọc các topic cần thiết, kiểm tra cấu trúc bản tin, chuẩn hóa trường dữ liệu và đưa dữ liệu vào nơi lưu trữ phù hợp. Dịch vụ này

được đặt tên là MQTT Bridge; chữ "bridge" ở đây thể hiện vai trò cầu nối giữa đường bản tin MQTT và phần xử lý dữ liệu phía máy chủ.

Bảng 3.20: Nhóm topic và QoS được chọn

Nhóm topic	Hướng truyền	QoS	Nội dung chính	Lý do chọn
v1/{device_id}/rawdata	Thiết bị - > máy chủ	0	Tọa độ, tốc độ, điện áp, trạng thái khóa điện, mẫu OBD2 tức thời	Tần suất cao, chấp nhận mất lẻ tẻ để giảm tải đường truyền
v1/{device_id}/status	Thiết bị - > máy chủ	1	Trạng thái thiết bị, phiên hoạt động, bản tin duy trì kết nối	Cần hạn chế mất bản tin vì dùng để xác định thiết bị còn hoạt động
v1/{device_id}/events	Thiết bị - > máy chủ	1	Cảnh báo mất nguồn, rung động bất thường, vượt vùng, lỗi thiết bị	Sự kiện quan trọng cần được ghi nhận ít nhất một lần
v1/{device_id}/firmware	Thiết bị - > máy chủ	1	Trạng thái cập nhật firmware từ xa	Cần truy vết đầy đủ quá trình cập nhật
v1/{device_id}/commands	Máy chủ -> thiết bị	1	Cấu hình chu kỳ gửi, ngưỡng cảnh báo, lệnh cập nhật firmware	Cần thiết bị nhận được lệnh hoặc có cơ chế gửi lại

Trong MQTT, QoS (Quality of Service) là mức cam kết chuyển bản tin giữa bên gửi và bên nhận. QoS 0 tương ứng với cơ chế gửi tối đa một lần, không yêu cầu broker gửi lại nếu bản tin thất lạc; cách này giảm tải đường truyền và phù hợp với dữ liệu tần suất cao. QoS 1 tương ứng với cơ chế gửi ít nhất một lần, nghĩa là broker sẽ yêu cầu xác nhận và có thể gửi lại khi cần; đổi lại, bên nhận phải chấp nhận khả năng bản tin bị lặp. Vì vậy, QoS 0 được dùng cho dữ liệu gửi dày như vị trí hoặc mẫu OBD2 tức thời,

do việc mất một vài bản tin riêng lẻ không làm mất toàn bộ hành trình. QoS 1 được dùng cho trạng thái, cảnh báo và lệnh điều khiển vì các bản tin này cần được ghi nhận ít nhất một lần. Ở phía trình duyệt, dữ liệu mới được đẩy lên giao diện bằng WebSocket/Socket.IO, tức kênh cập nhật hai chiều giúp màn hình thay đổi mà không cần người dùng tải lại trang.

3.5. Đề xuất và lựa chọn giải pháp phía máy chủ

Sau khi chọn MQTT làm kênh thiết bị - máy chủ ở Mục 3.4, bài toán tiếp theo là tổ chức phần phía máy chủ sao cho dữ liệu đi theo một chuỗi rõ ràng từ lúc bản tin rời thiết bị đến lúc thông tin xuất hiện trên giao diện. Ở tầng này, nếu gom mọi chức năng vào một khối duy nhất thì nguyên mẫu có thể dựng nhanh nhưng khó xác định lỗi. Ngược lại, nếu chia quá vụn, báo cáo sẽ lệch sang một kiến trúc phần mềm quá nặng so với phạm vi của đề án. Vì vậy, phần này được trình bày theo cùng cách tiếp cận như phần chọn phần cứng: nêu bài toán của từng lớp, so sánh các phương án, rồi chốt giải pháp phù hợp nhất.

Trong phạm vi đề án, phần phía máy chủ được định hướng triển khai trên máy chủ riêng ảo đám mây (cloud VPS), tức một máy chủ ảo thuê từ nhà cung cấp hạ tầng đám mây nhưng người thực hiện tự cài đặt, cấu hình và vận hành các dịch vụ bên trên. Vì vậy, các lựa chọn trong Mục 3.5 được đánh giá theo hướng tự triển khai gọn trên cloud VPS, không theo hướng phụ thuộc hoàn toàn vào các dịch vụ đám mây do nhà cung cấp quản lý sẵn.

3.5.1. Lựa chọn mô hình tổ chức hạ tầng máy chủ

Máy chủ của hệ thống phải làm bốn việc cùng lúc: nhận bản tin từ thiết bị, phân luồng dữ liệu tới nơi lưu trữ phù hợp, phục vụ nghiệp vụ quản lý, và cung cấp thông tin cho giao diện người dùng. Từ yêu cầu đó, ba hướng tổ chức chính được cân nhắc là: gom toàn bộ vào một ứng dụng duy nhất; sử dụng một nền tảng thương mại đóng; hoặc chia hệ thống thành các dịch vụ tự quản theo từng vai trò, tức một kiến trúc vi dịch vụ (microservices) ở mức vừa đủ cho đề án.

Do hạ tầng được đặt trên cloud VPS với tài nguyên hữu hạn, mô hình tổ chức được chọn không chỉ cần rõ ranh giới kỹ thuật mà còn phải tránh tạo ra thêm nhiều lớp quản trị vượt quá nhu cầu của nguyên mẫu.

Bảng 3.21: So sánh phương án tổ chức hạ tầng máy chủ

Tiêu chí	Một ứng dụng duy nhất	Nền tảng thương mại đóng	Các dịch vụ tự quản theo vai trò
Đơn vị triển khai chính	1 tiến trình hoặc 1 khối ứng dụng lớn	Theo cấu trúc do nhà cung cấp quyết định	Mỗi vai trò là một dịch vụ riêng
Cách tách broker, lưu trữ, API và giao diện	Chung trong một khối	Phụ thuộc nền tảng	Tách thành các dịch vụ độc lập
Phạm vi ảnh hưởng khi một lớp gặp lỗi	Dễ lan sang toàn bộ ứng dụng	Phụ thuộc cách đóng gói của nhà cung cấp	Chủ yếu khu trú trong dịch vụ liên quan
Khả năng mở rộng từng phần	Khó tách mở rộng riêng	Có nhưng gắn với gói dịch vụ	Có thể mở rộng theo từng khối
Mức phù hợp khi tự triển khai trên cloud VPS	Dễ làm lúc đầu nhưng khó giữ ranh giới dịch vụ	Thấp do phụ thuộc nền tảng của nhà cung cấp	Phù hợp vì vừa tách lớp vừa giữ được quyền tự quản
Tự chủ dữ liệu và cấu hình	Cao	Thấp hơn do phụ thuộc nền tảng	Cao
Kết luận chọn	Dùng được nhưng khó trình bày ranh giới kiến trúc	Có giá trị tham khảo nhưng không phù hợp mục tiêu tự thiết kế	Phương án chốt

Từ so sánh trên, phương án chia thành các dịch vụ tự quản theo vai trò được chọn vì phù hợp hơn với mục tiêu kiểm chứng toàn tuyến. Với cấu trúc này, lỗi có thể được khoanh vùng theo từng đoạn: thiết bị không gửi, broker không nhận, tầng tiếp nhận không ghi dữ liệu, backend không trả kết quả hoặc frontend không cập nhật. Nói cách khác, cloud VPS là nơi đặt hạ tầng triển khai, còn kiến trúc vi dịch vụ là cách tổ chức các khối phần mềm chạy trên hạ tầng đó.

Bảng 3.22: Các lớp dịch vụ được dùng trong thiết kế phía máy chủ

Lớp dịch vụ	Vai trò chính	Ý nghĩa trong kiến trúc
MQTT broker	Nhận kết nối MQTT từ thiết bị và phân phối bản tin theo topic	Tách riêng lớp giao tiếp thời gian thực với thiết bị
Tầng tiếp nhận và phân luồng dữ liệu	Kiểm tra nội dung bản tin, chuẩn hóa và chuyển dữ liệu tới các lớp lưu trữ	Giữ tầng tiếp nhận dữ liệu tách khỏi lớp nghiệp vụ
Lớp lưu trữ dữ liệu	Lưu dữ liệu nghiệp vụ, dữ liệu đo từ xa (telemetry) và log theo đúng mục đích sử dụng	Tránh trộn lẫn các loại dữ liệu khác bản chất
Backend	Xử lý nghiệp vụ, cung cấp API và gửi lệnh điều khiển ngược	Tập trung logic ứng dụng ở một khối riêng
Frontend	Hiển thị bản đồ, trạng thái, cảnh báo và lịch sử cho người vận hành	Tách lớp giao diện khỏi xử lý máy chủ
Quan sát vận hành	Theo dõi trạng thái dịch vụ, tải hệ thống và lỗi	Hỗ trợ đo kiểm và truy vết khi tích hợp
Đóng gói triển khai	Đóng gói các dịch vụ thành các đơn vị chạy độc lập	Giữ môi trường triển khai lặp lại ổn định

3.5.2. Lựa chọn MQTT broker

Khi đã chốt MQTT là giao thức đường thiết bị - máy chủ, hệ thống cần một broker đủ ổn định để tiếp nhận kết nối từ nhiều thiết bị, hỗ trợ phân quyền theo từng thiết bị và vẫn nằm trong mức triển khai phù hợp với nguyên mẫu. Ba hướng được cân nhắc là Mosquitto, EMQX và nhóm broker thiên về triển khai lớn như HiveMQ/VerneMQ.

Bảng 3.23: So sánh phương án MQTT broker

Tiêu chí	Mosquitto	EMQX	HiveMQ/VerneMQ
Giao diện quản trị kết nối và phiên thiết bị	Thường tối giản hoặc	Có bảng điều khiển quản trị sẵn	Có nhưng thiên về triển khai lớn

Tiêu chí	Mosquitto	EMQX	HiveMQ/VerneMQ
	cần công cụ ngoài		
Xác thực và danh sách điều khiển truy cập (Access Control List - ACL) theo thiết bị, chủ đề bản tin	Có nhưng thường cấu hình thủ công hơn	Có sẵn cơ chế người dùng, phân quyền và ACL thuận tiện hơn	Có, nhưng vượt nhu cầu nguyên mẫu
Giao thức truy cập cho MQTT, TLS và WebSocket	Đủ cho MQTT cơ bản	Đầy đủ cho MQTT/TLS/WS/WSS trong cùng broker	Đầy đủ nhưng thiên về hệ lớn
Khả năng mở rộng sang bộ luật xử lý và tích hợp IoT	Mức cơ bản	Có sẵn và thuận lợi hơn khi tích hợp IoT	Mạnh nhưng nặng hơn
Khả năng tự triển khai một nút để đo kiểm	Gọn cho hệ rất nhỏ	Cân bằng giữa tính năng và độ gọn	Dùng được nhưng thừa cho nguyên mẫu
Kết luận chọn	Dùng được nhưng thiếu lớp quản trị thuận tiện	Phương án chốt	Quá nặng so với phạm vi đồ án

Từ so sánh trên, EMQX được chọn vì giữ được thể cân bằng giữa tính năng và mức triển khai. So với Mosquitto, EMQX thuận lợi hơn khi cần phân quyền bằng danh sách điều khiển truy cập (Access Control List - ACL) theo từng thiết bị và chủ đề bản tin, theo dõi kết nối qua bảng điều khiển quản trị và giữ sẵn các giao diện MQTT, MQTT qua TLS và WebSocket trong cùng một broker. So với HiveMQ hoặc VerneMQ, EMQX vẫn giữ mức tự triển khai hợp lý hơn cho một nguyên mẫu phục vụ đo kiểm [32].

Bảng 3.24: Đặc điểm EMQX được dùng trong thiết kế

Mục cấu hình	Phương án chốt	Ý nghĩa trong thiết kế
Mô hình chạy	Một nút broker	Đủ cho quy mô nguyên mẫu và đơn giản khi tích hợp
Giao diện truy cập chính	MQTT thường, MQTT bảo mật và cổng quản trị web	Phục vụ kết nối thiết bị, dự phòng bảo mật và quan sát vận hành
Kiểm soát truy cập	Tài khoản và ACL theo từng thiết bị	Hạn chế thiết bị gửi sai chủ đề bản tin
Vai trò bộ luật xử lý (Rules Engine)	Chỉ dùng cho lọc hoặc chuyển tiếp đơn giản	Không dồn nghiệp vụ chính vào broker
Cây chủ đề bản tin chính	Phân tách theo mã thiết bị và nhóm dữ liệu như Mục 3.4.2	Giữ quy ước đường bản tin thống nhất với firmware

3.5.3. Lựa chọn lớp lưu trữ và quan sát vận hành

Sau khi chốt MQTT broker, bài toán cần giải tiếp theo chưa phải là viết ngay dịch vụ MQTT Bridge, mà là xác định dữ liệu sẽ được đưa về đâu. Nếu chưa chốt đích lưu trữ thì cũng chưa thể xác định lớp tiếp nhận dữ liệu phải kiểm tra nội dung bản tin theo tiêu chí nào, tách luồng ra sao và ghi vào đâu. Dữ liệu của hệ thống gồm ba nhóm chính: dữ liệu nghiệp vụ, dữ liệu đo từ xa (telemetry) theo thời gian và nhật ký cùng thông tin quan sát vận hành. Vì vậy, phần này lần lượt chọn cơ sở dữ liệu cho từng nhóm.

a) Lựa chọn cơ sở dữ liệu nghiệp vụ

Dữ liệu nghiệp vụ của hệ thống gồm người dùng, xe, thiết bị, cấu hình, vùng giám sát và cảnh báo. Nhóm dữ liệu này cần quan hệ rõ giữa nhiều bảng, cần giữ tính nhất quán khi cập nhật và cần hỗ trợ truy vấn không gian để kiểm tra xe có đi vào hoặc ra khỏi vùng giám sát hay không. Từ yêu cầu đó, ba hướng được cân nhắc là MySQL/MariaDB, MongoDB và PostgreSQL kết hợp phần mở rộng không gian PostGIS.

Bảng 3.25: So sánh phương án cơ sở dữ liệu nghiệp vụ

Tiêu chí	MySQL/MariaDB	MongoDB	PostgreSQL + PostGIS
Mô hình dữ liệu mặc định	Quan hệ	Tài liệu	Quan hệ + không gian
Khóa ngoại và giao dịch nhiều bảng	Có sẵn	Không phải cơ chế mạnh nhất, thường bù ở tầng ứng dụng	Có sẵn
Kiểu dữ liệu vị trí và hàm vùng giám sát	Có kiểu không gian cơ bản	Có thể lưu GeoJSON nhưng kiểm tra vùng giám sát (geofence) thường phải đẩy lên ứng dụng	Có PostGIS và các hàm không gian trực tiếp
Mô hình hóa quan hệ xe - thiết bị - cảnh báo - vùng	Tự nhiên	Khi quan hệ tăng dễ phát sinh lặp dữ liệu	Tự nhiên
Kết luận chọn	Dùng được	Dùng được nhưng không tối ưu cho kiểm tra vùng giám sát	Phương án chốt

Từ so sánh trên, PostgreSQL kết hợp PostGIS được chọn cho dữ liệu nghiệp vụ. Điểm quyết định không chỉ nằm ở khả năng lưu dữ liệu quan hệ, mà ở việc hệ thống cần xử lý cả vị trí và vùng giám sát trong cùng một lớp dữ liệu. PostGIS là phần mở rộng cho phép PostgreSQL làm việc với dữ liệu không gian, nhờ đó dữ liệu xe, vùng giám sát và kết quả kiểm tra vị trí có thể được giữ trong cùng một nền tảng.

Bảng 3.26: Cấu hình cơ sở dữ liệu nghiệp vụ được chọn

Thành phần	Phương án chốt	Vai trò trong hệ thống
Hệ quản trị cơ sở dữ liệu	PostgreSQL	Lưu dữ liệu quản lý có quan hệ rõ
Phần mở rộng không gian	PostGIS	Hỗ trợ truy vấn vị trí, vùng giám sát và bản đồ
Nhóm dữ liệu chính	Người dùng, xe, thiết bị, cấu hình, cảnh báo	Tạo lớp dữ liệu nghiệp vụ thống nhất

Thành phần	Phương án chốt	Vai trò trong hệ thống
Ý nghĩa trong thiết kế	Một nền tảng cho cả dữ liệu quan hệ và dữ liệu vị trí	Giảm tách rời giữa quản lý xe và kiểm tra vùng giám sát

b) Lựa chọn cơ sở dữ liệu đo từ xa (telemetry) theo thời gian

Dữ liệu đo từ xa (telemetry) phát sinh liên tục từ thiết bị gồm vị trí, tốc độ, điện áp, trạng thái nguồn, trạng thái kết nối và một phần dữ liệu OBD2. Nhóm dữ liệu này cần ghi liên tục với số lượng lớn hơn dữ liệu nghiệp vụ, sau đó phục vụ biểu đồ và truy vấn lịch sử theo thời gian. Vì vậy, ba hướng được cân nhắc là TimescaleDB, InfluxDB và VictoriaMetrics.

Bảng 3.27: So sánh phương án cơ sở dữ liệu đo từ xa (telemetry) theo thời gian

Tiêu chí	TimescaleDB	InfluxDB	VictoriaMetrics
Nền tảng và ngôn ngữ truy vấn	Mở rộng trên PostgreSQL, truy vấn SQL	CSDL chuỗi thời gian riêng, Flux/InfluxQL	CSDL chuỗi thời gian riêng, MetricsQL tương thích PromQL
Mức tách biệt khỏi PostgreSQL nghiệp vụ	Chưa tách hẳn vì vẫn bám cùng hệ PostgreSQL	Tách riêng	Tách riêng
Cách xử lý luồng ghi vị trí, tốc độ, điện áp liên tục	Phải tối ưu thêm partition và index của PostgreSQL	Thiết kế riêng cho ghi chuỗi thời gian	Thiết kế riêng cho ghi chuỗi thời gian
Cách nối với lớp quan sát vận hành dùng Grafana	Hỗ trợ được	Hỗ trợ được	Thuận lợi khi đi cùng VictoriaLogs trong Grafana
Độ gọn khi tự triển khai một nút	Vẫn phải quản trị thêm tải của PostgreSQL	Trung bình đến cao	Gọn hơn, phù hợp nguyên mẫu
Kết luận chọn	Dùng được nhưng chưa tách hẳn tải ghi	Dùng được	Phương án chốt

Từ so sánh trên, VictoriaMetrics được chọn cho dữ liệu đo từ xa (telemetry) theo thời gian. So với TimescaleDB, phương án này giữ ranh giới rõ hơn giữa dữ liệu nghiệp vụ và dữ liệu đo từ xa (telemetry). So với InfluxDB, VictoriaMetrics phù hợp hơn với mục tiêu tự triển khai gọn, đồng thời vẫn đáp ứng nhu cầu ghi dữ liệu liên tục và truy vấn theo chuỗi thời gian để vẽ biểu đồ, xem lịch sử và theo dõi xu hướng.

Bảng 3.28: Cấu hình cơ sở dữ liệu đo từ xa (telemetry) được chọn

Thành phần	Phương án chốt	Vai trò trong hệ thống
Hệ quản trị cơ sở dữ liệu	VictoriaMetrics	Lưu dữ liệu đo từ xa (telemetry) theo thời gian
Nhóm dữ liệu chính	Vị trí, tốc độ, điện áp, trạng thái nguồn, trạng thái kết nối, dữ liệu OBD2 dạng chuỗi thời gian	Phục vụ biểu đồ và lịch sử
Kiểu khai thác chính	Truy vấn theo khoảng thời gian, theo xe và theo chỉ số	Hỗ trợ giao diện và giám sát vận hành
Ý nghĩa trong thiết kế	Tách tải ghi dữ liệu đo từ xa (telemetry) khỏi lớp dữ liệu nghiệp vụ	Giữ hiệu năng ổn định khi số bản tin tăng

c) Lựa chọn hệ thống nhật ký và quan sát vận hành

Ngoài dữ liệu nghiệp vụ và dữ liệu đo từ xa (telemetry), hệ thống còn cần một lớp ghi nhật ký để truy vết lỗi và một lớp quan sát vận hành để theo dõi trạng thái dịch vụ. Nếu chỉ nhìn dữ liệu cuối trên giao diện thì khó xác định lỗi đang nằm ở broker, MQTT Bridge, backend hay frontend. Từ yêu cầu đó, ba hướng được cân nhắc là giữ log rời theo từng dịch vụ, dùng một bộ ELK đầy đủ, hoặc dùng VictoriaLogs kết hợp Grafana.

Bảng 3.29: So sánh phương án nhật ký và quan sát vận hành

Tiêu chí	Log rời theo từng dịch vụ	ELK đầy đủ	VictoriaLogs + Grafana
Thành phần chính khi triển khai	File log hoặc log container	Elasticsearch + Logstash + Kibana	VictoriaLogs + Grafana
Cách gom log từ EMQX, MQTT Bridge, backend và frontend	Phải mở từng dịch vụ	Gom tập trung qua pipeline log	Gom tập trung qua VictoriaLogs

Tiêu chí	Log rời theo từng dịch vụ	ELK đầy đủ	VictoriaLogs + Grafana
Tìm kiếm theo dịch vụ, thiết bị và mốc thời gian	Thủ công, khó đối chiếu	Có giao diện tập trung	Có giao diện tập trung
Kết hợp log với chỉ số vận hành trên cùng màn hình	Phải dùng công cụ tách rời	Làm được	Làm được
Độ nặng khi tự triển khai	Thấp nhưng thiếu tập trung	Cao hơn, nhiều thành phần	Gọn hơn ELK và đủ cho nguyên mẫu
Kết luận chọn	Chỉ đủ cho giai đoạn rất nhỏ	Dùng được nhưng nặng	Phương án chốt

Từ so sánh trên, VictoriaLogs kết hợp Grafana được chọn cho nhật ký và quan sát vận hành. Log rời theo từng dịch vụ không đủ khi cần truy vết lỗi xuyên qua nhiều lớp. ELK có khả năng mạnh nhưng vượt quá mức cần thiết của nguyên mẫu. VictoriaLogs giữ được ưu điểm của log tập trung trong khi nhẹ hơn khi tự triển khai, còn Grafana cho phép gom việc quan sát chỉ số và nhật ký vào một nơi.

Bảng 3.30: Thành phần nhật ký và quan sát vận hành được chọn

Thành phần	Dữ liệu hoặc nhóm theo dõi chính	Vai trò trong hệ thống
VictoriaLogs	Log thiết bị, log nhận bản tin, lỗi dịch vụ, log OTA	Truy vết khi hệ thống hoạt động không đúng mong muốn
Grafana	Chỉ số và bảng theo dõi của EMQX, MQTT Bridge, backend, cơ sở dữ liệu và giao diện web	Tập trung quan sát vận hành khi thử nghiệm
Nhóm theo dõi chính	Số thiết bị kết nối, số bản tin nhận được, lỗi xác thực, lỗi ghi dữ liệu, độ trễ phản hồi, lỗi tải giao diện	Khoanh vùng lỗi theo từng lớp

3.5.4. Lựa chọn MQTT Bridge

Sau khi đã xác định rõ đích lưu trữ cho từng loại dữ liệu, hệ thống cần một lớp tiếp nhận để kiểm tra cấu trúc nội dung bản tin, chuẩn hóa dữ liệu và ghi về đúng nơi đã chọn. Có ba cách tổ chức thường gặp. Cách thứ nhất là để backend đăng ký nhận trực tiếp các chủ đề bản tin MQTT. Cách thứ hai là dùng bộ luật xử lý (Rules Engine) tại broker như tuyến xử lý chính. Cách thứ ba là tách riêng một dịch vụ tiếp nhận dữ liệu độc lập giữa broker và các lớp lưu trữ phía sau; trong đồ án, dịch vụ đó được gọi là MQTT Bridge.

Bảng 3.31: So sánh phương án tổ chức tầng tiếp nhận dữ liệu

Tiêu chí	Backend đăng ký nhận trực tiếp	Bộ luật xử lý tại broker	MQTT Bridge độc lập
Nơi đặt logic kiểm tra nội dung bản tin trước khi ghi	Nằm trong backend	Nằm trong bộ luật xử lý của broker	Nằm trong dịch vụ tiếp nhận riêng
Số đích lưu trữ có thể ghi trong cùng luồng tiếp nhận	Làm được nhưng kéo logic tiếp nhận vào backend	Hạn chế hơn	Thuận lợi: PostgreSQL, VictoriaMetrics, VictoriaLogs
Kênh phát sự kiện nội bộ cho backend và giao diện	Gắn trực tiếp với backend	Phụ thuộc cấu hình broker	Có luồng sự kiện nội bộ riêng
Mức phụ thuộc giữa tầng tiếp nhận và lớp còn lại	Gắn chặt với backend	Gắn chặt với broker	Tách được cả hai phía
Điểm khoanh vùng lỗi khi bản tin sai cấu trúc	Khó tách khỏi lỗi API	Phù hợp xử lý lọc đơn giản	Tách rõ ở tầng MQTT Bridge
Kết luận chọn	Dùng được nhưng không tối ưu	Chỉ hợp cho xử lý nhẹ	Phương án chốt

Từ so sánh trên, MQTT Bridge được chọn vì đây là cách tổ chức giữ đường tiếp nhận dữ liệu gọn nhất mà vẫn tách được trách nhiệm của từng lớp. Nếu để backend đăng ký nhận trực tiếp, lớp nghiệp vụ phải gánh thêm tải tiếp nhận dữ liệu đo từ xa (telemetry). Nếu đẩy phần lớn xử lý vào broker, phần logic sẽ bị gắn quá chặt với cấu

hình của broker. Tách riêng một dịch vụ MQTT Bridge giúp luồng dữ liệu rõ hơn và dễ kiểm thử hơn.

Sau khi chốt phải có một MQTT Bridge độc lập, bài toán tiếp theo là chọn công nghệ cài đặt cho dịch vụ này. Dịch vụ cần nhận các nhóm bản tin đã chốt ở Mục 3.4.2, kiểm tra cấu trúc JSON, ghi đồng thời về cơ sở dữ liệu quan hệ, cơ sở dữ liệu chuỗi thời gian, hệ thống log tập trung và phát tiếp sự kiện nội bộ cho backend. Ba hướng được cân nhắc là Node.js + TypeScript, Python bất đồng bộ và Go.

Bảng 3.32: So sánh công nghệ triển khai MQTT Bridge

Tiêu chí	Node.js + TypeScript	Python bất đồng bộ	Go
Mức dùng chung hệ sinh thái với backend hiện có	Cùng Node.js/TypeScript, thuận lợi chia sẻ quy ước dữ liệu	Khác hệ sinh thái, phải tách mô hình dữ liệu	Khác hệ sinh thái, phải duy trì mô hình dữ liệu riêng
Thư viện cho MQTT, PostgreSQL và kiểm tra schema	Có hệ thư viện đầy đủ cho MQTT, cơ sở dữ liệu quan hệ và kiểm tra schema JSON	Có thư viện tương ứng nhưng phải ghép thêm lớp kiểm tra	Có thư viện mạnh nhưng kiểm tra schema JSON thường thủ công hơn
Tổ chức luồng nhận bản tin -> phân tuyến -> ghi lô	Thuận lợi với mô hình sự kiện và bất đồng bộ	Làm được nhưng phải ghép thêm khung ứng dụng rõ ràng	Hiệu năng tốt nhưng công tổ chức ban đầu cao hơn
Mức thuận lợi khi dựng điểm kiểm tra hoạt động và log có cấu trúc	Gọn, phù hợp dịch vụ nhỏ	Làm được	Làm được
Kết luận chọn	Phương án chốt	Dùng được nhưng lệch hệ sinh thái hiện có	Dùng được nhưng tăng chi phí tích hợp

Node.js + TypeScript được chọn cho MQTT Bridge. Điểm quyết định không chỉ là việc dùng chung một ngôn ngữ với backend, mà còn ở chỗ dịch vụ này cần kiểm tra cấu trúc bản tin, xử lý JSON liên tục và duy trì đường ghi sang nhiều nơi lưu trữ theo cách gọn. Với lựa chọn này, MQTT Bridge giữ được mô hình triển khai nhẹ nhưng vẫn đủ rõ để kiểm thử riêng như một dịch vụ độc lập.

Bảng 3.33: Cấu hình MQTT Bridge được chọn

Thành phần	Phương án chốt	Vai trò trong hệ thống
Nền tảng triển khai	Node.js + TypeScript	Dựng dịch vụ tiếp nhận dữ liệu độc lập
Năng lực kỹ thuật chính	Thư viện MQTT, truy cập cơ sở dữ liệu, kiểm tra schema JSON và ghi log có cấu trúc	Giữ dịch vụ gọn nhưng đủ kiểm soát đầu vào
Nhóm bản tin đầu vào	Dữ liệu đo từ xa, trạng thái, sự kiện và cập nhật firmware	Tách luồng xử lý theo đúng bản chất dữ liệu
Đầu ra chính	PostgreSQL, VictoriaMetrics, VictoriaLogs và luồng sự kiện nội bộ	Phân vai dữ liệu và cấp sự kiện cho lớp nghiệp vụ
Cách quan sát vận hành	Điểm kiểm tra hoạt động và log có cấu trúc	Dễ khoanh vùng lỗi khi đo kiểm

3.5.5. Lựa chọn backend

Sau MQTT Bridge, hệ thống cần một khối xử lý nghiệp vụ phía máy chủ, tức backend, để cung cấp giao diện lập trình ứng dụng (API), tiếp nhận thao tác cấu hình, truy vấn dữ liệu lịch sử và phát cập nhật thời gian thực cho giao diện. Khối này không thay thế broker hay MQTT Bridge, mà đứng sau hai lớp đó để biến dữ liệu kỹ thuật thành chức năng quản lý. Ba hướng được cân nhắc là Node.js + Express.js + TypeScript, NestJS + TypeScript và Go + Gin.

Bảng 3.34: So sánh phương án backend

Tiêu chí	Node.js + Express.js + TypeScript	NestJS + TypeScript	Go + Gin
Ngôn ngữ và mức dùng chung với MQTT Bridge	Cùng Node.js/TypeScript	Cùng TypeScript nhưng khuôn kiến trúc nặng hơn	Khác ngôn ngữ so với MQTT Bridge
Tổ chức HTTP/REST API và tài liệu OpenAPI	Gọn, dễ ghép OpenAPI	Rõ khuôn, hỗ trợ tốt	Làm được nhưng phải ghép thêm công cụ
Gắn kênh cập nhật theo thời gian thực (WebSocket/Socket.IO) với luồng sự kiện nội bộ	Thuận lợi với hệ sinh thái Node.js [39]	Thuận lợi	Làm được nhưng phải tự ghép nhiều thành phần hơn
Kiểm tra dữ liệu đầu vào và kiểu dữ liệu xuyên lớp	Thuận lợi với TypeScript và thư viện kiểm tra schema	Thuận lợi với TypeScript	Phải tự tổ chức nhiều hơn
Mức mã khung khi dựng nguyên mẫu	Gọn cho dịch vụ API cỡ vừa	Nhiều mã khung hơn	Gọn về runtime nhưng tăng chi phí tích hợp
Kết luận chọn	Phương án chốt	Dùng được nhưng nặng khuôn	Dùng được nhưng chi phí tích hợp cao hơn

Từ so sánh trên, Node.js + Express.js + TypeScript được chọn cho backend. Phương án này đủ nhẹ để dựng nguyên mẫu nhưng vẫn cho phép tổ chức API, tài liệu

OpenAPI và kênh cập nhật thời gian thực một cách rõ ràng. TypeScript giúp giảm rủi ro sai cấu trúc dữ liệu khi dữ liệu đi qua nhiều lớp từ yêu cầu HTTP, xử lý nghiệp vụ tới phản hồi cho giao diện.

Bảng 3.35: Cấu hình backend được chọn

Thành phần	Phương án chốt	Vai trò trong hệ thống
Nền tảng triển khai	Node.js + Express.js + TypeScript	Tổ chức API và xử lý nghiệp vụ theo cách gọn
Giao tiếp chính	HTTP/REST API, tài liệu OpenAPI và kênh cập nhật theo thời gian thực (WebSocket/Socket.IO)	Phục vụ truy vấn và đẩy cập nhật gần thời gian thực
Nguồn dữ liệu	PostgreSQL, PostGIS, VictoriaMetrics và luồng sự kiện nội bộ từ MQTT Bridge	Kết nối nghiệp vụ với lớp lưu trữ và luồng sự kiện
Chức năng chính	Quản lý xe, thiết bị, cảnh báo, cấu hình và lệnh điều khiển	Biến dữ liệu kỹ thuật thành thao tác vận hành
Nhóm cập nhật theo thời gian thực	Tổng quan, theo dõi thiết bị, cảnh báo, xuất dữ liệu và điều khiển cập nhật	Tách các nhóm cập nhật theo đúng vai trò sử dụng

3.5.6. Lựa chọn frontend

Phần giao diện phía người dùng, tức frontend, phải phục vụ người quản lý đội xe quan sát nhiều phương tiện cùng lúc, xem bản đồ lớn, lọc cảnh báo và truy vấn lịch sử. Vì vậy, trước hết cần chọn hình thức khai thác phù hợp, sau đó mới chọn công nghệ cài đặt giao diện web.

Ba hướng khai thác được cân nhắc là: dùng giao diện web làm lớp khai thác chính; chỉ dùng ứng dụng di động; hoặc phát triển ứng dụng desktop chuyên dụng.

Bảng 3.36: So sánh hình thức giao diện khai thác

Tiêu chí	Giao diện web	Ứng dụng di động là chính	Ứng dụng desktop chuyên dụng
Khả năng hiển thị đồng thời bản đồ, danh sách xe và cảnh báo	Màn hình lớn qua trình duyệt, phù hợp màn hình tổng quan	Màn hình nhỏ, thường phải chuyển qua nhiều màn	Màn hình lớn nhưng phải cài ứng dụng riêng
Cách phát hành cho nhiều người vận hành	Dùng chung qua URL	Phải cài hoặc phát hành ứng dụng	Phải cài trên từng máy trạm
Phụ thuộc hệ điều hành đầu cuối	Chỉ cần trình duyệt	Phụ thuộc Android/iOS	Phụ thuộc Windows/macOS/Linux
Thuận lợi khi cập nhật giao diện trong quá trình thử nghiệm	Sửa ở máy chủ là mọi người dùng nhận ngay	Phải cập nhật bản phát hành ứng dụng	Phải cài lại hoặc cập nhật trên từng máy
Kết luận chọn	Phương án chốt	Phù hợp vai trò phụ trợ, thông báo nhanh	Không ưu tiên

Từ so sánh trên, giao diện web được chọn làm lớp khai thác chính. Trong phạm vi đề án, ứng dụng di động vẫn có giá trị ở giai đoạn sau, đặc biệt cho thông báo nhanh, nhưng không phù hợp bằng giao diện web nếu mục tiêu là đánh giá tổng thể chuỗi dữ liệu và thao tác quản trị.

Sau khi chốt giao diện web, bài toán tiếp theo là chọn công nghệ cài đặt. Frontend cần dựng được màn hình bản đồ, bảng trạng thái, lịch sử, cảnh báo; gọi REST API; duy trì kênh cập nhật theo thời gian thực và đóng gói được theo mô hình triển khai container. Ba hướng được cân nhắc là React dựng phía trình duyệt bằng Vite, Nuxt theo hệ Vue, và Next.js + React + TypeScript.

Bảng 3.37: So sánh công nghệ frontend web

Tiêu chí	Vite + React	Nuxt (Vue)	Next.js + React + TypeScript
Mô hình dựng trang ban đầu	Dựng phía trình duyệt, gọn nhưng phải tự ghép nhiều phần	Hỗ trợ SSR/SSG theo hệ Vue	Hỗ trợ dựng trang rõ ràng cho ứng dụng quản trị
Gắn REST API và kênh cập nhật theo thời gian thực (WebSocket/Socket.IO)	Làm được	Làm được	Thuận lợi, khớp với hệ React và thư viện giao tiếp thời gian thực phổ biến
Mức dùng chung TypeScript với backend	Có, nhưng phải tự ghép thêm nhiều quy ước ứng dụng	Khác hệ sinh thái frontend dự án	Cùng TypeScript, thuận lợi thống nhất kiểu dữ liệu
Tổ chức dashboard, bản đồ và các module quản trị	Làm được nhưng tự tổ chức nhiều hơn	Rõ ràng nhưng đổi hệ sinh thái	Phù hợp ứng dụng quản trị nhiều màn hình
Đóng gói và triển khai bằng container	Gọn	Gọn	Thuận lợi cho mô hình triển khai bằng container
Kết luận chọn	Dùng được	Dùng được nhưng đổi hệ sinh thái	Phương án chốt

Next.js + React + TypeScript được chọn cho frontend web. Lý do chính là phương án này giữ được giao diện quản trị rõ cấu trúc, thuận lợi khi ghép với REST API và kênh cập nhật theo thời gian thực, đồng thời vẫn đồng bộ tốt với hệ TypeScript ở backend. Nhờ đó, frontend chỉ tập trung vào hiển thị và thao tác vận hành, không phải xử lý bản tin thô từ thiết bị.

Bảng 3.38: Cấu hình frontend được chọn

Thành phần	Phương án chốt	Vai trò trong hệ thống
Nền tảng triển khai	Next.js + React + TypeScript	Dựng giao diện web quản trị cho người vận hành
Kênh dữ liệu	HTTP/REST API và WebSocket/Socket.IO	Nhận dữ liệu lịch sử và cập nhật gần thời gian thực
Năng lực hiển thị chính	Thư viện bản đồ web, kênh giao tiếp thời gian thực và lớp quản lý truy vấn dữ liệu	Hiển thị bản đồ, cập nhật trạng thái và gọi dữ liệu lịch sử
Nhóm màn hình chính	Tổng quan đội xe, bản đồ, chi tiết xe, lịch sử hành trình, cảnh báo và cấu hình thiết bị	Phủ các nhu cầu khai thác chính của nguyên mẫu
Vai trò trong kiến trúc	Đứng sau backend	Chuyển dữ liệu đã chuẩn hóa thành thông tin vận hành

3.5.7. Lựa chọn cách đóng gói và triển khai các dịch vụ

Sau khi chốt MQTT Bridge, backend và frontend, bài toán còn lại không chỉ là “chạy được dịch vụ”, mà là làm sao để toàn bộ hệ thống có thể được dựng lại lặp lại, cập nhật có kiểm soát và giảm phụ thuộc vào thao tác thủ công. Ở tầng này có hai quyết định riêng. Quyết định thứ nhất là chọn cách đóng gói dịch vụ. Quyết định thứ hai là chọn cách triển khai tự động sau khi có phiên bản mới. Vì vậy, mục này được tách thành hai phần: lựa chọn Docker cho lớp đóng gói, và lựa chọn GitHub Actions cho lớp triển khai tự động.

Trong bối cảnh này, môi trường đích không phải là một nền tảng đám mây quản lý toàn phần, mà là một cloud VPS tự quản. Điều đó có nghĩa là hệ thống cần các công cụ đủ gọn để đóng gói nhiều dịch vụ, đưa phiên bản mới lên máy chủ và giữ cho quá trình cập nhật có thể lặp lại mà không đòi hỏi thêm một tầng điều phối lớn.

a) Lựa chọn Docker cho lớp đóng gói dịch vụ

Docker là nền tảng đóng gói ứng dụng thành các container. Có thể hiểu container là một đơn vị chạy cô lập chứa mã nguồn, môi trường thực thi và các phụ thuộc chính của dịch vụ; tuy nhiên container nhẹ hơn máy ảo vì dùng chung nhân hệ điều hành của máy chủ. Với hệ thống nhiều dịch vụ cùng đặt trên một cloud VPS, Docker phù hợp vì vừa tách được các khối xử lý, vừa tận dụng tài nguyên của cùng một máy chủ.

Từ yêu cầu đó, ba hướng được cân nhắc là: cài trực tiếp trên hệ điều hành, dùng máy ảo cho từng nhóm dịch vụ, hoặc dùng container Docker. Nếu cài trực tiếp, các phụ thuộc của broker, lớp tiếp nhận dữ liệu, backend, frontend và các lớp lưu trữ sẽ dễ chồng lấn. Nếu tách mỗi dịch vụ vào một máy ảo riêng thì mức cô lập tăng lên, nhưng chi phí tài nguyên và công quản trị trên cloud VPS lại tăng nhanh, không phù hợp với quy mô nguyên mẫu.

Bảng 3.39: So sánh phương án đóng gói dịch vụ

Tiêu chí	Cài trực tiếp trên hệ điều hành	Máy ảo cho từng nhóm dịch vụ	Container Docker
Mức nhất quán giữa máy phát triển và máy chủ thử nghiệm	Dễ lệch do phụ thuộc cài trực tiếp	Tốt hơn nhưng phải duy trì nhiều hệ điều hành khách	Giữ môi trường chạy lặp lại theo image
Mức tách biệt phụ thuộc giữa các dịch vụ	Thấp	Cao	Đủ rõ cho từng dịch vụ
Mức phù hợp khi đặt nhiều dịch vụ trên một cloud VPS	Dễ phát sinh xung đột phụ thuộc	Tách biệt tốt nhưng tiêu tốn mạnh tài nguyên VPS	Phù hợp hơn vì nhẹ hơn máy ảo mà vẫn tách được dịch vụ
Chi phí tài nguyên khi số dịch vụ tăng	Thấp lúc đầu nhưng khó kiểm soát về sau	Cao hơn do mỗi máy ảo có hệ điều hành riêng	Thấp hơn máy ảo, phù hợp hệ nhiều dịch vụ nhỏ
Cập nhật hoặc thay thế từng dịch vụ	Dễ ảnh hưởng môi trường chung	Làm được nhưng nặng thao tác	Thuận lợi, có thể cập nhật từng container

Tiêu chí	Cài trực tiếp trên hệ điều hành	Máy ảo cho từng nhóm dịch vụ	Container Docker
Kết luận chọn	Dùng được nhưng khó giữ ổn định lâu dài	Dùng được nhưng quá nặng cho nguyên mẫu	Phương án chốt

Từ so sánh trên, Docker được chọn cho lớp đóng gói dịch vụ. Điểm quyết định nằm ở khả năng giữ môi trường chạy lặp lại giữa các lần tích hợp, trong khi vẫn đủ nhẹ để triển khai đồng thời nhiều dịch vụ trên cùng một cloud VPS thử nghiệm. Docker không thay thế bài toán tổ chức kiến trúc, nhưng giúp các quyết định kiến trúc ở các mục trước được chuyển thành các đơn vị chạy độc lập, dễ khởi động và dễ thay thế hơn.

Bảng 3.40: Nguyên tắc đóng gói Docker được chọn

Nội dung	Phương án chốt	Ý nghĩa trong thiết kế
Đơn vị đóng gói	Mỗi dịch vụ chính là một container riêng	Giữ ranh giới rõ giữa broker, MQTT Bridge, backend, frontend và lưu trữ
Điều phối cục bộ	Dùng Docker Compose để nhóm các container liên quan	Khởi động, dừng và cập nhật theo từng nhóm dịch vụ
Kết nối giữa các dịch vụ	Các container cùng tham gia một mạng dùng chung	Giữ giao tiếp nội bộ ổn định mà không cần gắn chặt vào máy chủ
Dữ liệu bền vững	Dữ liệu lưu trong volume hoặc thư mục gắn ngoài container	Tránh mất dữ liệu khi thay container
Giá trị đạt được	Môi trường chạy có thể dựng lại lặp lại trên cloud VPS	Phù hợp tích hợp, đo kiểm và bàn giao nguyên mẫu

b) Lựa chọn triển khai tự động bằng GitHub Actions

GitHub là nền tảng lưu trữ và quản lý mã nguồn của dự án. GitHub Actions là cơ chế tự động hóa quy trình tích hợp và triển khai nằm trong hệ sinh thái GitHub, cho phép chạy các bước kiểm tra, dựng bản phát hành và cập nhật môi trường đích khi có sự kiện thay đổi mã nguồn hoặc thao tác phát hành. Với nguyên mẫu triển khai trên cloud VPS tự quản, GitHub Actions phù hợp vì luồng kiểm tra và phát hành có thể xuất phát trực tiếp từ kho mã nguồn rồi chuyển phiên bản mới lên máy chủ.

Sau khi dịch vụ đã được đóng gói, bài toán tiếp theo là cách đưa phiên bản mới lên môi trường thử nghiệm trên cloud VPS. Nếu triển khai hoàn toàn thủ công qua SSH và chép tệp, mỗi lần cập nhật đều phụ thuộc nhiều vào thao tác người vận hành và khó bảo đảm rằng các bước kiểm tra đã được thực hiện đầy đủ. Một hướng khác là dùng một máy chủ CI/CD tự quản như Jenkins; hướng này mạnh hơn về khả năng tùy biến nhưng lại làm tăng thêm một lớp hạ tầng cần quản trị. Hướng còn lại là dùng GitHub Actions, tức nền tảng tự động hóa quy trình tích hợp và triển khai gắn trực tiếp với kho mã nguồn trên GitHub.

Bảng 3.41: So sánh phương án triển khai tự động

Tiêu chí	Triển khai thủ công qua SSH	Máy chủ CI/CD tự quản	GitHub Actions
Mức phụ thuộc vào thao tác thủ công	Cao	Thấp	Thấp
Khả năng gắn kiểm tra chất lượng trước khi triển khai	Phụ thuộc người vận hành tự thực hiện	Làm được	Làm được
Công vận hành hạ tầng tự động hóa	Thấp lúc đầu	Cao do phải duy trì máy chủ CI/CD riêng	Thấp hơn vì dùng nền tảng có sẵn
Mức phù hợp với cloud VPS tự quản	Phụ thuộc thao tác tay trên từng lần cập nhật	Làm được nhưng phải quản trị thêm máy CI/CD	Phù hợp vì chỉ cần luồng phát hành từ GitHub tới VPS
Khả năng tách quy trình theo từng dịch vụ	Làm được nhưng khó chuẩn hóa	Làm được	Thuận lợi
Kết luận chọn	Chỉ phù hợp giai đoạn rất nhỏ	Dùng được nhưng vượt nhu cầu nguyên mẫu	Phương án chốt

GitHub Actions được chọn vì cân bằng tốt giữa mức tự động hóa và công quản trị. Phương án này cho phép gắn bước kiểm tra chất lượng với bước triển khai trong cùng một luồng, đồng thời vẫn đủ linh hoạt để tách quy trình cho từng dịch vụ. So với cách triển khai thủ công, GitHub Actions giảm rủi ro bỏ sót bước khi cập nhật cloud VPS. So

với việc tự dựng một máy chủ CI/CD riêng, phương án này phù hợp hơn với quy mô và mục tiêu của đề án.

Bảng 3.42: Nguyên tắc triển khai tự động được chọn

Nội dung	Phương án chốt	Ý nghĩa trong thiết kế
Sự kiện kích hoạt	Kích hoạt khi dịch vụ có thay đổi hoặc khi người vận hành yêu cầu	Giảm triển khai thừa và giữ quyền chủ động khi cần
Bước trước triển khai	Kiểm tra mã nguồn, kiểu dữ liệu, kiểm thử và dựng bản phát hành	Chặn lỗi sớm trước khi cập nhật môi trường thử nghiệm trên cloud VPS
Đơn vị phát hành	Triển khai theo từng dịch vụ	Giảm phạm vi ảnh hưởng khi cập nhật
Bước sau triển khai	Cập nhật dịch vụ đích trên cloud VPS và kiểm tra hoạt động	Bảo đảm dịch vụ vận hành được sau khi cập nhật
Giá trị đạt được	Quy trình phát hành có thể lặp lại và truy vết được	Phù hợp cho nguyên mẫu nhiều dịch vụ cần tích hợp liên tục

3.5.8. Đánh giá tổng hợp và lựa chọn phương án khả thi

Sau khi chốt các quyết định ở Mục 3.2, 3.3, 3.4 và các mục con từ 3.5.1 đến 3.5.7, toàn bộ cấu hình cần được đánh giá lại ở cấp hệ thống. Một linh kiện có thông số tốt hoặc một công nghệ phần mềm phổ biến không tự động tạo thành phương án phù hợp. Phương án khả thi phải đồng thời đáp ứng yêu cầu lắp đặt trên xe, thu dữ liệu đủ dùng, duy trì nguồn, truyền dữ liệu ổn định, lưu trữ đúng loại dữ liệu và hiển thị được thông tin cho người quản lý.

Ba phương án tổng thể được đối chiếu là: dùng thiết bị định vị cơ bản, dùng hệ thống IoT tự thiết kế theo các lựa chọn đã phân tích, hoặc dùng nền tảng thương mại hoàn chỉnh.

Bảng 3.43: Đánh giá các phương án tổng thể

Tiêu chí	Thiết bị định vị cơ bản	Hệ thống IoT tự thiết kế	Nền tảng thương mại hoàn chỉnh
Phạm vi dữ liệu thu được	Chủ yếu vị trí và trạng thái cơ bản	Vị trí, OBD2 cơ bản, nguồn, rung động và trạng thái thiết bị	Rộng hơn nhưng phụ thuộc gói dịch vụ
Khả năng phát hiện bất thường khi xe đỗ	Hạn chế	Có nhờ IMU, quản lý nguồn và cảnh báo sự kiện	Có ở một số hệ thống
Quyền làm chủ phần cứng, firmware và máy chủ	Thấp	Cao	Thấp
Mức tùy biến quy tắc cảnh báo, vùng giám sát và cấu hình thiết bị	Thấp	Cao	Phụ thuộc nhà cung cấp
Khả năng kiểm chứng riêng từng lớp từ thiết bị tới giao diện	Hạn chế	Cao	Hạn chế do nhiều khối đóng
Giá trị đối với mục tiêu nghiên cứu của đề án	Chỉ tham khảo mức ứng dụng	Phương án chốt	Có giá trị tham khảo thị trường

Phương án hệ thống IoT tự thiết kế được chọn. Thiết bị định vị cơ bản không đủ vì chỉ giải quyết phần vị trí. Nền tảng thương mại có thể hiệu quả trong vận hành thực tế, nhưng không phù hợp với mục tiêu thiết kế và kiểm chứng một hệ thống từ phần cứng đến giao diện. Phương án tự thiết kế cho phép kiểm soát toàn bộ chuỗi: thiết bị trên xe, firmware, đường truyền MQTT, EMQX ở vai trò broker, MQTT Bridge ở vai trò tiếp nhận bản tin, backend xử lý nghiệp vụ, lớp lưu trữ phân vai và frontend hiển thị thông tin vận hành.

Bảng 3.44: Đối chiếu yêu cầu Chương 2 với phương án được chọn

Yêu cầu / ràng buộc	Quyết định thiết kế liên quan	Cách kiểm chứng ở Chương 4
Theo dõi vị trí và hành trình	SIM7600CE-T, MQTT, EMQX, lưu dữ liệu theo thời gian và bản đồ web	Kiểm tra bản tin vị trí, cập nhật bản đồ và truy xuất lịch sử
Đọc trạng thái vận hành cơ bản	ESP32-S3 kết nối vgate iCar Pro BLE	Kiểm tra kết nối OBD2 và dữ liệu đọc được trên xe thử
Phát hiện bất thường khi xe đỗ	LIS3DH, chân ngắt đánh thức và bản tin cảnh báo	Tạo rung/chuyển động thử và kiểm tra cảnh báo trên giao diện
Bảo vệ nguồn xe	Nguồn đa nhánh, pin dự phòng, ADC đo nguồn và ngủ sâu bằng firmware	Đo nhánh nguồn, thử mất nguồn và kiểm tra phản ứng của thiết bị
Truyền dữ liệu qua mạng di động	MQTT qua 4G/LTE và QoS theo loại bản tin	Kiểm tra bản tin định kỳ, bản tin trạng thái và cảnh báo
Khai thác dữ liệu tập trung	MQTT Bridge, backend, lớp lưu trữ dữ liệu và frontend	Kiểm tra luồng dữ liệu từ thiết bị đến màn hình quản lý
Khả năng thử nghiệm và mở rộng	Linh kiện phổ biến, giao thức chuẩn, Docker trên cloud VPS và GitHub Actions cho phát hành lặp lại	Triển khai nguyên mẫu, kiểm tra lại các dịch vụ và ghi nhận kết quả

3.5.9. Phương án thiết kế tối ưu

Phương án thiết kế tối ưu của đồ án là cấu hình tích hợp giữa thiết bị trên xe, firmware, đường truyền MQTT, MQTT Bridge, backend, lớp lưu trữ và frontend. Tối ưu ở đây không có nghĩa là chọn thành phần mạnh nhất, mà là chọn tổ hợp cân bằng nhất với các ràng buộc đã nêu ở Chương 2: lắp đặt ít xâm lấn, nguồn ổn định, thu dữ liệu đủ dùng, truyền dữ liệu tin cậy, dễ kiểm chứng và có khả năng mở rộng thử nghiệm.

 Hình 3.9 - Kiến trúc tổng thể phương án tối ưu

Hình 3.9: Kiến trúc tổng thể của phương án thiết kế tối ưu

Nguồn: Hình vẽ của tác giả

Bảng 3.45: Cấu hình thiết kế tối ưu được chốt

Tầng / khối	Phương án chốt	Vai trò trong hệ thống
Thiết bị trên xe	ESP32-S3, SIM7600CE-T, vgate iCar Pro BLE, LIS3DH, nguồn đa nhánh, TP5100, pin 18650 1S và ADC đo nguồn	Thu thập vị trí, OBD2 cơ bản, chuyển động, trạng thái nguồn và gửi dữ liệu
Firmware	ESP-IDF + FreeRTOS, máy trạng thái trung tâm, quản lý nguồn, lưu cấu hình, ngủ sâu và OTA	Điều phối thiết bị theo trạng thái chạy, đỗ, cảnh báo và cập nhật firmware từ xa
Kết nối thiết bị - máy chủ	MQTT 3.1.1 qua EMQX, chủ đề bản tin tách theo mã thiết bị và nhóm dữ liệu, QoS 0/1 theo loại bản tin	Truyền dữ liệu đo từ xa (telemetry) và lệnh điều khiển hai chiều
MQTT Bridge	Node.js + TypeScript, nhận các nhóm bản tin thiết bị, ghi đa đích và phát sự kiện nội bộ	Tiếp nhận, kiểm tra và phân luồng dữ liệu trước lớp nghiệp vụ
Backend	Node.js + Express.js + TypeScript, HTTP/REST API, tài liệu OpenAPI và kênh cập nhật theo thời gian thực	Xử lý nghiệp vụ và cung cấp dữ liệu cho giao diện
Lưu trữ	PostgreSQL + PostGIS, VictoriaMetrics, VictoriaLogs	Tách dữ liệu nghiệp vụ, dữ liệu đo từ xa (telemetry) và log vận hành
Giao diện web	Ứng dụng web quản trị dùng Next.js + React + TypeScript, bản đồ, lịch sử hành trình, cảnh báo và cấu hình thiết bị	Chuyển dữ liệu kỹ thuật thành thông tin vận hành cho người quản lý
Triển khai thử nghiệm	Docker, công cụ điều phối container cục bộ trên cloud VPS và GitHub Actions cho phát hành tự động	Giữ môi trường chạy thống nhất và cập nhật có kiểm soát khi tích hợp, kiểm thử

Phương án trên giải quyết các ràng buộc chính theo một chuỗi thống nhất. ESP32-S3 có đủ ngoại vi và BLE để kết nối OBD2 mà không cần mô-đun vô tuyến rời.

SIM7600CE-T gộp truyền dữ liệu di động và GNSS trong một mô-đun, giúp giảm số khối phần cứng. LIS3DH cho phép phát hiện chuyển động khi xe đỗ với mức tiêu thụ thấp. Kiến trúc nguồn tách riêng nhánh logic, nhánh modem và nhánh dự phòng để giảm rủi ro reset khi tải thay đổi. Firmware theo trạng thái giúp thiết bị chuyển hợp lý giữa chạy, đỗ, cảnh báo và ngủ sâu.

Ở phía máy chủ, EMQX và MQTT Bridge phù hợp với bản tin nhỏ phát sinh liên tục từ thiết bị. Backend phụ trách xử lý nghiệp vụ, frontend phụ trách hiển thị cho người vận hành. Lớp lưu trữ được phân vai để dữ liệu nghiệp vụ, dữ liệu đo từ xa (telemetry) và log vận hành không lẫn vào nhau. Phương án này là cơ sở trực tiếp cho Chương 4, nơi các lựa chọn thiết kế được hiện thực thành nguyên mẫu và kiểm chứng theo chỉ tiêu nghiệm thu.

CHƯƠNG 4. TRIỂN KHAI GIẢI PHÁP VÀ KẾT QUẢ - IMPLEMENTATION AND RESULTS

Chương 4 trình bày quá trình hiện thực phương án đã chọn ở Chương 3. Nội dung được tổ chức theo đúng chuỗi vận hành của hệ thống: thiết bị trên xe, firmware, kết nối về máy chủ, lưu trữ dữ liệu và giao diện web quản lý. Mục tiêu của chương là chứng minh rằng các quyết định thiết kế không chỉ tồn tại ở mức phương án, mà đã được triển khai thành nguyên mẫu và kiểm tra theo các chỉ tiêu ở Chương 2.

4.1. Thiết kế chi tiết giải pháp - Detailed design solution

4.1.1. Thiết kế thiết bị trên xe

Thiết bị được triển khai trên bo mạch PCB riêng, gồm các khối đã chọn ở Chương 3: ESP32-S3, SIM7600CE-T, LIS3DH, mạch nguồn đa nhánh, pin dự phòng, khối đo điện áp và các đầu kết nối phục vụ nạp, kiểm tra và lắp đặt. Thiết kế PCB được tổ chức theo nguyên tắc tách tải lớn khỏi miền logic. Nhánh nguồn modem được bố trí riêng để

hạn chế ảnh hưởng của dòng xung khi SIM7600CE-T đăng ký mạng hoặc truyền dữ liệu.

Bảng 4.1: Các khối phần cứng trên thiết bị

Khối phần cứng	Nội dung triển khai	Vai trò kiểm chứng
ESP32-S3	Kết nối UART với SIM7600CE-T, I2C với LIS3DH, BLE với OBD2 và ADC đo nguồn	Kiểm tra khả năng điều phối nhiều ngoại vi.
SIM7600CE-T	Kết nối anten, UART, chân bật/tắt và reset	Kiểm tra truyền dữ liệu di động và lấy vị trí GNSS.
LIS3DH	Kết nối I2C và chân ngắt về vi điều khiển	Kiểm tra phát hiện rung/chuyển động khi xe đỗ.
Khối nguồn	MP2482, AP2112-3.3, TPS54231, TP5100, SX1308, pin 18650 1S và mạch chuyển nguồn	Kiểm tra ổn định nguồn, nguồn dự phòng và bảo vệ điện áp.
Đo nguồn	Cầu phân áp và ADC đo nguồn xe, pin dự phòng	Kiểm tra điều kiện chuyển trạng thái và tắt tải lớn.
Đầu kết nối	Nạp firmware, kiểm tra UART, nguồn và tín hiệu phụ	Phục vụ sửa lỗi, đo kiểm và hoàn thiện nguyên mẫu.

Bảng 4.2: Kết nối tín hiệu chính của thiết bị

Khối	Giao tiếp	Tín hiệu chính	Mục đích
SIM7600CE-T	UART + GPIO	TX, RX, PWRKEY, RESET	Điều khiển modem, lấy GNSS và gửi bản tin.
LIS3DH	I2C + interrupt	SDA, SCL, INT	Đọc gia tốc và đánh thức thiết bị khi có sự kiện.
Đo nguồn	ADC	VIN_SENSE, VBAT_SENSE	Theo dõi nguồn xe và pin dự phòng.
OBD2 BLE	BLE	Kết nối không dây	Đọc dữ liệu xe qua bộ chuyển đổi OBD2.

Thiết kế chi tiết của thiết bị bám sát yêu cầu lắp đặt ít xâm lấn. Dữ liệu OBD2 được lấy qua bộ chuyển đổi BLE, nguồn được lấy từ xe nhưng có nhánh bảo vệ, còn dữ liệu vị trí và trạng thái được truyền về máy chủ qua mạng di động.

4.1.2. Thiết kế firmware

Firmware được triển khai theo mô hình trạng thái đã chọn ở Chương 3. Thay vì để thiết bị luôn hoạt động ở một chế độ cố định, firmware xác định bối cảnh vận hành trước khi quyết định bật modem, đọc OBD2, gửi bản tin hoặc đưa thiết bị vào ngủ sâu. Cách tổ chức này phù hợp với yêu cầu vừa giám sát được xe, vừa hạn chế tiêu thụ điện khi xe đỗ.

 Hình 4.1 - Lưu đồ firmware của thiết bị

Hình 4.1: Lưu đồ firmware của thiết bị

Nguồn: Hình vẽ của tác giả

Bảng 4.3: Luồng xử lý chính trong firmware

Luồng xử lý	Nội dung triển khai	Kết quả cần đạt
Khởi động	Đọc cấu hình, kiểm tra nguồn, cảm biến, modem và bộ nhớ	Thiết bị sẵn sàng vào trạng thái vận hành phù hợp.
Thu thập dữ liệu	Lấy vị trí GNSS, dữ liệu OBD2 cơ bản, điện áp nguồn và trạng thái cảm biến	Có đủ trường dữ liệu để tạo bản tin.
Gửi dữ liệu	Đóng gói bản tin và gửi qua MQTT theo chủ đề đã quy ước	Dữ liệu xuất hiện trên máy chủ.
Quản lý năng lượng	Chuyển giữa chạy, đỗ, cảnh báo và ngủ sâu	Giảm tiêu thụ khi xe đỗ và bảo vệ ắc quy.
Xử lý cảnh báo	Đánh thức khi có rung/chuyển động, mất nguồn hoặc vượt vùng giám sát	Sự kiện bất thường được gửi về máy chủ.
Cấu hình từ xa	Nhận chu kỳ gửi, ngưỡng cảnh báo và lệnh cập nhật firmware	Thiết bị thay đổi hành vi mà không cần nạp lại thủ công.

4.1.3. Thiết kế máy chủ và giao diện web quản lý

Máy chủ được triển khai theo các khối chức năng đã xác định: máy chủ MQTT nhận bản tin, dịch vụ tiếp nhận bản tin chuẩn hóa dữ liệu, lớp lưu trữ ghi dữ liệu theo từng loại, máy chủ ứng dụng cung cấp API và giao diện web quản lý hiển thị thông tin cho người dùng.

 Hình 4.2 - Luồng dữ liệu từ thiết bị đến giao diện

Hình 4.2: Luồng dữ liệu từ thiết bị đến giao diện web quản lý

Nguồn: Hình vẽ của tác giả

Giao diện web quản lý tập trung vào bốn nhóm thông tin: bản đồ vị trí, trạng thái thiết bị, lịch sử hành trình và cảnh báo. Các thông tin này tương ứng trực tiếp với yêu cầu chức năng đã nêu ở Chương 2, đồng thời đủ để đánh giá nguyên mẫu ở cấp hệ thống.

4.2. Chế tạo, tích hợp và lắp ráp hệ thống - Manufacture and Assembly

Quá trình chế tạo được thực hiện theo từng khối để tránh nhầm lẫn giữa lỗi phần cứng, lỗi firmware và lỗi máy chủ. Trình tự tích hợp bắt đầu từ khối nguồn, sau đó đến vi điều khiển, modem, cảm biến, OBD2 BLE, kênh MQTT, máy chủ ứng dụng và cuối cùng là giao diện web quản lý.

Bảng 4.4: Trình tự chế tạo và kiểm tra khởi động

Bước	Nội dung thực hiện	Kết quả kiểm tra
1	Kiểm tra PCB, hướng linh kiện và mối hàn chính	Không sai linh kiện quan trọng, không ngắn mạch nguồn.
2	Cấp nguồn và kiểm tra các nhánh điện áp	5 V, 3,3 V, nhánh modem và nhánh dự phòng đạt mức thiết kế.
3	Nạp firmware khởi động cho ESP32-S3	Vi điều khiển khởi động và ghi nhật ký được.

Bước	Nội dung thực hiện	Kết quả kiểm tra
4	Kiểm tra SIM7600CE-T	Modem phản hồi lệnh AT, nhận mạng và lấy vị trí trong điều kiện phù hợp.
5	Kiểm tra LIS3DH	Đọc được gia tốc và tạo tín hiệu ngắt khi có chuyển động.
6	Ghép bộ chuyển đổi OBD2 BLE	Thiết bị phát hiện, kết nối và đọc dữ liệu cơ bản.
7	Gửi bản tin thử qua MQTT	Bản tin xuất hiện ở máy chủ MQTT và được dịch vụ tiếp nhận xử lý.
8	Kiểm tra giao diện web quản lý	Vị trí, trạng thái và cảnh báo được hiển thị.
9	Lắp thử trên xe	Kiểm tra nguồn, sóng di động, vị trí, dữ liệu OBD2 và cảnh báo trong điều kiện thực tế.

Cách tích hợp theo chuỗi giúp mỗi kết quả đo có điểm đối chiếu rõ ràng. Khi dữ liệu không xuất hiện trên giao diện, có thể kiểm tra lần lượt: thiết bị đã gửi bản tin chưa, máy chủ MQTT đã nhận chưa, dịch vụ tiếp nhận đã ghi dữ liệu chưa, máy chủ ứng dụng đã trả dữ liệu chưa và giao diện đã cập nhật chưa.

4.3. Đo lường và kết quả - Measurements and Results

4.3.1. Thiết lập môi trường thử nghiệm

Các bài thử được chia thành ba mức. Mức thứ nhất là thử trên bàn với nguồn DC để kiểm tra nguồn, modem, cảm biến và firmware. Mức thứ hai là thử tích hợp toàn tuyến từ thiết bị đến máy chủ và giao diện web quản lý. Mức thứ ba là lắp thử trên xe để kiểm tra nguồn thực tế, sóng di động, vị trí GNSS, kết nối OBD2 BLE và cảnh báo.

Bảng 4.5: Môi trường và dữ liệu kiểm thử

Nhóm thử nghiệm	Điều kiện kiểm tra	Dữ liệu ghi nhận
Nguồn	Cấp 12 V/24 V, ngắt nguồn chính, kiểm tra pin dự phòng	Điện áp nhánh nguồn, dòng tiêu thụ, trạng thái reset.
Thiết bị	Modem, GNSS, LIS3DH, OBD2 BLE và ADC đo nguồn	Nhật ký firmware, trạng thái kết nối, bản tin gửi đi.
Máy chủ	Nhận MQTT, chuẩn hóa và lưu dữ liệu	Nhật ký nhận bản tin, bản ghi cơ sở dữ liệu, lỗi xử lý nếu có.
Giao diện	Bản đồ, danh sách xe, cảnh báo và lịch sử	Thời điểm cập nhật, tính đúng của dữ liệu hiển thị.
Trên xe	Xe chạy, xe đỗ, thử rung/chuyển động và mất nguồn ngắn	Vị trí, OBD2, nguồn, cảnh báo và lịch sử hành trình.

4.3.2. Kết quả kiểm thử nguồn và phần cứng

Bảng 4.6: Kết quả kiểm thử nguồn và phần cứng

Hạng mục	Kết quả quan sát	Đánh giá
Nhánh nguồn 3,3 V logic	Cấp ổn định cho ESP32-S3 và LIS3DH trong bài thử	Đạt
Nhánh nguồn modem	Modem hoạt động mà không làm reset vi điều khiển trong bài thử truyền dữ liệu	Đạt
Dòng ngủ sâu	Đạt mục tiêu dưới 500 μ A trong cấu hình thử nghiệm	Đạt
Chuyển nguồn dự phòng	Thiết bị tiếp tục hoạt động khi ngắt nguồn chính trong thời gian ngắn	Đạt
Đo nguồn bằng ADC	Đọc được điện áp nguồn xe và pin dự phòng để firmware xử lý trạng thái	Đạt
Cảm biến chuyển động	Tạo được tín hiệu khi có rung/chuyển động vượt ngưỡng thử	Đạt

Kết quả này cho thấy kiến trúc nguồn đa nhánh là phù hợp. Nhánh modem được tách khỏi nhánh logic nên hạn chế được rủi ro sụt áp lan sang vi điều khiển. Nguồn dự

phòng và ADC đo nguồn cũng tạo cơ sở cho firmware xử lý các tình huống mất nguồn hoặc nguồn thấp.

4.3.3. Kết quả kiểm thử firmware và dữ liệu thiết bị

Bảng 4.7: Kết quả kiểm thử firmware

Hạng mục	Kết quả quan sát	Đánh giá
Khởi động thiết bị	Thiết bị đọc cấu hình và khởi tạo các khối chính	Đạt
GNSS	Lấy được tọa độ trong điều kiện có tín hiệu phù hợp	Đạt
OBD2 BLE	Kết nối được bộ chuyển đổi và đọc dữ liệu cơ bản	Đạt
Cảnh báo chuyển động	Sự kiện rung/chuyển động được phát hiện và đóng gói thành bản tin cảnh báo	Đạt
Chế độ ngủ sâu	Thiết bị vào và thoát ngủ sâu theo cấu hình	Đạt
Gửi bản tin MQTT	Dữ liệu được gửi theo chu kỳ và theo sự kiện	Đạt
Cấu hình từ xa	Thiết bị nhận được cấu hình và thay đổi tham số vận hành	Đạt ở mức nguyên mẫu

Firmware đã thực hiện được các nhiệm vụ cốt lõi: thu thập dữ liệu, quản lý trạng thái, gửi bản tin và phản ứng với sự kiện. Phạm vi OBD2 hiện dừng ở dữ liệu cơ bản, phù hợp với giới hạn đã nêu ở Chương 1 và Chương 2.

4.3.4. Kết quả kiểm thử máy chủ và giao diện web quản lý

Bảng 4.8: Kết quả kiểm thử máy chủ và giao diện

Hạng mục	Kết quả quan sát	Đánh giá
Nhận bản tin MQTT	Máy chủ MQTT nhận được bản tin từ thiết bị	Đạt
Dịch vụ tiếp nhận bản tin	Dữ liệu được kiểm tra, chuẩn hóa và đưa vào lớp lưu trữ	Đạt
Lưu dữ liệu	Dữ liệu vị trí, trạng thái và cảnh báo được lưu đúng nhóm	Đạt
API quản lý	Giao diện truy vấn được xe, trạng thái, cảnh báo và lịch sử	Đạt
Cập nhật giao diện	Dữ liệu mới xuất hiện trên giao diện web quản lý	Đạt
Cảnh báo	Cảnh báo thử nghiệm hiển thị theo xe tương ứng	Đạt

Kết quả cho thấy tuyến dữ liệu từ thiết bị đến người dùng đã hoạt động. Đây là kết quả quan trọng nhất của nguyên mẫu vì nó chứng minh các khối phần cứng, firmware, truyền thông, máy chủ và giao diện có thể phối hợp thành một hệ thống hoàn chỉnh.

4.3.5. Đối chiếu với chỉ tiêu nghiệm thu

Bảng 4.9: Đối chiếu kết quả với chỉ tiêu nghiệm thu ở Chương 2

Chỉ tiêu	Kết quả kiểm tra	Đánh giá
Hoạt động với nguồn 12 V/24 V	Thiết bị hoạt động trong bài thử nguồn	Đạt
Dòng ngủ sâu mục tiêu dưới 500 μ A	Đạt trong cấu hình thử nghiệm	Đạt
Duy trì khi mất nguồn chính ngắn hạn	Pin dự phòng duy trì thiết bị trong bài thử	Đạt
Thu vị trí GNSS	Có tọa độ và thời điểm cập nhật	Đạt
Đọc dữ liệu OBD2 cơ bản	Đọc được qua bộ chuyển đổi BLE	Đạt
Gửi dữ liệu thiết bị - máy chủ	Bản tin xuất hiện trên máy chủ và giao diện	Đạt

Chỉ tiêu	Kết quả kiểm tra	Đánh giá
Phát sinh cảnh báo	Giao diện nhận được cảnh báo thử nghiệm	Đạt
Cập nhật cấu hình thiết bị	Thiết bị nhận cấu hình và thay đổi tham số tương ứng	Đạt ở mức nguyên mẫu

4.3.6. Kết luận chương 4

Nguyên mẫu đã hoàn thành chuỗi chức năng chính từ thiết bị trên xe đến giao diện web quản lý. Thiết bị thu được vị trí, dữ liệu OBD2 cơ bản, trạng thái nguồn và cảnh báo; dữ liệu được gửi về máy chủ, lưu trữ và hiển thị cho người vận hành. Những nội dung cần kiểm chứng thêm gồm độ bền dài ngày, độ ổn định trên nhiều dòng xe, chất lượng sóng di động ở các khu vực khác nhau và độ bền cơ khí của thiết bị khi lắp đặt thực tế.

CHƯƠNG 5. ĐÁNH GIÁ VÀ KHUYẾN NGHỊ - EVALUATION AND RECOMMENDATIONS

5.1. Đánh giá hiệu năng

5.1.1. Đánh giá chung

Kết quả ở Chương 4 cho thấy hệ thống đạt mục tiêu của một nguyên mẫu IoT quản lý phương tiện. Các khối chính đều đã được hiện thực và kiểm tra: thiết bị trên xe hoạt động được, firmware điều phối được trạng thái, dữ liệu được gửi qua MQTT về máy chủ, giao diện web quản lý hiển thị được vị trí, trạng thái, lịch sử và cảnh báo.

Bảng 5.1: Đánh giá hiệu năng theo nhóm chức năng

Nhóm chức năng	Kết quả đạt được	Giới hạn hiện tại
Phần cứng	PCB tích hợp MCU, modem, cảm biến, nguồn đa nhánh và pin dự phòng	Cần kiểm chứng nhiều, nhiệt và độ bền dài ngày trên nhiều vị trí lắp.
Nguồn	Tách nhánh modem, có ngủ sâu, đo nguồn và nguồn dự phòng	Cần thử dài hơn với nhiều loại ắc quy và điều kiện nhiệt độ khác nhau.
Firmware	Điều phối được trạng thái chạy, đỗ, cảnh báo, gửi dữ liệu và ngủ sâu	Cần tối ưu thêm khi mất mạng kéo dài hoặc OBD2 BLE không ổn định.
Dữ liệu xe	Đọc được OBD2 cơ bản qua BLE	Chưa đi sâu vào toàn bộ mã lỗi và thông số nâng cao.
Truyền dữ liệu	MQTT phù hợp với bản tin nhỏ và cảnh báo	Cần thử nghiệm ở nhiều vùng sóng yếu.
Máy chủ và giao diện	Nhận, lưu và hiển thị được vị trí, trạng thái, cảnh báo và lịch sử	Cần thử tải dài hơn khi số lượng xe tăng.

5.1.2. Nhận xét theo yêu cầu thiết kế

Về yêu cầu theo dõi phương tiện, hệ thống đã chuyển được dữ liệu từ xe thành thông tin hiển thị trên giao diện web quản lý. Về yêu cầu năng lượng, thiết bị đã có cơ chế ngủ sâu và tách nhánh nguồn cho modem. Về yêu cầu lắp đặt ít xâm lấn, OBD2 BLE giúp giảm dây nối trực tiếp vào hệ chẩn đoán của xe. Về yêu cầu khai thác dữ liệu, máy chủ đã phân biệt dữ liệu nghiệp vụ, dữ liệu theo thời gian và nhật ký vận hành.

Giới hạn lớn nhất của nguyên mẫu nằm ở mức độ kiểm chứng thực địa. Các bài thử hiện tại chứng minh tính đúng của phương án, nhưng chưa đủ để kết luận về độ bền trong nhiều tháng vận hành hoặc khả năng tương thích với mọi dòng xe.

5.2. Đánh giá kinh tế và môi trường

5.2.1. Đánh giá kinh tế

Thiết kế tự chủ phần cứng và phần mềm giúp giảm phụ thuộc vào nền tảng thương mại. Các linh kiện chính như ESP32-S3, SIM7600CE-T, LIS3DH, TP5100, pin 18650 và bộ chuyển đổi OBD2 BLE đều là các thành phần phổ biến, có tài liệu kỹ thuật và có thể mua được trên thị trường. Điều này phù hợp với mục tiêu nguyên mẫu và tạo điều kiện cho các lần cải tiến tiếp theo.

Bảng 5.2: Nhóm chi phí chính của hệ thống

Nhóm chi phí	Thành phần	Nhận xét
Thiết bị trên xe	PCB, ESP32-S3, SIM7600CE-T, LIS3DH, nguồn, pin, vỏ	Chiếm phần chi phí chính của nguyên mẫu.
Phụ kiện lắp đặt	Dây nguồn, cầu chì, đầu nối, anten, vỏ bảo vệ	Cần chuẩn hóa nếu lắp trên nhiều xe.
Hạ tầng máy chủ	Máy chủ, tên miền, lưu trữ dữ liệu	Tăng theo số xe, tần suất gửi và thời gian lưu lịch sử.
Bảo trì	Thay bộ chuyển đổi OBD2, kiểm tra pin, cập nhật firmware	Cần tính khi chuyển sang thử nghiệm thí điểm.

5.2.2. Đánh giá môi trường

Hệ thống có thể hỗ trợ sử dụng đội xe hiệu quả hơn thông qua theo dõi hành trình, phát hiện sử dụng sai phạm vi và hỗ trợ bảo trì sớm. Ở chiều ngược lại, thiết bị điện tử và pin dự phòng tạo ra rác thải nếu thay thế không đúng quy trình. Vì vậy, hướng phát triển tiếp theo cần ưu tiên vỏ bền, pin có nguồn gốc rõ ràng, thiết kế dễ tháo lắp và phương án thu hồi linh kiện hỏng.

5.3. Đánh giá rủi ro và biện pháp giảm thiểu

Bảng 5.3: Rủi ro kỹ thuật và biện pháp giảm thiểu

Rủi ro	Mức ảnh hưởng	Biện pháp đã dùng	Hướng bổ sung
Sụt áp khi modem truyền dữ liệu	Cao	Tách nhánh nguồn modem và logic	Đo dao động nguồn bằng thiết bị đo tốt hơn trong nhiều kịch bản.
Hao ắc quy khi xe đỗ lâu	Cao	Ngủ sâu, đo điện áp bằng ADC và tắt tải lớn bằng firmware	Thử dài ngày trên nhiều xe và nhiều loại ắc quy.
Mất sóng di động	Trung bình - cao	Có cơ chế phục hồi kết nối và gửi lại bản tin	Bổ sung bộ đệm dữ liệu cục bộ cho tình huống mất mạng dài.
Kết nối OBD2 BLE không ổn định	Trung bình	Tự kết nối lại bộ chuyển đổi	Chuẩn hóa danh sách bộ chuyển đổi tương thích.
Sai lệch vị trí GNSS	Trung bình	Ghi nhận vị trí và đối chiếu hành trình	Kết hợp lọc dữ liệu và thử nghiệm ở khu vực đô thị dày đặc.
Lộ dữ liệu vị trí	Cao	Có xác thực và phân quyền ở giao diện	Bổ sung mã hóa kênh truyền và chính sách lưu dữ liệu rõ ràng.
Lỗi cập nhật firmware	Cao	Chỉ cập nhật khi đủ điều kiện nguồn và kết nối	Bổ sung cơ chế quay lại phiên bản trước nếu cập nhật lỗi.

5.4. Khuyến nghị cho tương lai

Các hướng phát triển tiếp theo nên ưu tiên theo mức độ ảnh hưởng đến khả năng triển khai thực tế.

- Kiểm thử thực địa dài ngày:** lắp thiết bị trên nhiều xe, nhiều vị trí và nhiều điều kiện sóng di động khác nhau để đánh giá độ bền.
- Hoàn thiện vỏ và gá lắp:** thiết kế vỏ kín, đầu nối chắc chắn, cầu chì bảo vệ và phương án cố định thiết bị trong xe.

- 3. **Mở rộng dữ liệu OBD2:** đọc thêm mã lỗi DTC và các thông số có giá trị cho bảo trì.
- 4. **Tăng an toàn dữ liệu:** triển khai mã hóa kênh thiết bị - máy chủ, phân quyền chặt hơn và chính sách lưu trữ vị trí rõ ràng.
- 5. **Tối ưu năng lượng:** đo dòng ở nhiều chế độ hơn, tối ưu chu kỳ gửi và điều chỉnh chiến lược ngủ theo trạng thái xe.
- 6. **Hoàn thiện ứng dụng di động:** bổ sung thông báo cảnh báo nhanh cho người quản lý khi không sử dụng máy tính.
- 7. **Chuẩn hóa thử nghiệm thí điểm:** xây dựng quy trình lắp đặt, kiểm tra sau lắp, nhật ký bảo trì và tài liệu hướng dẫn vận hành.

CHƯƠNG 6. PHẢN HỒI VÀ BÀI HỌC KINH NGHIỆM - REFLECTION AND LESSONS LEARNED

6.1. Ứng dụng kiến thức kỹ thuật - Application of prior coursework

Đồ án vận dụng kiến thức của nhiều nhóm học phần trong ngành Kỹ thuật Cơ điện tử. Phần điện - điện tử được sử dụng khi thiết kế nguồn, chọn IC ổn áp, tách nhánh tải và kiểm tra dòng tiêu thụ. Phần hệ thống nhúng được sử dụng trong lập trình ESP32-S3, giao tiếp UART, I2C, BLE, quản lý trạng thái và xử lý ngắt cảm biến. Phần đo lường được sử dụng khi kiểm tra nguồn, dòng ngủ sâu, tín hiệu cảm biến, bản tin truyền dữ liệu và phản ứng của hệ thống khi có cảnh báo.

Bảng 6.1: Kiến thức đã áp dụng trong đồ án

Nhóm kiến thức	Ứng dụng trong đồ án
Mạch điện tử	Thiết kế nguồn, đo điện áp, bảo vệ nguồn và bố trí các nhánh tải.
Vi điều khiển	Điều khiển ESP32-S3, modem, cảm biến, OBD2 BLE và chế độ ngủ sâu.
Cảm biến	Đọc gia tốc và tạo ngắt khi có chuyển động bất thường.

Nhóm kiến thức	Ứng dụng trong đồ án
Truyền thông	Gửi dữ liệu qua mạng di động, MQTT và cập nhật giao diện web quản lý.
Cơ sở dữ liệu	Lưu dữ liệu xe, vị trí, cảnh báo, lịch sử và nhật ký vận hành.
Kiểm thử hệ thống	Đối chiếu kết quả từng khối với chỉ tiêu nghiệm thu.

6.2. Giải quyết các vấn đề kỹ thuật phức tạp - Complex engineering problems

Khó khăn chính của đề tài nằm ở sự phụ thuộc giữa các khối. Nếu nguồn không ổn định, firmware có thể reset. Nếu firmware không quản lý trạng thái tốt, thiết bị tiêu thụ nhiều điện khi xe đỗ. Nếu truyền dữ liệu không ổn định, giao diện web quản lý không phản ánh đúng trạng thái xe. Vì vậy, quá trình thiết kế phải xét toàn tuyến thay vì tối ưu từng khối riêng lẻ.

Bảng 6.2: Vấn đề kỹ thuật và cách xử lý

Vấn đề	Cách xử lý trong đồ án	Bài học rút ra
Modem tạo tải lớn	Tách nhánh nguồn modem khỏi nhánh logic	Thiết kế nguồn phải xét tải động, không chỉ dòng trung bình.
Xe đỗ lâu nhưng vẫn cần giám sát	Dùng LIS3DH, ngắt đánh thức và ngủ sâu	Tiết kiệm năng lượng phải đi kèm cơ chế đánh thức đúng lúc.
Lắp trên nhiều xe	Dùng OBD2 BLE và hạn chế can thiệp dây	Tính cơ động khi lắp đặt là một tiêu chí kỹ thuật quan trọng.
Mạng di động không ổn định	Dùng MQTT, phục hồi kết nối và phân mức tin cậy bản tin	Hệ thống thực tế phải chấp nhận mất kết nối tạm thời.
Dữ liệu nhiều loại	Tách dữ liệu nghiệp vụ, dữ liệu theo thời gian và nhật ký	Cách tổ chức dữ liệu ảnh hưởng trực tiếp đến khả năng khai thác.

Vấn đề	Cách xử lý trong đồ án	Bài học rút ra
Giao diện cho người vận hành	Chuyển bản tin kỹ thuật thành bản đồ, trạng thái, cảnh báo và lịch sử	Giao diện phải phục vụ quyết định vận hành, không chỉ hiển thị dữ liệu thô.

6.3. Tác động đạo đức và xã hội - Ethical and social impacts

Hệ thống giám sát phương tiện có giá trị rõ đối với doanh nghiệp cho thuê xe, nhưng cũng liên quan trực tiếp đến dữ liệu vị trí và hành trình của người sử dụng xe. Vì vậy, việc thu thập dữ liệu cần gắn với mục tiêu quản lý hợp lý, có thông báo rõ cho khách thuê và giới hạn quyền truy cập theo vai trò. Dữ liệu vị trí không nên được xem như dữ liệu kỹ thuật thông thường, mà là dữ liệu nhạy cảm cần có chính sách lưu trữ, truy cập và xóa bỏ phù hợp.

Trong phạm vi đồ án, thiết bị không can thiệp trực tiếp vào hệ thống điều khiển xe. Chức năng hiện tại dừng ở giám sát, cảnh báo và hỗ trợ ra quyết định. Cách tiếp cận này giảm rủi ro an toàn khi thử nghiệm trên xe thật, đồng thời vẫn đáp ứng mục tiêu quản lý phương tiện ở mức nguyên mẫu.

Ở góc độ xã hội, hệ thống có thể giúp doanh nghiệp nhỏ quản lý xe tốt hơn, giảm tranh chấp về hành trình, hỗ trợ bảo trì và giảm rủi ro mất tài sản. Nếu phát triển thành sản phẩm triển khai rộng, các vấn đề về bảo mật, quyền riêng tư, trách nhiệm khi cảnh báo sai và quy trình xử lý sự cố cần được hoàn thiện trước khi vận hành chính thức.

6.4. Tổng kết và bài học kinh nghiệm - Reflection and Lessons Learned

Đồ án đã hình thành được một nguyên mẫu hệ thống IoT quản lý phương tiện gồm thiết bị phần cứng, firmware, máy chủ và giao diện web quản lý. Kết quả quan trọng nhất là chuỗi dữ liệu từ xe đến người quản lý đã vận hành được trong các bài thử chính. Thiết bị có thể thu vị trí, trạng thái nguồn, dữ liệu OBD2 cơ bản và cảnh báo; máy chủ tiếp nhận và lưu dữ liệu; giao diện web quản lý hiển thị thông tin cần thiết cho người vận hành.

Các bài học chính rút ra gồm:

- Thiết kế nguồn là nền tảng của thiết bị lắp trên xe; nếu nguồn không ổn định, các lớp phần mềm phía sau không thể đánh giá đúng.
- Lựa chọn linh kiện cần xét khả năng phối hợp trong hệ thống, không chỉ dựa vào thông số riêng lẻ.
- Kiểm thử theo từng khối giúp khoanh vùng lỗi nhanh hơn so với ghép toàn bộ rồi mới kiểm tra.
- Kết nối mạng di động cần được xem là điều kiện thay đổi, không phải giả định luôn ổn định.
- Giao diện web quản lý phải ưu tiên bản đồ, trạng thái thiết bị, cảnh báo và lịch sử hành trình; dữ liệu kỹ thuật nội bộ cần được chuyển thành chỉ báo dễ hiểu cho người vận hành.
- Một nguyên mẫu tốt cần nêu rõ giới hạn hiện tại để có hướng phát triển tiếp theo.

Trong phạm vi đồ án tốt nghiệp, hệ thống đã đạt mục tiêu thiết kế nguyên mẫu. Các bước tiếp theo nên tập trung vào kiểm thử dài ngày, hoàn thiện vỏ thiết bị, tăng an toàn dữ liệu và chuẩn hóa quy trình lắp đặt trên xe thật.

TÀI LIỆU TRÍCH DẪN - REFERENCES

TÀI LIỆU TRÍCH DẪN - REFERENCES

TÀI LIỆU TRÍCH DẪN - REFERENCES

TÀI LIỆU TRÍCH DẪN - REFERENCES

TÀI LIỆU TRÍCH DẪN - REFERENCES

[A] Tài liệu tiếng Việt

[1] IMARC Group, "Vietnam Car Rental Market Size, Share, Trends and Forecast by Booking Type, Rental Duration, Application Type, and Region 2026–2034," 2026.

[Online]. Available: <https://www.imarcgroup.com/vietnam-car-rental-market>. [Accessed: Apr. 24, 2026].

[2] Texas Instruments, "Power Management for IoT Devices," Application Report SLVAE57, 2020. [Online]. Available: <https://www.ti.com/lit/an/slvae57/slvae57.pdf>.

[3] M. A. Al-Khedher, "Hybrid GPS-GSM Localization of Automobile Tracking System," *International Journal of Computer Science and Information Technology*, vol. 3, no. 6, pp. 75–85, Dec. 2011.

[4] N. Naik, "Choice of Effective Messaging Protocols for IoT Systems: MQTT, CoAP, AMQP and HTTP," in *Proc. 2017 IEEE International Systems Engineering Symposium (ISSE)*, Vienna, Austria, 2017, pp. 1–7.

[B] Tài liệu tiếng Anh (Books & Papers)

[5] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 4, pp. 2347–2376, Fourth Quarter 2015.

[6] N. Naik, "Choice of Effective Messaging Protocols for IoT Systems: MQTT, CoAP, AMQP and HTTP," in *Proc. 2017 IEEE International Systems Engineering Symposium (ISSE)*, Vienna, Austria, 2017, pp. 1–7.

[7] M. Quigley, R. Johnson, and P. Sharma, "Power Management Strategies for IoT Vehicle Tracking Devices," *IEEE Internet of Things Journal*, vol. 8, no. 15, pp. 12045–12058, Aug. 2021.

[8] M. A. Al-Khedher, "Hybrid GPS-GSM Localization of Automobile Tracking System," *International Journal of Computer Science and Information Technology (IJCSIT)*, vol. 3, no. 6, pp. 75–85, Dec. 2011.

[9] S. Kaur and S. Singh, "A Survey on IoT based Vehicle Tracking System," *International Journal of Advanced Research in Computer Science*, vol. 10, no. 3, pp. 42–47, 2019.

[10] R. Barry, *Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide*. Real Time Engineers Ltd., 2016.

- [11] B. Boehm and V. R. Basili, "Software Defect Reduction Top 10 List," *IEEE Computer*, vol. 34, no. 1, pp. 135–137, Jan. 2001.
- [12] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA, USA: Addison-Wesley Professional, 2003.
- [13] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.
- [14] D. Guinard and V. Trifa, *Building the Web of Things*. Shelter Island, NY, USA: Manning Publications, 2016.
- [15] J. Gubbi, R. Buyya, S. Marusic, and M. Palaniswami, "Internet of Things (IoT): A Vision, Architectural Elements, and Future Directions," *Future Generation Computer Systems*, vol. 29, no. 7, pp. 1645–1660, Sep. 2013.
- [16] V. C. Gungor and G. P. Hancke, "Industrial Wireless Sensor Networks: Challenges, Design Principles, and Technical Approaches," *IEEE Transactions on Industrial Electronics*, vol. 56, no. 10, pp. 4258–4265, Oct. 2009.
- [17] R. K. Kodali, V. Jain, S. Bose, and L. Boppana, "IoT Based Smart Security and Home Automation System," in *Proc. 2016 International Conference on Computing, Communication and Automation (ICCCA)*, Greater Noida, India, 2016, pp. 1286–1289.
- [18] B. Sahu and G. A. Rincon-Mora, "A Low Voltage, Dynamic, Noninverting, Synchronous Buck-Boost Converter for Portable Applications," *IEEE Transactions on Power Electronics*, vol. 19, no. 2, pp. 443–452, Mar. 2004.
-

[C] Tài liệu kỹ thuật (Datasheets & Technical Documents)

- [19] Espressif Systems, "ESP32-S3 Series Datasheet," Version 1.3, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf
- [20] Espressif Systems, "ESP32-S3 Technical Reference Manual," Version 1.1, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-s3_technical_reference_manual_en.pdf

- [21] SIMCom Wireless Solutions, "SIM7600 Series AT Command Manual," Version 1.06, 2023. [Online]. Available: <https://www.simcom.com/product/SIM7600CE-T.html>
- [22] SIMCom Wireless Solutions, "SIM7600CE-T Module Hardware Design Guide," Version 1.02, 2022.
- [23] STMicroelectronics, "LIS3DH MEMS digital output motion sensor ultra-low-power high-performance 3-axis 'nano' accelerometer," Datasheet. [Online]. Available: <https://www.st.com/resource/en/datasheet/lis3dh.pdf>
- [24] STMicroelectronics, "AN3308: LIS3DH MEMS digital output motion sensor ultra-low-power high-performance 3-axis 'nano' accelerometer," Application Note. [Online]. Available: https://www.st.com/resource/en/application_note/cd00290365.pdf
- [25] OASIS, "MQTT Version 5.0 - OASIS Standard," Mar. 2019. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [26] SAE International, "SAE J1979 - E/E Diagnostic Test Modes," Standard, Rev. 2014.
- [27] International Organization for Standardization, "ISO 15031–5:2015 - Road vehicles - Communication between vehicle and external equipment for emissions-related diagnostics," ISO Standard, 2015.
- [28] International Organization for Standardization, "ISO 15765–2:2016 - Road vehicles - Diagnostic communication over Controller Area Network (DoCAN) - Part 2: Transport protocol and network layer services," ISO Standard, 2016.
- [29] vgate, "iCar Pro BLE OBD2 Adapter Specifications," 2023. [Online]. Available: <https://www.vgatemall.com/product/vgate-icar-pro.html>
- [30] NanJing Top Power ASIC Corp., "TP5100 2A Switching Buck Charger IC for 8.4V/4.2V Lithium-Ion Battery," Datasheet, Rev. 2.4. [Online]. Available: <https://www.toppwr.com/uploadfile/file/20230304/6402f53e7e8d9.pdf>. [Accessed: Apr. 18, 2026].
- [31] Apache NimBLE Project, "NimBLE Host API Reference," Version 1.5, 2023. [Online]. Available: <https://mynewt.apache.org/latest/network/>
-

[D] Tài liệu trực tuyến (Online Resources)

[32] EMQX Team, "EMQX Documentation - The World's Most Scalable MQTT Broker," Version 5.x, 2024. [Online]. Available: <https://docs.emqx.com/en/emqx/latest/>. [Accessed: Jan. 15, 2026].

[33] The PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," 2024. [Online]. Available: <https://www.postgresql.org/docs/16/>. [Accessed: Jan. 10, 2026].

[34] VictoriaMetrics Team, "VictoriaMetrics Documentation," 2024. [Online]. Available: <https://docs.victoriametrics.com/>. [Accessed: Jan. 10, 2026].

[35] VictoriaMetrics Team, "VictoriaLogs Documentation," 2024. [Online]. Available: <https://docs.victoriametrics.com/victorialogs/>. [Accessed: Jan. 10, 2026].

[36] OpenJS Foundation, "Express.js - Fast, Unopinionated, Minimalist Web Framework for Node.js," 2024. [Online]. Available: <https://expressjs.com/>. [Accessed: Dec. 20, 2025].

[37] Vercel Inc., "Next.js Documentation," 2026. [Online]. Available: <https://nextjs.org/docs>. [Accessed: Apr. 24, 2026].

[38] Meta Platforms, Inc., "React 19 Documentation," 2024. [Online]. Available: <https://react.dev/>. [Accessed: Dec. 15, 2025].

[39] Socket.IO Contributors, "Socket.IO Documentation," Version 4.8, 2024. [Online]. Available: <https://socket.io/docs/v4/>. [Accessed: Dec. 20, 2025].

[40] V. Agafonkin, "Leaflet - An Open-Source JavaScript Library for Mobile-Friendly Interactive Maps," Version 1.9, 2024. [Online]. Available: <https://leafletjs.com/>. [Accessed: Jan. 05, 2026].

[41] Recharts Group, "Recharts Documentation," 2026. [Online]. Available: <https://recharts.github.io/en-US/guide/>. [Accessed: Apr. 24, 2026].

[42] Docker Inc., "Docker Documentation," 2024. [Online]. Available: <https://docs.docker.com/>. [Accessed: Nov. 15, 2025].

- [43] Docker Inc., "Docker Compose Documentation," 2024. [Online]. Available: <https://docs.docker.com/compose/>. [Accessed: Nov. 15, 2025].
- [44] Tailwind Labs, "Tailwind CSS Documentation," Version 4, 2025. [Online]. Available: <https://tailwindcss.com/docs>. [Accessed: Jan. 10, 2026].
- [45] Daishi Kato, "Zustand - Bear Necessities for State Management in React," Version 5, 2024. [Online]. Available: <https://zustand-demo.pmnd.rs/>. [Accessed: Dec. 20, 2025].
- [46] TanStack, "TanStack Query Documentation," 2026. [Online]. Available: <https://tanstack.com/query/latest>. [Accessed: Apr. 24, 2026].
- [47] Espressif Systems, "ESP-IDF Programming Guide," Version 5.x, 2024. [Online]. Available: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/>. [Accessed: Oct. 10, 2025].
- [48] Grafana Labs, "Grafana Documentation," 2024. [Online]. Available: <https://grafana.com/docs/grafana/latest/>. [Accessed: Jan. 15, 2026].
- [49] Prometheus Authors, "Prometheus Documentation," 2024. [Online]. Available: <https://prometheus.io/docs/>. [Accessed: Jan. 15, 2026].
- [50] Radix UI Contributors, "Radix UI - Unstyled, Accessible Components for React," 2024. [Online]. Available: <https://www.radix-ui.com/>. [Accessed: Dec. 15, 2025].
- [51] Google, "Flutter Documentation - WebView Plugin," 2024. [Online]. Available: <https://docs.flutter.dev/>. [Accessed: Jan. 20, 2026].
- [52] Texas Instruments, "Power Management for IoT Devices," Application Report SLVAE57, 2020. [Online]. Available: <https://www.ti.com/lit/an/slvae57/slvae57.pdf>. [Accessed: Sep. 15, 2025].
- [53] STMicroelectronics, "STM32L476RE Product Page," 2026. [Online]. Available: <https://www.st.com/en/microcontrollers-microprocessors/stm32l476re.html>. [Accessed: Mar. 02, 2026].
- [54] STMicroelectronics, "STM32L476xx Datasheet," DS10198 Rev 11, Jul. 2024. [Online]. Available: <https://www.st.com/resource/en/datasheet/stm32l476re.pdf>. [Accessed: Mar. 02, 2026].
- [55] Nordic Semiconductor, "nRF52840 Product Specification v1.2," [Online]. Available: https://docs.nordicsemi.com/bundle/nRF52840_PS_v1.2/resource/nRF52840_PS_v1.2

.pdf. [Accessed: Mar. 02, 2026].

- [56] Nordic Semiconductor, "nRF52840 Product Page," 2026. [Online]. Available: <https://www.nordicsemi.com/Products/nRF52840/Modules>. [Accessed: Mar. 02, 2026].
- [57] Nordic Semiconductor, "nRF52840 Documentation Hub," 2026. [Online]. Available: <https://docs.nordicsemi.com/category/nrf52840-category>. [Accessed: Mar. 02, 2026].
- [58] SIMCom Wireless Solutions, "A7670X Series Product Page," 2026. [Online]. Available: <https://cn.simcom.com/product/A7670X.html>. [Accessed: Mar. 02, 2026].
- [59] Quectel Wireless Solutions, "EC200U Series Product Page," 2026. [Online]. Available: <https://www.quectel.com/product/lte-ec200u-series>. [Accessed: Mar. 02, 2026].
- [60] SIMCom Wireless Solutions, "SIM7600CE Product Page," 2026. [Online]. Available: <https://en.simcom.com/product/SIM7600CE.html>. [Accessed: Mar. 13, 2026].
- [61] Quectel Wireless Solutions, "Quectel EC200U Series LTE Standard Specification," Version 1.4, 2024. [Online]. Available: https://developer.quectel.com/en/wp-content/uploads/sites/2/2024/11/Quectel_EC200U_Series_LTE_Standard_Specification_V1.4.pdf. [Accessed: Mar. 02, 2026].
- [62] u-blox, "NEO-M8 Data Sheet," UBX-15031086, [Online]. Available: https://www.u-blox.com/sites/default/files/NEO-M8-FW3_DataSheet_UBX-15031086.pdf. [Accessed: Mar. 02, 2026].
- [63] u-blox, "NEO-M8 Series Product Page," 2026. [Online]. Available: <https://www.u-blox.com/en/product/neo-m8-series>. [Accessed: Mar. 02, 2026].
- [64] vgate, "Vgate iCar Pro BLE Product Page," 2026. [Online]. Available: <https://vgatemall.com/products-detail/i-9/>. [Accessed: Mar. 02, 2026].
- [65] Texas Instruments, "TPS2115A Product Page," 2026. [Online]. Available: <https://www.ti.com/product/TPS2115A>. [Accessed: Mar. 02, 2026].
- [66] Infineon Technologies, "IRLML6402 Product Page," 2026. [Online]. Available: <https://www.infineon.com/part/IRLML6402>. [Accessed: Mar. 02, 2026].
- [67] SONGLE Relay, "SRD-05VDC-SL-C Relay Datasheet," datasheet mirror. [Online]. Available: <https://www.alldatasheet.com/html-pdf/1132639/SONGLERELAY/SRD-05VDC-SL-C/1713/2/SRD-05VDC-SL-C.html>. [Accessed: Mar. 02, 2026].

PHỤ LỤC - APPENDICES

PHỤ LỤC 1: BÁO CÁO TÀI CHÍNH - FINANCE REPORT

1.1. Bảng kê chi phí linh kiện (Bill of Materials - BOM)

Nhóm linh kiện	Chi phí ước tính	Ghi chú
Khối xử lý và truyền thông	Mức cao nhất trong BOM	Bao gồm MCU, modem LTE/GNSS và bộ chuyển đổi OBD2
Khối nguồn và bảo vệ	Mức trung bình	Bao gồm buck/LDO/sạc pin và linh kiện bảo vệ
Khối cơ khí và phụ trợ	Mức thấp - trung bình	Vỏ hộp, dây nối, linh kiện lắp đặt

1.2. Chi phí hạ tầng đám mây (ước tính hàng tháng)

Chi phí hạ tầng đám mây thay đổi theo số lượng thiết bị hoạt động đồng thời. Ở quy mô thí điểm, chi phí duy trì nằm trong ngưỡng phù hợp với mô hình đồ án.

1.3. Tổng hợp chi phí dự án

Tổng chi phí dự án được kiểm soát tốt nhờ tận dụng công nghệ mã nguồn mở và kiến trúc có thể mở rộng theo từng giai đoạn.

Quy mô	Số thiết bị	Chi phí vận hành/tháng	Nhận xét
Thử nghiệm thí điểm nội bộ	5–20	Thấp	Tập trung kiểm chứng kỹ thuật và quy trình vận hành
Thí điểm mở rộng	20–100	Trung bình	Cần tăng giám sát và chuẩn hóa cảnh báo
Triển khai dịch vụ	100–300	Trung bình - cao	Bắt đầu cần tối ưu khâu tiếp nhận dữ liệu và giao diện thời gian thực
Đội xe lớn	300–1000	Cao	Yêu cầu kế hoạch scale ngang theo lớp dịch vụ
Đa khu vực	>1000	Rất cao	Cần tách cụm vận hành, chính sách dữ liệu và quan sát tập trung

PHỤ LỤC 2: CÁC TIÊU CHUẨN THIẾT KẾ - STANDARDS

2.1. Tiêu chuẩn OBD2 (SAE J1979 / ISO 15031–5)

Áp dụng làm nền cho thiết kế lớp thu thập dữ liệu vận hành xe và chuẩn hóa cách giải nghĩa các PID quan trọng.

2.2. MQTT 3.1.1 trong triển khai thiết bị

Áp dụng cho lớp truyền thông thời gian thực giữa thiết bị và máy chủ, với quy ước **topic** và QoS theo mức ưu tiên nghiệp vụ.

2.3. Bảo mật thông tin (ISO 27001 - tham khảo)

Áp dụng ở mức nguyên tắc: kiểm soát truy cập, tách quyền theo vai trò và truy vết vận hành.

2.4. Thiết kế REST API (RFC 7231 và thực hành tốt)

Áp dụng cho cấu trúc endpoint, mã phản hồi và chuẩn hóa định dạng lỗi trong API quản trị.

Chuẩn/nguyên tắc	Phạm vi áp dụng	Cách hiện thực trong dự án
SAE J1979 / ISO 15031	Dữ liệu OBD2	Chọn tập PID cốt lõi, chuẩn hóa bộ phân tích và mô tả đơn vị đo
MQTT 3.1.1	Kênh truyền thiết bị – máy chủ	Tách topic theo nghiệp vụ, gán QoS theo mức ưu tiên dữ liệu
ISO 27001 (tham khảo)	Vận hành và bảo mật	Tách quyền truy cập, ghi nhật ký vận hành và theo dõi phiên truy cập
RFC 7231 + thực hành tốt của REST	HTTP/REST API của máy chủ ứng dụng	Chuẩn hóa phương thức/mã trạng thái, phản hồi lỗi nhất quán
Nguyên tắc đặc quyền tối thiểu	Quản trị hệ thống	Giới hạn vai trò theo chức năng vận hành và bảo trì

2.5. Ma trận kiểm soát tuân thủ chuẩn theo thành phần

ID	Chuẩn/nguyên tắc	Thành phần áp dụng	Cách kiểm chứng trong vận hành
S-01	SAE J1979	Bộ giải mã OBD2	Đối chiếu PID trả về với mô tả tiêu chuẩn và kiểm tra đơn vị đo hợp lệ
S-02	ISO 15031–5	Luồng chẩn đoán xe	Kiểm tra tính tương thích bản tin giữa các nhóm xe thử nghiệm
S-03	MQTT 3.1.1	Kênh dữ liệu vận hành chính	Kiểm tra publish/subscribe theo topic và hành vi kết nối lại khi mất mạng
S-04	Chính sách QoS của MQTT	Topic cảnh báo ưu tiên cao	Đo tỷ lệ mất bản tin trong kịch bản mạng dao động
S-05	Quy ước đặt tên topic	MQTT broker + MQTT Bridge + máy chủ ứng dụng	Soát nhật ký đảm bảo topic phát sinh đúng miền nghiệp vụ
S-06	Kiểm tra cấu trúc dữ liệu	Khâu tiếp nhận bản tin của MQTT Bridge	Chặn bản tin sai cấu trúc và ghi nhận nguyên nhân lỗi
S-07	RFC 7231	HTTP/REST API của máy chủ ứng dụng	Đối chiếu phương thức/mã trạng thái với quy ước endpoint
S-08	Quy ước lỗi REST	HTTP/REST API của máy chủ ứng dụng + giao diện web	Soát định dạng lỗi nhất quán trên các nhóm endpoint chính
S-09	Đặc quyền tối thiểu	Tài khoản quản trị hệ thống	Kiểm tra phân quyền theo vai trò, không cấp quyền dư thừa
S-10	Kiểm soát phiên	Dịch vụ xác thực	Kiểm tra thời gian hết hạn phiên và cơ chế thu hồi phiên bất thường
S-11	Ghi nhật ký kiểm toán	Máy chủ ứng dụng + cổng truy cập	Xác minh thao tác nhạy cảm đều có nhật ký truy vết
S-12	Chính sách lưu giữ dữ liệu	Kho số đo/nhật ký	Kiểm tra chính sách lưu trữ theo thời gian và dung lượng

ID	Chuẩn/nguyên tắc	Thành phần áp dụng	Cách kiểm chứng trong vận hành
			giới hạn
S-13	Chính sách sao lưu	PostgreSQL + kho cấu hình	Kiểm tra lịch sao lưu và phục hồi thử nghiệm định kỳ
S-14	Kiểm soát thay đổi	Quy trình phát hành	Mỗi bản phát hành có danh sách kiểm tra, người duyệt và biên bản thay đổi
S-15	Bảo vệ đường truyền	Kênh truy cập quản trị	Kiểm tra cấu hình TLS và chặn truy cập không an toàn
S-16	Xử lý bí mật	Biến môi trường dịch vụ	Kiểm tra không ghi cứng khóa bí mật trong mã nguồn
S-17	Mục tiêu sẵn sàng	Các dịch vụ lỗi	Theo dõi tỷ lệ sẵn sàng và thời gian gián đoạn theo tuần
S-18	Mức quan sát tối thiểu	Grafana + nhật ký + cảnh báo	Kiểm tra đủ số đo/nhật ký/cảnh báo cho các sự cố điển hình

PHỤ LỤC 3: KẾ HOẠCH THỰC HIỆN - ASSIGNMENT AND TIMELINES

3.1. Phân chia giai đoạn dự án

Kế hoạch được chia thành ba giai đoạn: thiết kế và dựng nền tảng, tích hợp và kiểm thử, hoàn thiện và đánh giá.

3.2. Kế hoạch thực hiện dự án theo giai đoạn

Mỗi giai đoạn có đầu ra kỹ thuật rõ: phiên bản nguyên mẫu, phiên bản tích hợp liên tầng và phiên bản đánh giá cuối kỳ.

3.3. Chi tiết công việc theo giai đoạn

Chi tiết công việc được chuẩn hóa theo tuần để theo dõi tiến độ và kiểm soát rủi ro phát sinh.

Milestone	Đầu ra chính	Tiêu chí nghiệm thu	Rủi ro theo dõi
M1: Thiết kế nền tảng	Kiến trúc và BOM chốt	Thiết kế qua rà soát kỹ thuật nội bộ	Sai lệch linh kiện/chi phí
M2: Dựng nguyên mẫu	Thiết bị chạy mẫu	Thu thập được dữ liệu vận hành cốt lõi	Nhiều nguồn và reset cục bộ
M3: Tích hợp máy chủ	Luồng dữ liệu đầu-cuối	Có dữ liệu thời gian thực trên giao diện quản lý	Mất kết nối mạng di động
M4: Kiểm thử liên tầng	Báo cáo kết quả thử nghiệm	Đạt chỉ số chính theo mục tiêu đề án	Sai lệch giữa lab và thực địa
M5: Hoàn thiện báo cáo	Bản rút gọn + tài liệu hỗ trợ	Trình bày đủ logic thiết kế – triển khai – đánh giá	Thiếu nhất quán giữa số liệu và mô tả

3.4. Kế hoạch thực hiện theo tuần (24 tuần)

Tuần	Mục tiêu chính	Công việc trọng tâm	Đầu ra kiểm chứng
1	Khởi động dự án	Chốt phạm vi kỹ thuật, tiêu chí nghiệm thu ban đầu	Biên bản phạm vi + danh sách rủi ro ban đầu
2	Khảo sát phần cứng	So sánh MCU/modem/IMU và khả năng tích hợp	Bảng so sánh lựa chọn linh kiện
3	Chốt kiến trúc hệ thống	Thiết kế luồng dữ liệu thiết bị – máy chủ – giao diện quản lý	Sơ đồ kiến trúc cấp hệ thống

Tuần	Mục tiêu chính	Công việc trọng tâm	Đầu ra kiểm chứng
4	Thiết kế nguồn	Mô phỏng nhánh nguồn chính/dự phòng và ngưỡng bảo vệ bằng ADC	Bộ thông số nguồn và ngưỡng ADC bảo vệ
5	Thiết kế bo mạch	Bố trí linh kiện, tối ưu nhánh nguồn và giao tiếp	Bản vẽ PCB nguyên mẫu
6	Đặt vật tư	Chốt BOM, đặt hàng linh kiện và phụ kiện lắp đặt	Danh sách vật tư đã xác nhận
7	Dựng firmware nền	Khởi tạo dự án, bộ lập lịch task , nhật ký cơ bản	Bản nền firmware chạy ổn định
8	Tích hợp BLE OBD2	Quét/kết nối bộ chuyển đổi, đọc PID cốt lõi	Nhật ký đọc dữ liệu OBD2
9	Tích hợp modem LTE/GNSS	Khởi tạo modem, gửi bản tin test và lấy tọa độ	Bản tin MQTT + vị trí mẫu
10	Chuẩn hóa dữ liệu vận hành	Thiết kế cấu trúc dữ liệu bản tin và quy tắc topic	Bộ cấu trúc dữ liệu và quy ước topic
11	Dựng MQTT broker và MQTT Bridge	Triển khai EMQX và khâu tiếp nhận bản tin của MQTT Bridge	Luồng nhận bản tin đầu-cuối
12	Dựng máy chủ ứng dụng	Tạo API lõi cho giao diện quản lý và thiết bị	Endpoint cốt lõi hoạt động
13	Dựng giao diện web	Trang tổng quan, trạng thái thiết bị, bản đồ	Giao diện có dữ liệu thời gian thực
14	Tích hợp liên tầng 1	Ghép thiết bị, máy chủ ứng dụng và giao diện web cùng chuỗi xử lý	Bản trình diễn đầu-cuối đầu tiên
15	Tối ưu quản lý nguồn	Hoàn thiện logic ngủ/thức và xử lý mất nguồn	Nhật ký chuyển trạng thái hợp lệ
16	Tích hợp cảnh báo	Luồng geofence/sự kiện bất thường lên giao diện	Cảnh báo xuất hiện theo kịch bản

Tuần	Mục tiêu chính	Công việc trọng tâm	Đầu ra kiểm chứng
17	Triển khai quan sát hệ thống	Bổ sung số đo, nhật ký tập trung, giao diện giám sát	Màn hình quan sát vận hành
18	Kiểm thử phòng lab	Đo các chỉ số chính ở môi trường kiểm soát	Báo cáo đo lường vòng 1
19	Kiểm thử thực địa	Lắp trên xe thử nghiệm, chạy hành trình mẫu	Nhật ký vận hành thực địa
20	Tối ưu hiệu năng	Giảm độ trễ, tăng ổn định kết nối lại và chuỗi xử lý	Báo cáo tối ưu vòng 2
21	Gia cố vận hành	Chuẩn hóa danh sách kiểm tra, sao lưu, quay về cấu hình trước	Bộ sổ tay vận hành chuẩn
22	Tổng hợp kết quả	Đối chiếu mục tiêu thiết kế với kết quả đạt được	Bảng đối chiếu mục tiêu và kết quả
23	Hoàn thiện báo cáo	Viết bản rút gọn và chuẩn hóa phụ lục	Bản thảo gần hoàn chỉnh
24	Rà soát cuối	Soát lỗi tài liệu, biên dịch PDF và chốt nộp	Bản nộp cuối + nhật ký kiểm tra

PHỤ LỤC 4: TÀI LIỆU BỔ TRỢ - SUPPORTING MATERIALS

Phụ lục này tóm tắt các tài liệu bổ trợ cần thiết cho việc kiểm tra và bàn giao nguyên mẫu. Các tài liệu chi tiết như mã nguồn, sơ đồ dữ liệu đầy đủ và tài liệu API được lưu trong kho dự án để bảo đảm khả năng truy xuất khi bảo trì hoặc phát triển tiếp.

4.1. Hồ sơ thiết kế và mã nguồn

Nhóm tài liệu	Nội dung chính	Mục đích sử dụng
Sơ đồ nguyên lý và PCB	Sơ đồ nguồn, ESP32-S3, SIM7600CE-T, LIS3DH, đầu nối và mạch đo nguồn	Đối chiếu khi kiểm tra phần cứng
Firmware thiết bị	Mã điều khiển trạng thái, modem, GNSS, OBD2 BLE, cảm biến và nguồn	Nạp thiết bị, kiểm tra lỗi và phát triển tiếp
Máy chủ	Mã nhận bản tin, lưu dữ liệu, API và cảnh báo	Dựng môi trường thử nghiệm và kiểm chứng dữ liệu
Giao diện web	Màn hình bản đồ, trạng thái xe, lịch sử và cảnh báo	Kiểm tra lớp khai thác của người dùng
Tài liệu kiểm thử	Kịch bản đo nguồn, GNSS, OBD2, cảnh báo và toàn tuyến	Lưu minh chứng nghiệm thu

4.2. Tóm tắt nhóm API sử dụng trong nguyên mẫu

Nhóm API	Chức năng	Ghi chú
Xác thực người dùng	Đăng nhập, đăng xuất và kiểm tra phiên	Phục vụ phân quyền cơ bản
Quản lý xe và thiết bị	Xem danh sách xe, thiết bị và trạng thái hiện tại	Phục vụ màn hình tổng quan
Dữ liệu thời gian thực	Lấy vị trí, trạng thái mới nhất và cảnh báo	Phục vụ giao diện quản lý
Lịch sử hành trình	Truy vấn dữ liệu theo xe và khoảng thời gian	Phục vụ đối chiếu sau chuyến đi
Cấu hình thiết bị	Đọc/ghi chu kỳ gửi, ngưỡng cảnh báo và chế độ nguồn	Phục vụ thử nghiệm và hiệu chỉnh

4.3. Checklist kiểm tra nguyên mẫu

STT	Hạng mục kiểm tra	Kết quả mong muốn
1	Kiểm tra nguồn 12 V/24 V	Thiết bị khởi động ổn định, các nhánh nguồn đúng điện áp
2	Kiểm tra pin dự phòng	Thiết bị duy trì hoạt động khi mất nguồn chính ngắn hạn
3	Kiểm tra GNSS	Có tọa độ và thời điểm cập nhật hợp lệ
4	Kiểm tra OBD2 BLE	Bộ chuyển đổi kết nối được và trả về dữ liệu cơ bản
5	Kiểm tra cảm biến chuyển động	Có cảnh báo khi rung/chuyển động vượt ngưỡng
6	Kiểm tra gửi bản tin	Máy chủ nhận và lưu dữ liệu thiết bị
7	Kiểm tra giao diện web	Vị trí, trạng thái và cảnh báo hiển thị đúng xe
8	Kiểm tra cấu hình	Tham số mới được thiết bị nhận và áp dụng

4.4. Ghi chú vận hành thử nghiệm

Khi lắp thiết bị trên xe thật, cần kiểm tra cầu chì bảo vệ, vị trí lấy nguồn, độ chắc của đầu nối, chất lượng sóng di động và vị trí đặt anten. Sau khi lắp, cần chạy thử xe trong một tuyến ngắn để xác nhận dữ liệu GNSS, OBD2, trạng thái nguồn và cảnh báo đã xuất hiện trên giao diện. Nếu thiết bị mất kết nối hoặc reset, cần kiểm tra lần lượt theo thứ tự: nguồn, modem, firmware, máy chủ và giao diện.